PES UNIVERSITY · BIG DATA PROJECT

Distributed Fraud Monitoring System

Baseline and four stress experiments: evidence report

Kafka → Spark Structured Streaming → Apache Iceberg on HDFS → Streamlit dashboard → verified against ground truth

Baseline	Exp 1 · Executor failure	Exp 2 · Master + driver failure	Exp 3 · Late < 1 min	Exp 4 · Late > 1 min
Run 1	Run 2	Run 3	Run 4	Run 5
PASS	PASS	PASS	PASS	FAIL (expected)

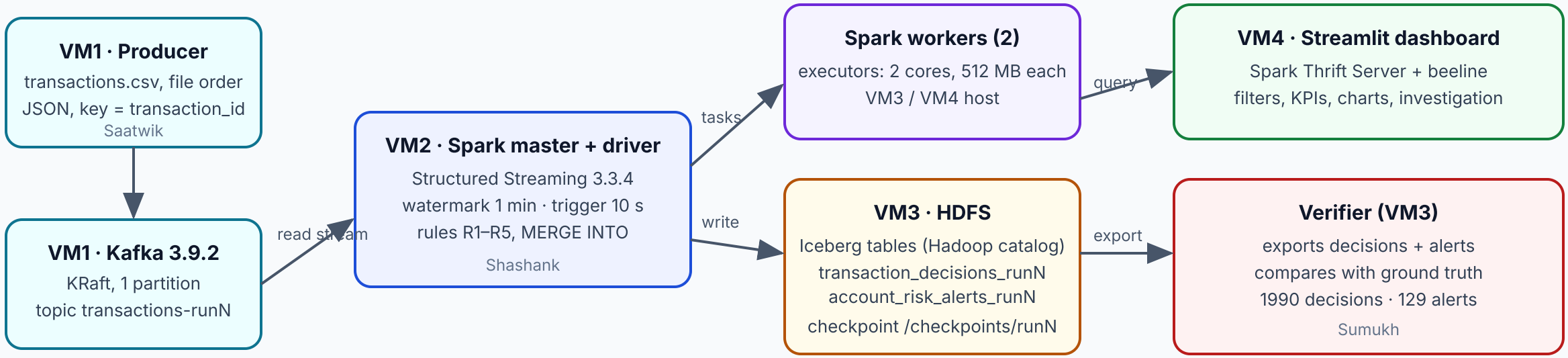
Saatwik
VM1 · Kafka and producer

Shashank
VM2 · Spark master and driver

Sumukh Shandilya
VM3 + VM4 · HDFS, workers, dashboard, verifier

12 November 2026 · all times IST (UTC+05:30) unless marked UTC

System and test set-up



DATA

2,000 input rows, 1,990 unique transaction ids, 10 duplicate rows. Supplied ground truth: 1,990 decisions (1,659 approve, 231 review, 100 block) and 129 account-risk alerts (R2 90, R4 25, R3 14).

PROCESSING

Rules R1–R5, 2-minute activity windows sliding every 30 s, 1-minute lateness (watermark), duplicate removal. Output written to Iceberg with MERGE INTO so a replayed batch cannot create duplicates. State checkpointed on HDFS.

CLUSTER

Spark 3.3.4 standalone: 1 master, 2 workers, 2 executors (2 cores, 512 MB each). Iceberg runtime 1.4.1 on HDFS, trigger 10 s. Every run uses its own Kafka topic, checkpoint and Iceberg tables named by run number.

Results at a glance

RUN	PURPOSE	FAULT OR CONDITION	DECISIONS actual / expected	ALERTS actual / expected	MISSING	VERIFIER	NOTE
Run 6	Baseline, first attempt	none	1,990 / 1,990	FAILED: 129 alerts with window times 5 h 30 min early	—	FAIL	Timezone bug found, fixed
Run 1	Baseline	none	1,990 / 1,990	129 / 129	0	PASS	Reference run
Run 2	Exp 1 · Executor failure	kill -9 one executor mid-stream	1,990 / 1,990	129 / 129	0	PASS	Replacement executor in 2 s, no manual step
Run 3	Exp 2 · Master + driver failure	stop master and driver mid-batch	1,990 / 1,990	129 / 129	0	PASS	Resumed from checkpoint, no duplicates, manual restart ≈ 2 min 48 s
Run 4	Exp 3 · Late under 60 s	7 rows 30–48 s late	1,990 / 1,990	129 / 129	0	PASS	Late rows accepted
Run 5	Exp 4 · Late over 60 s	7 rows 93–129 s late	1,983 / 1,990	129 / 129	7	FAIL (expected)	The 7 missing = the 7 late rows

WHAT HELD

With no fault the output equals the ground truth exactly. An executor failure, and a master and driver failure, changed nothing in the output. Lateness inside the 1-minute watermark is handled.

WHAT DID NOT

Rows more than 60 s late are dropped: 7 of 1,990 decisions (0.35 %) in run 5. No alert was affected in this data set. Recovery from a master and driver failure needs a manual restart.

HOW WE KNOW

Every run is checked by the same independent verifier against the supplied ground truth, plus the producer log, driver log, Spark UI, HDFS listing and the dashboard.

How each run was executed

1	**Fresh state**	New Kafka topic transactions-runN, new HDFS checkpoint /checkpoints/runN and new Iceberg tables for run N.
2	**Start the job**	Shashank submits FraudDetectionEngine-runN to the standalone cluster before the producer starts.
3	**Stream the data**	Saatwik runs the producer. Baseline mode sends the file in order and never sorts it. Lateness modes hold selected rows back.
4	**Inject the fault**	Executor kill, master and driver stop, or late rows, depending on the experiment.
5	**Check Kafka**	verify_topic.py proves the topic holds exactly the input rows.
6	**Verify the output**	Sumukh exports both Iceberg tables and runs the verifier against the ground truth. The verifier is the hard pass or fail gate.
7	**Collect evidence**	Screenshots, logs, HDFS listings and verifier JSON are saved per experiment with IP addresses redacted.

READING THE FIGURES

Each page shows one screenshot (or a few from the same moment), what it shows, and why it matters. The tag at the top right says who captured it and on which VM.

TIMES

Spark driver logs are in IST. Kafka, HDFS and kill commands on VM1 and VM3 print UTC (IST minus 5 h 30 min). This document converts to IST.

RUN NUMBERS

Run 1 is the official baseline. Run 6 was the first end-to-end attempt; it exposed a timezone bug and is kept as evidence. Runs 2 to 5 are experiments 1 to 4.

BASELINE

Baseline

Run 1 (official) · Run 6 (first attempt)

Baseline: the system does what the specification says

OBJECTIVE

Show that Kafka, Spark Structured Streaming, Iceberg on HDFS and the dashboard produce exactly the ground-truth decisions and alerts when nothing goes wrong, and that the verifier catches defects.

WHAT HAPPENED (IST)

13:54	Run 6 · producer sends 2,000 messages (104.6 s) into transactions-run6
19:14	Run 6 · Spark job starts and handles all 1,990 unique records in one batch
after	Run 6 · verifier: decisions PASS, alerts FAIL. Window times are exactly 5 h 30 min early (timezone bug). Fixed in stream_job.py.
20:20	Run 1 · Spark job FraudDetectionEngine-run1 starts
20:26	Run 1 · producer streams the file (104.1 s)
20:27–20:29	Run 1 · batches 1 to 6, last one at 20:29:42, 77 s after the last message
after	Run 1 · verifier PASS, exit code 0

VERDICT

Run 1: PASS **Run 6: FAIL, defect fixed**

KEY NUMBERS

Run 1 decisions	**1,990 / 1,990**
Run 1 alerts	**129 / 129**
Missing / extra / mismatched	**0 / 0 / 0**
Duplicate decision ids	**0**
Longest batch	**40.2 s**
Checkpoint commits	**0–7**

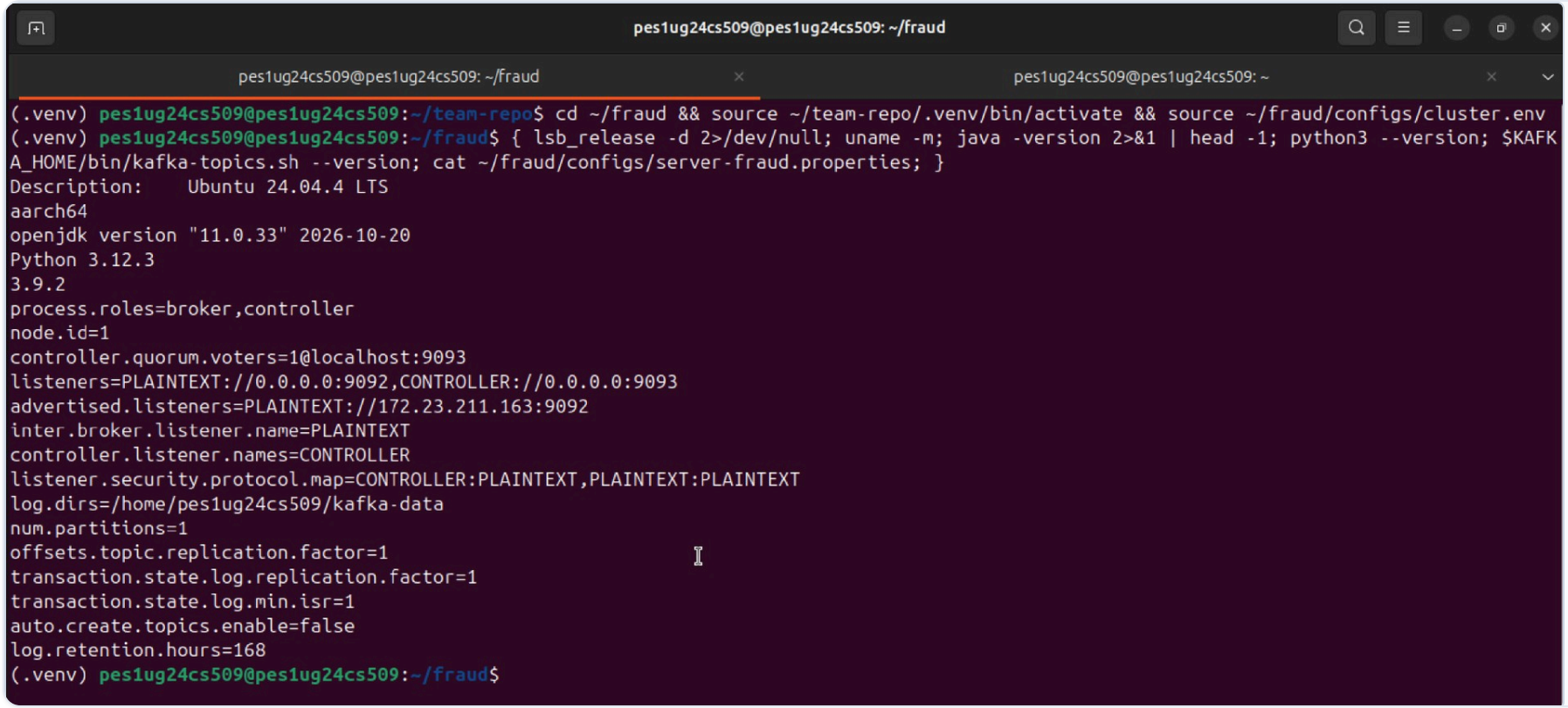
WHAT IT DEMONSTRATES

The pipeline is correct end to end. The first attempt (run 6) shows the verifier doing its job: a real defect in the alert window times, easy to miss by eye, was found and fixed before the experiments.

BASELINE

VM1 environment and Kafka broker configuration

Fig 1 · Saatwik · VM1 (Kafka + producer)



WHAT YOU ARE LOOKING AT

Ubuntu 24.04 (aarch64), OpenJDK 11, Python 3.12.3 and Kafka 3.9.2. The broker runs in KRaft mode (one node acting as broker and controller), listens on the ZeroTier address so the other VMs can reach it, has 1 partition per topic, replication factor 1, retention 168 h, and automatic topic creation switched off.

WHY IT MATTERS

Topics must be created on purpose. That is how each run gets its own fresh topic and cannot see data from an earlier run.

BASELINE

Run 6: fresh topic created and checked

Fig 2 · Saatwik · VM1 (Kafka + producer)

```
log.retention.hours=168
(.venv) pes1ug24cs509@pes1ug24cs509:~/fraud$ $KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server $KAFKA_BOOTSTRAP --describe --topic transac
tions-run6
$KAFKA_HOME/bin/kafka-get-offsets.sh --bootstrap-server $KAFKA_BOOTSTRAP --topic transactions-run6
Topic: transactions-run6        TopicId: 9ncO3DAxSDqfj0WF-4KAhQ PartitionCount: 1       ReplicationFactor: 1    Configs:
        Topic: transactions-run6        Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
transactions-run6:0:2000
(.venv) pes1ug24cs509@pes1ug24cs509:~/fraud$
```

```
(.venv) pes1ug24cs509@pes1ug24cs509:~/fraud$ cd ~/team-repo && python3 vm1/verify_topic.py --topic transactions-run6 --input data/transacti
ons.csv --expect-order
csv_rows                        2000
topic_messages                  2000
missing_from_topic              0
extra_in_topic                  0
same_content_ignoring_order     True
same_order_as_file              True
key_equals_transaction_id       True
duplicate_ids_kept              True
all_values_are_strings          True
RESULT: PASS
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$
```

WHAT YOU ARE LOOKING AT

Topic transactions-run6 is described (1 partition, replica 1, in-sync) and its end offset is 2000. The topic verifier then compares the topic with data/transactions.csv: 2000 rows in the file, 2000 messages in the topic, none missing, none extra, same order as the file, key equal to transaction_id, duplicates kept, all values strings. RESULT: PASS.

WHY IT MATTERS

Proves the producer delivered the input unchanged, including the 10 duplicate rows. Anything the pipeline later gets wrong is therefore not a producer problem.

BASELINE

Run 6: what a Kafka message looks like

Fig 3 · Saatwik · VM1 (Kafka + producer)

```
RESULT: PASS
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$ $KAFKA_HOME/bin/kafka-console-consumer.sh --bootstrap-server $KAFKA_BOOTSTRAP --topic tran
sactions-run6 --from-beginning --max-messages 3
{"event_id": "E0000001", "transaction_id": "TXN0000001", "user_id": "U100133", "customer_id": "CUST2000133", "source_account": "5793790505"
, "destination_account": "9635212070", "merchant_id": "MRC10504", "device_id": "d13632673", "event_timestamp": "2026-10-27T08:00:02Z", "amo
unt": "1003.00", "currency": "INR", "transaction_type": "PAYMENT", "location": "Mumbai", "channel": "UPI", "account_age_days": "3092"}
{"event_id": "E0000002", "transaction_id": "TXN0000002", "user_id": "U100124", "customer_id": "CUST2000124", "source_account": "5649380713"
, "destination_account": "9252914319", "merchant_id": "MRC10728", "device_id": "d533e577a", "event_timestamp": "2026-10-27T08:00:29Z", "amo
unt": "540.00", "currency": "INR", "transaction_type": "PAYMENT", "location": "Bengaluru", "channel": "UPI", "account_age_days": "2227"}
{"event_id": "E0000003", "transaction_id": "TXN0000003", "user_id": "U100018", "customer_id": "CUST2000018", "source_account": "5845243862"
, "destination_account": "9933838458", "merchant_id": "MRC10105", "device_id": "d10dc5cce", "event_timestamp": "2026-10-27T08:01:50Z", "amo
unt": "455.28", "currency": "INR", "transaction_type": "PAYMENT", "location": "Kolkata", "channel": "UPI", "account_age_days": "1377"}
Processed a total of 3 messages
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$
```

```
transactions-run1:0:0
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$ cat ~/fraud/logs/run6_baseline_summary.json
{
  "run_id": "run6",
  "mode": "baseline",
  "topic": "transactions-run6",
  "bootstrap": "172.23.211.163:9092",
  "input": "data/transactions.csv",
  "input_sha256": "980dd52388d1c4088e4806abd9efd70cb407ed0e9421113f3b25c3095b33d1a3",
  "host": "pes1ug24cs509",
  "dry_run": false,
  "rows_in_file": 2000,
  "unique_transaction_ids": 1990,
  "messages_sent": 2000,
  "broker_acks": 2000,
  "send_errors": [],
  "n_errors": 0,
  "seed": 42,
  "fraction": 0.02,
  "target_event_lag_s": null,
  "hard_cap_s": null,
  "send_delay_s": 0.05,
  "min_wall_hold_s": 0.0,
  "injected": 0,
  "injected_actually_late": 0,
  "injected_lag_gt_60s": 0,
  "all_rows_behind_running_max": 0,
  "all_rows_lag_gt_60s": 0,
  "start_utc": "2026-11-12T08:24:24.584+00:00",
  "end_utc": "2026-11-12T08:26:09.156+00:00",
  "duration_s": 104.6
}(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$
```

WHAT YOU ARE LOOKING AT

Top: the first three messages read with the console consumer. One CSV row is one JSON message and every value is a string, as the project requires. Bottom: the producer summary file for run 6. 2000 messages sent, 2000 broker acknowledgements, 0 errors, 104.6 s at 0.05 s per message, no late rows injected. The input SHA-256 is recorded.

WHY IT MATTERS

The SHA-256 ties every run to the exact same input file. The acknowledgement count equals the sent count, so nothing was lost at the producer.

BASELINE

Run 6: producer output files

Fig 4 · Saatwik · VM1 (Kafka + producer)



WHAT YOU ARE LOOKING AT

Is of the log folder after the run: run6_baseline_summary.json and run6_baseline_arrival.csv, next to the earlier test, dry and rehearsal folders.

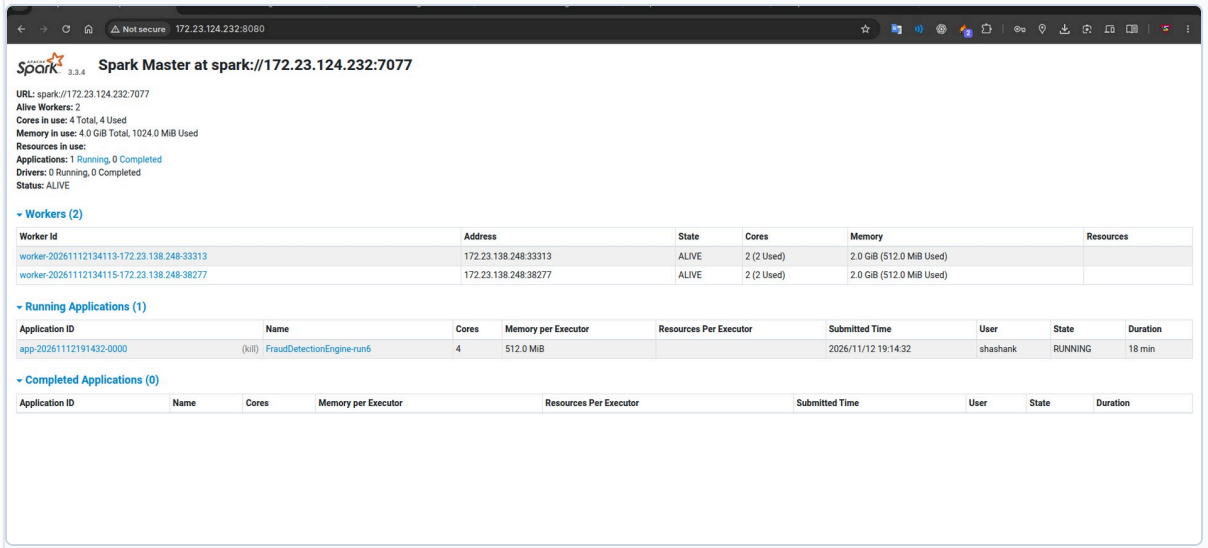
WHY IT MATTERS

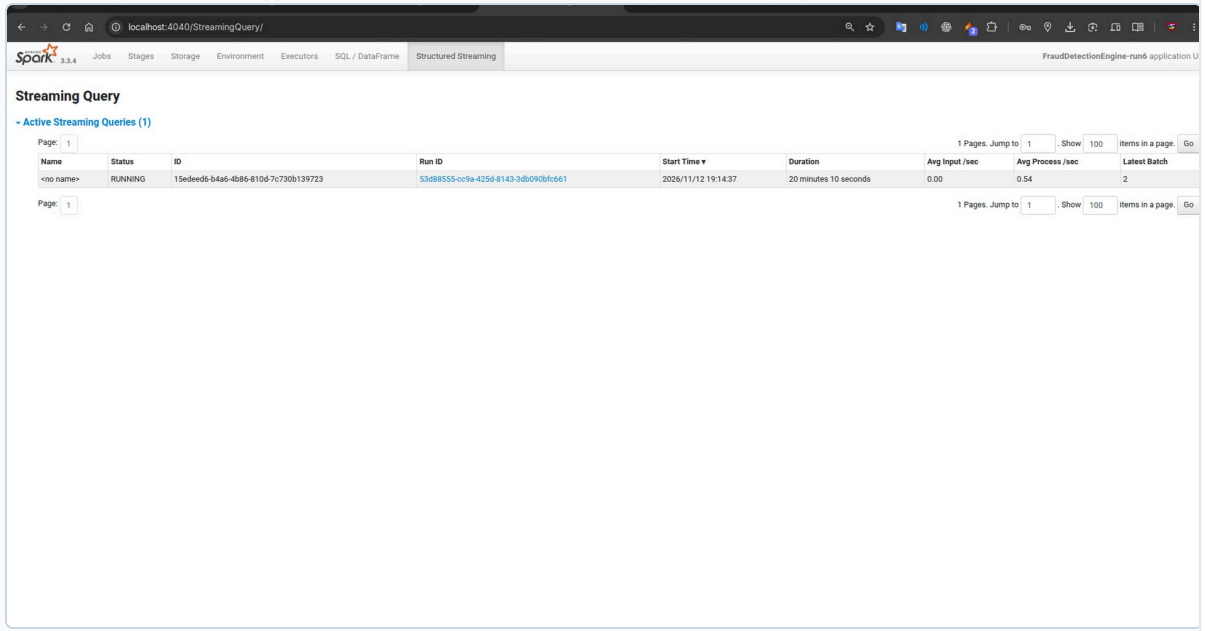
Each run leaves a summary and a per-message arrival log that are archived as evidence.

BASELINE

Run 6: Spark cluster and application

Fig 5 · Shashank · VM2 (Spark master + driver)





WHAT YOU ARE LOOKING AT

Spark master at spark://...:7077 with 2 workers alive (4 cores, 4 in use). One running application, FraudDetectionEngine-run6, started 19:14:32 IST with 4 cores and 512 MB per executor. The Structured Streaming tab shows the single active query.

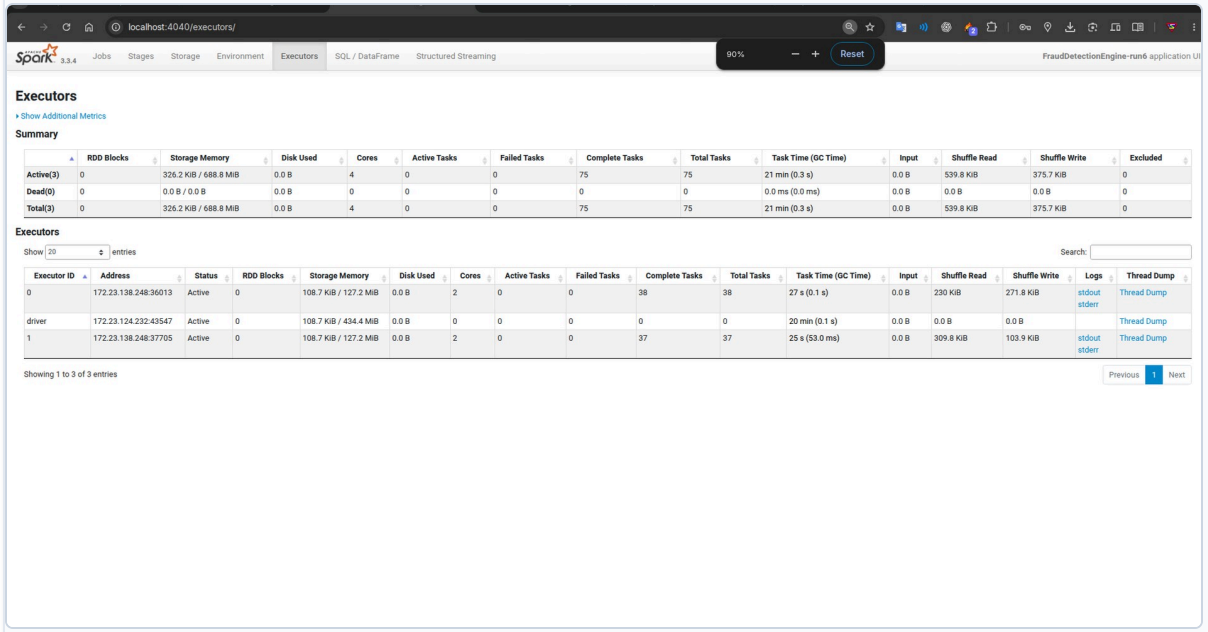
WHY IT MATTERS

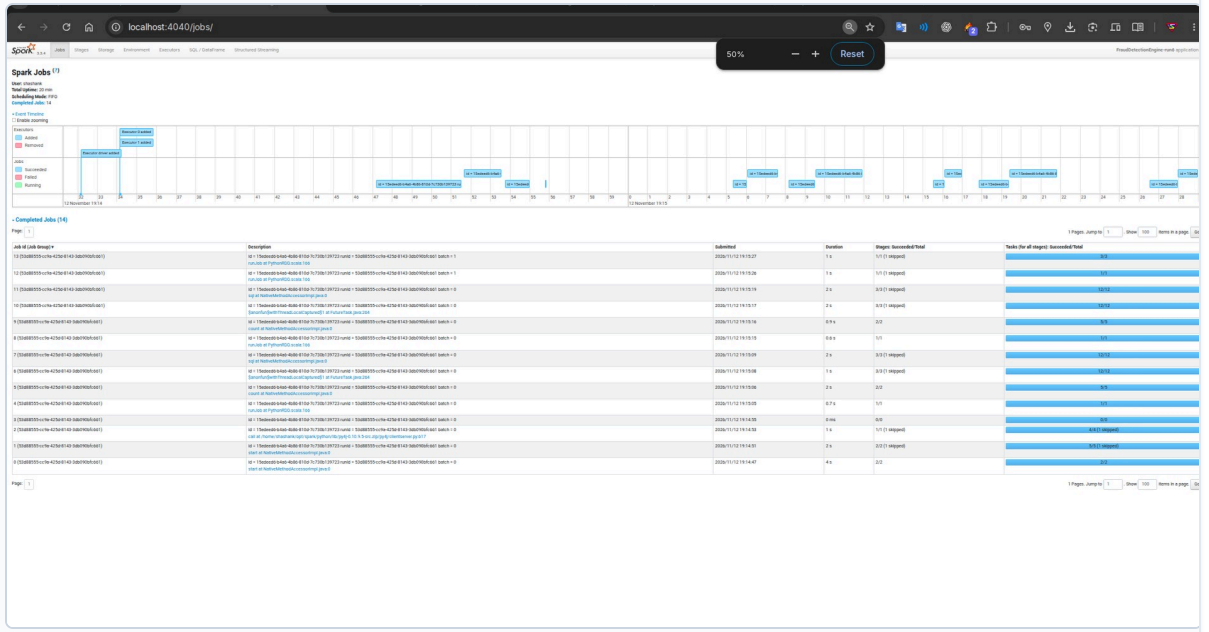
Shows the job really runs on the standalone cluster (master, 2 workers, 2 executors), not in local mode.

BASELINE

Run 6: executors and jobs

Fig 6 · Shashank · VM2 (Spark master + driver)





WHAT YOU ARE LOOKING AT

Executors tab: two active executors plus the driver, 75 completed tasks, no failed tasks. Jobs tab: the timeline of Spark jobs triggered by the stream.

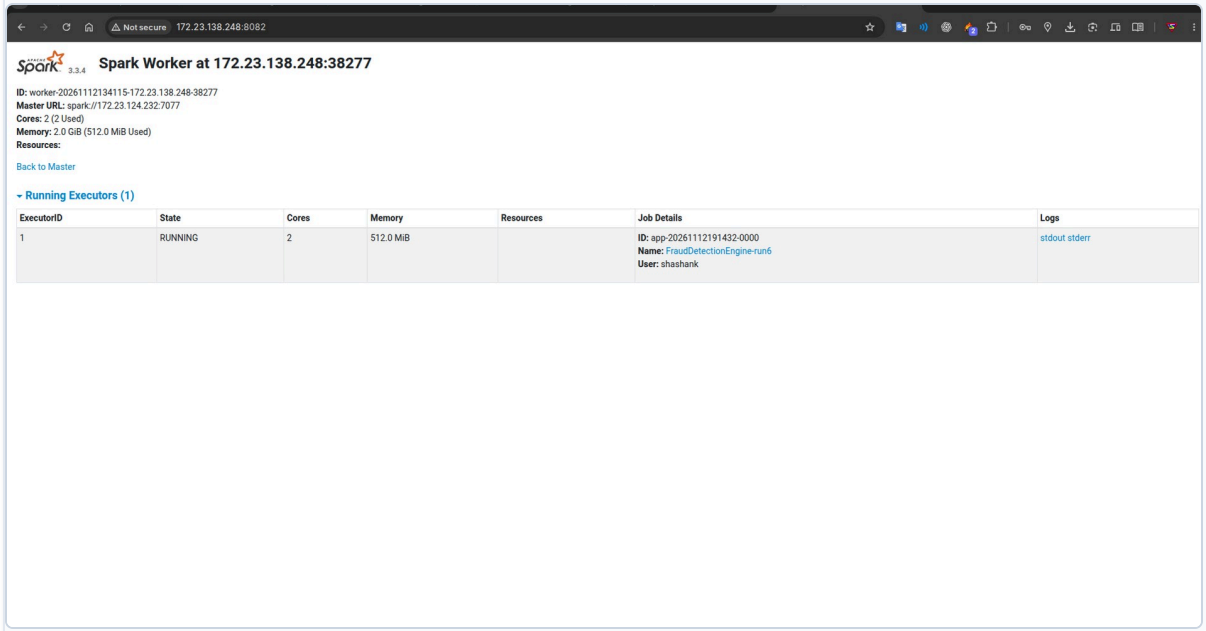
WHY IT MATTERS

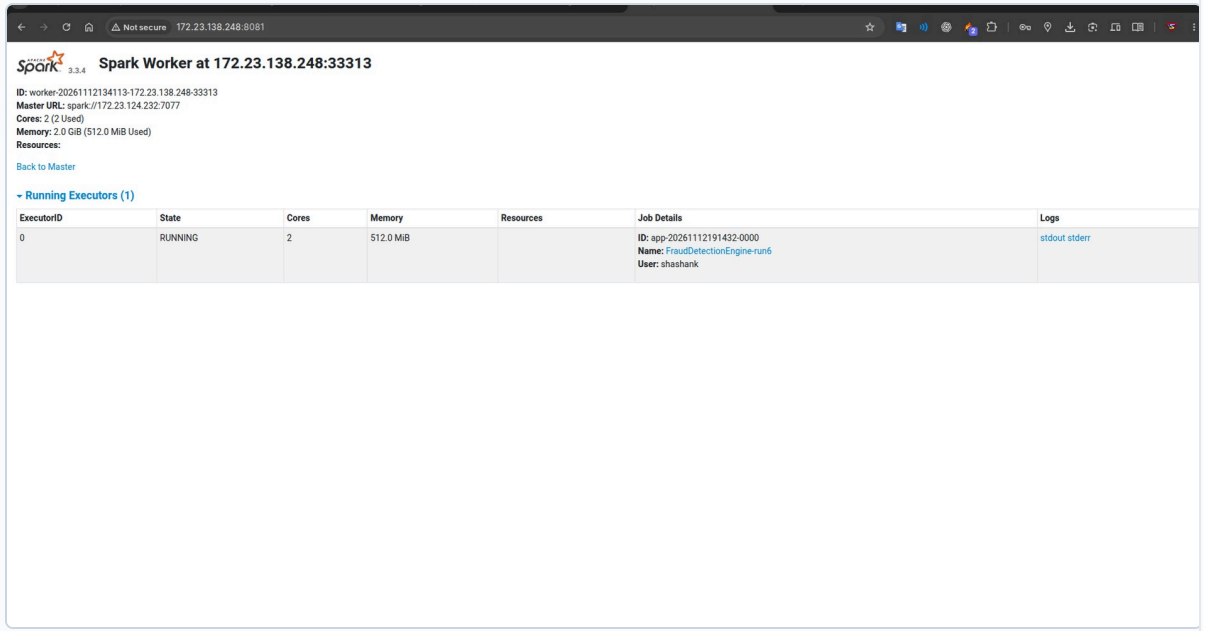
Baseline view of a healthy cluster, used later as the reference for the failure experiments.

BASELINE

Run 6: workers register their executors

Fig 7 · Shashank · VM2 (Spark master + driver)





WHAT YOU ARE LOOKING AT

The two worker pages. Each worker runs one executor (2 cores, 512 MB) for the application.

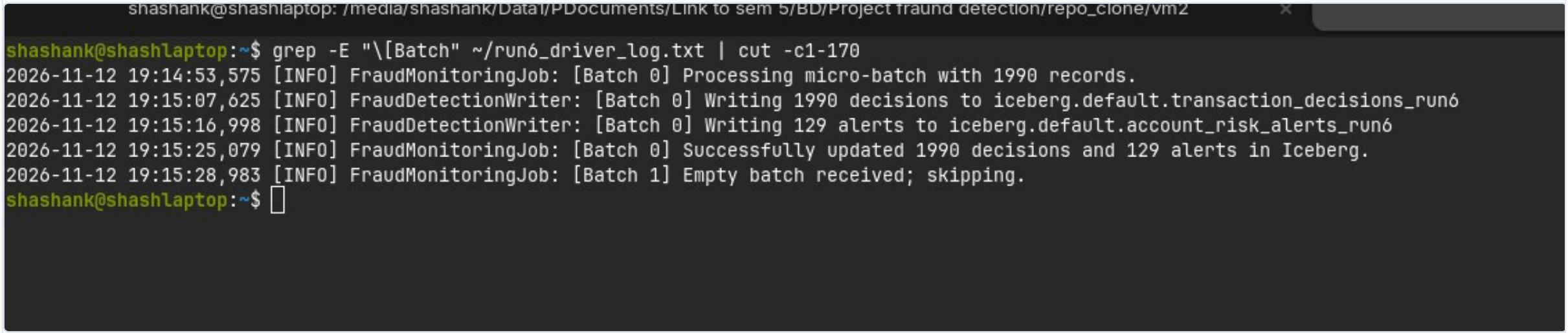
WHY IT MATTERS

Confirms the work is split over both workers on the VM3/VM4 host.

BASELINE

Run 6: the single big batch

Fig 8 · Shashank · VM2 (Spark master + driver)



WHAT YOU ARE LOOKING AT

Driver log: Batch 0 processes 1990 records (the 2000 messages minus 10 duplicates), writes 1990 decisions and 129 alerts to Iceberg and finishes at 19:15:25 IST. Batch 1 is empty and skipped.

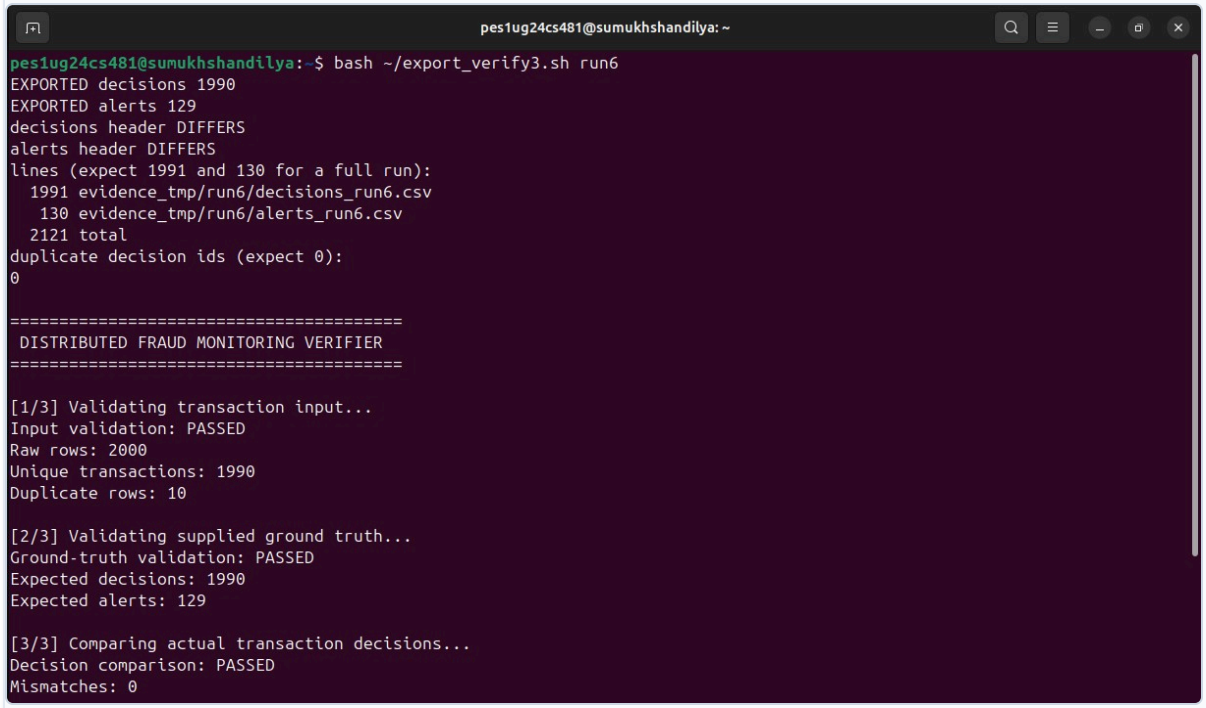
WHY IT MATTERS

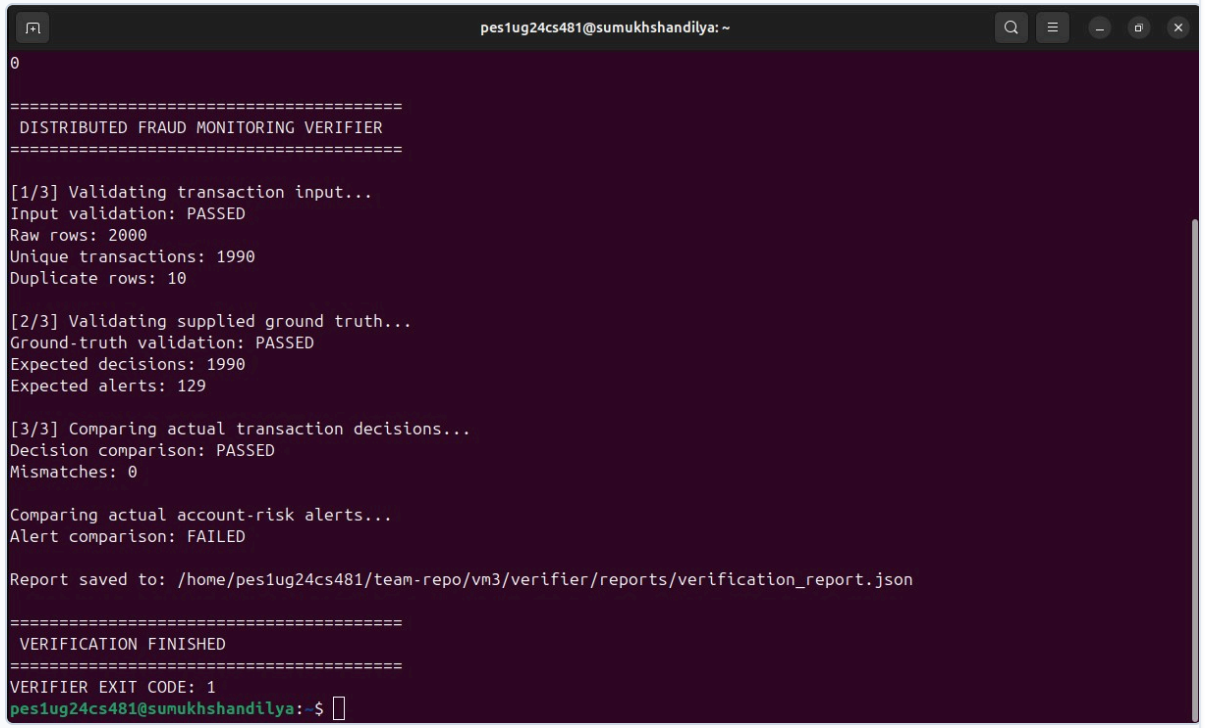
Run 6 was started after the producer had finished, so the whole topic arrived as one batch (32 s). The streaming code path is the same as in the later runs.

BASELINE

Run 6: verifier fails on the alerts

Fig 9 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)





WHAT YOU ARE LOOKING AT

Verifier output for run 6. Input validation and ground-truth validation pass. Decision comparison passes with 0 mismatches. Alert comparison FAILS and the verifier exits with code 1.

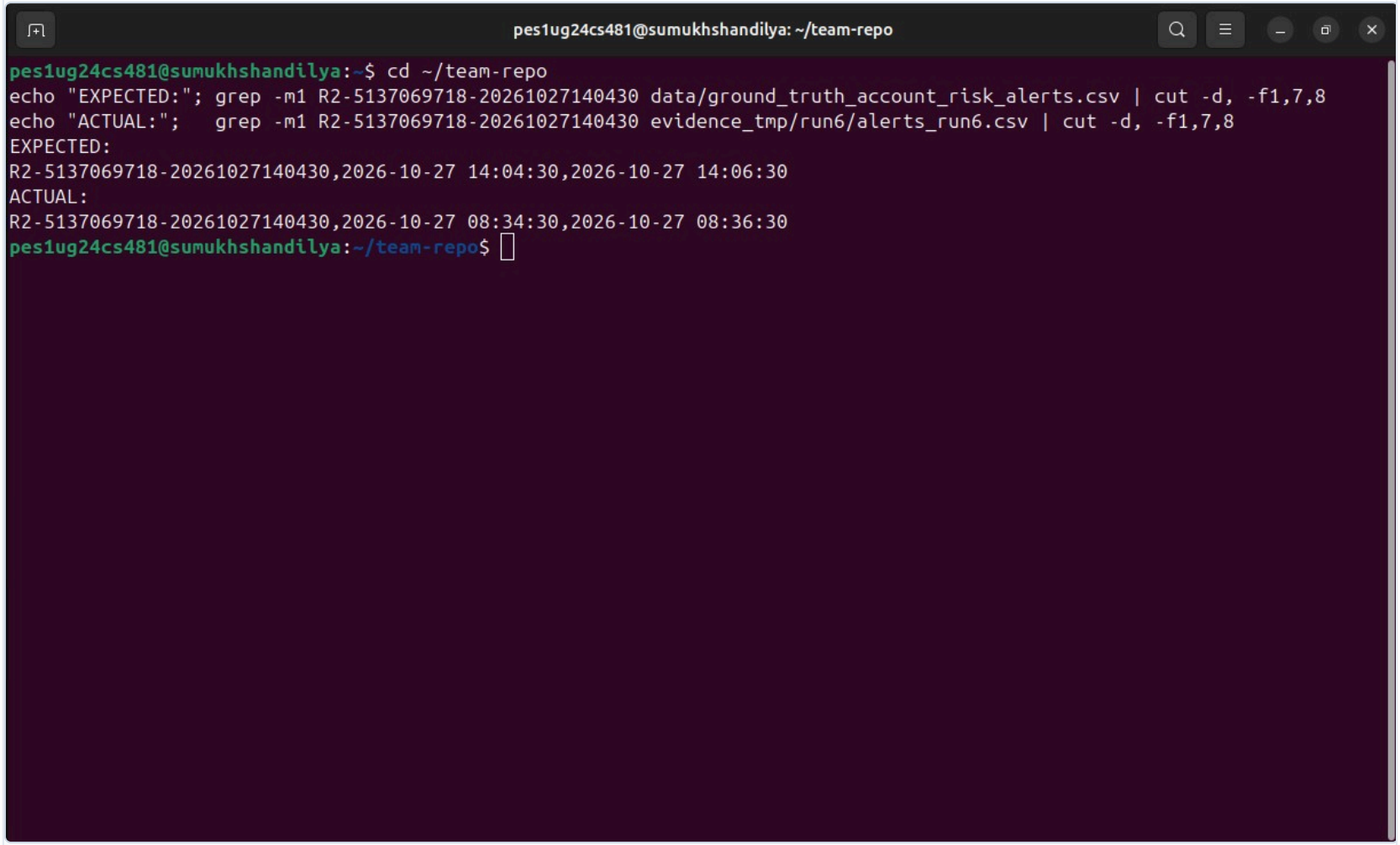
WHY IT MATTERS

The verifier is the correctness gate and it caught a real defect. Decisions were right but the alerts were not.

BASELINE

Run 6: root cause is a timezone shift

Fig 10 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)



WHAT YOU ARE LOOKING AT

The same alert, R2-5137069718-20261027140430, in the ground truth and in the output. Expected window: 14:04:30 to 14:06:30. Actual window: 08:34:30 to 08:36:30. The output is exactly 5 h 30 min earlier.

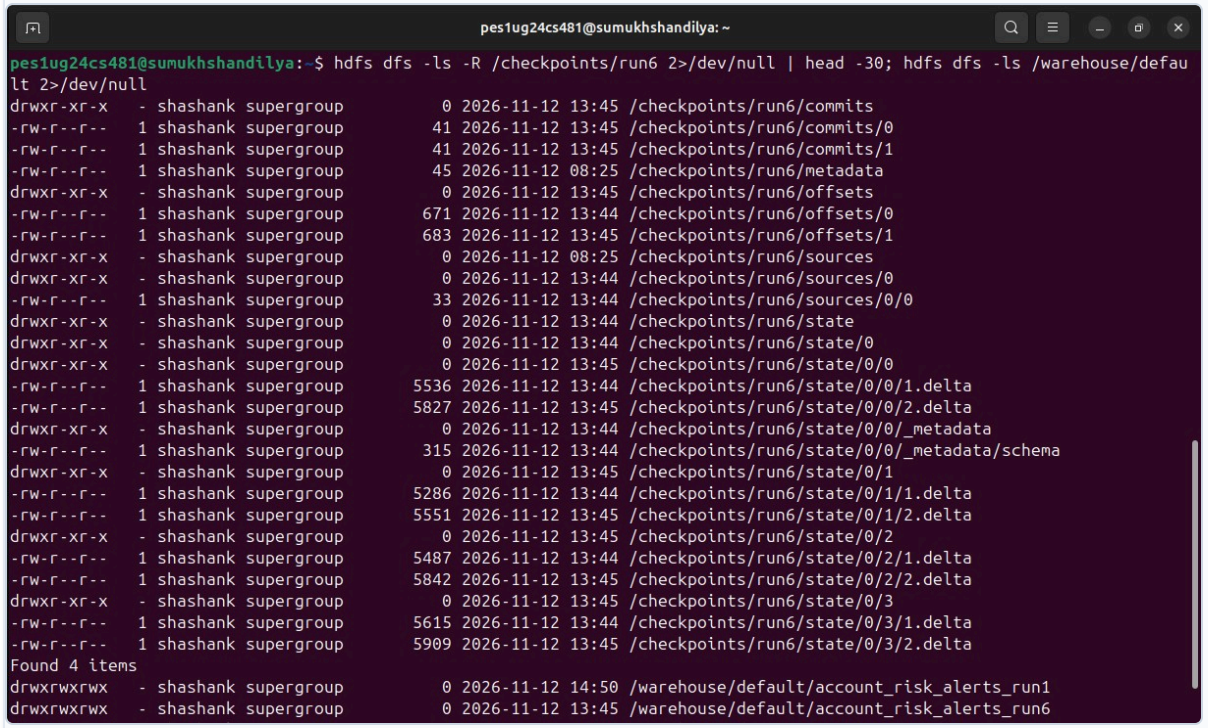
WHY IT MATTERS

A shift of 5 h 30 min is the IST offset. The alert window timestamps were converted with the local timezone of the driver instead of UTC. Fix: attach tzinfo=UTC when the pandas timestamps are converted to Spark timestamps (vm2/app/stream_job.py). Run 1 is the clean re-run after this fix.

BASELINE

Run 6: HDFS checkpoint and Iceberg tables

Fig 11 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)



WHAT YOU ARE LOOKING AT

HDFS listing of /checkpoints/run6 (commits 0 and 1, offsets 0 and 1, sources, and the state store with 4 partitions) and of /warehouse/default (account_risk_alerts and transaction_decisions for runs 1 and 6). The bottom of the second screenshot shows the Thrift/beeline query used to count rows, and free -m: the VM has 7.9 GB of memory with about 2 GB still available.

WHY IT MATTERS

The checkpoint and the Iceberg tables live on HDFS and are named by run number, as required.

BASELINE

Run 1: empty topic created

Fig 12 · Saatwik · VM1 (Kafka + producer)

```
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$ source ~/fraud/configs/cluster.env
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server $KAFKA_BOOTSTRAP --delete --topic transactions-run1; sleep 8
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server $KAFKA_BOOTSTRAP --create --topic transactions-run1 --partitions 1 --replication-factor
1 && echo topic-ready
$KAFKA_HOME/bin/kafka-get-offsets.sh --bootstrap-server $KAFKA_BOOTSTRAP --topic transactions-run1
Created topic transactions-run1.
topic-ready
transactions-run1:0:0
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$
```

WHAT YOU ARE LOOKING AT

Topic transactions-run1 is deleted and recreated. The end offset is 0 (transactions-run1:0:0), so the run starts from nothing.

WHY IT MATTERS

Each run uses a new, empty topic. No earlier data can leak into the result.

BASELINE

Run 1: producer streams the file

Fig 13 · Saatwik · VM1 (Kafka + producer)

```
  "duration_s": 104.6
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$ cd ~/team-repo && source .venv/bin/activate && source ~/fraud/configs/cluster.env && mkdir -p ~/fraud/logs/run11
python3 vm1/producer.py --mode baseline --topic transactions-run1 --run-id run1 --input data/transactions.csv --bootstrap $KAFKA_BOOTSTRAP --log-dir ~/fraud/logs/run1
/home/pes1ug24cs509/team-repo/vm1/producer.py:132: DeprecationWarning: key_serializer does not implement kafka.serializer.Serializer
  if __name__ == '__main__': main()
/home/pes1ug24cs509/team-repo/vm1/producer.py:132: DeprecationWarning: value_serializer does not implement kafka.serializer.Serializer
  if __name__ == '__main__': main()
sent 500/2000
sent 1000/2000
sent 1500/2000
sent 2000/2000
{
  "run_id": "run1",
  "mode": "baseline",
  "topic": "transactions-run1",
  "bootstrap": "172.23.211.163:9092",
  "input": "data/transactions.csv",
  "input_sha256": "980dd52388d1c4088e4806abd9efd70cb407ed0e9421113f3b25c3095b33d1a3",
  "host": "pes1ug24cs509",
  "dry_run": false,
  "rows_in_file": 2000,
  "unique_transaction_ids": 1990,
  "messages_sent": 2000,
  "broker_acks": 2000,
  "send_errors": [],
  "n_errors": 0,
  "seed": 42,
  "fraction": 0.02,
  "target_event_lag_s": null,
  "hard_cap_s": null,
  "send_delay_s": 0.05,
  "min_wall_hold_s": 0.0,
  "injected": 0,
  "injected_actually_late": 0,
  "injected_lag_gt_60s": 0,
  "all_rows_behind_running_max": 0,
  "all_rows_lag_gt_60s": 0,
  "start_utc": "2026-11-12T14:56:41.862+00:00",
  "end_utc": "2026-11-12T14:58:25.947+00:00",
  "duration_s": 104.1
}
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$
```

WHAT YOU ARE LOOKING AT

The baseline producer reads transactions.csv in file order, without sorting, and sends one JSON message per row. Progress 500, 1000, 1500, 2000. Summary: 2000 sent, 2000 acknowledged, 0 errors, 104.1 s, started 14:56:41 UTC.

WHY IT MATTERS

This is the clean baseline stream: no lateness injected (injected = 0).

BASELINE

Run 1: Kafka topic verified

Fig 14 · Saatwik · VM1 (Kafka + producer)

```
  "end_utc": "2026-11-12T14:58:25.947+00:00",
  "duration_s": 104.1
}
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$ source ~/fraud/configs/cluster.env
# 📸 run1_saatwik_topic.png
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server $KAFKA_BOOTSTRAP --describe --topic transactions-run1
$KAFKA_HOME/bin/kafka-get-offsets.sh --bootstrap-server $KAFKA_BOOTSTRAP --topic transactions-run1
# 📸 run1_saatwik_verify_topic.png
cd ~/team-repo && python3 vm1/verify_topic.py --topic transactions-run1 --input data/transactions.csv --expect-order
Topic: transactions-run1        TopicId: 15krrkYXRTKyTsJw7tSJuA PartitionCount: 1       ReplicationFactor: 1    Configs:
        Topic: transactions-run1        Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
transactions-run1:0:2000
csv_rows                        2000
topic_messages                  2000
missing_from_topic              0
extra_in_topic                  0
same_content_ignoring_order     True
same_order_as_file              True
key_equals_transaction_id       True
duplicate_ids_kept              True
all_values_are_strings          True
RESULT: PASS
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$
```

WHAT YOU ARE LOOKING AT

transactions-run1 has end offset 2000. verify_topic.py: 2000 rows = 2000 messages, none missing or extra, same order as the file, duplicates kept. RESULT: PASS.

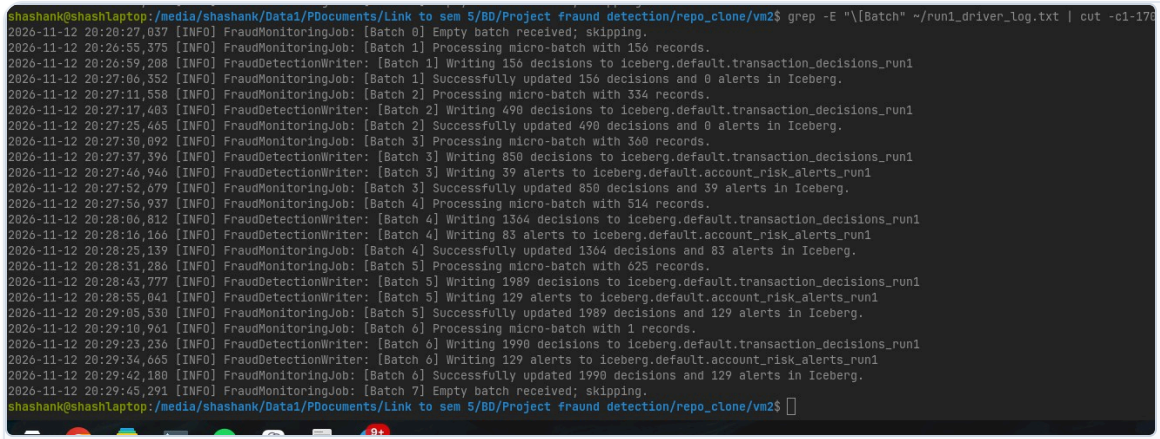
WHY IT MATTERS

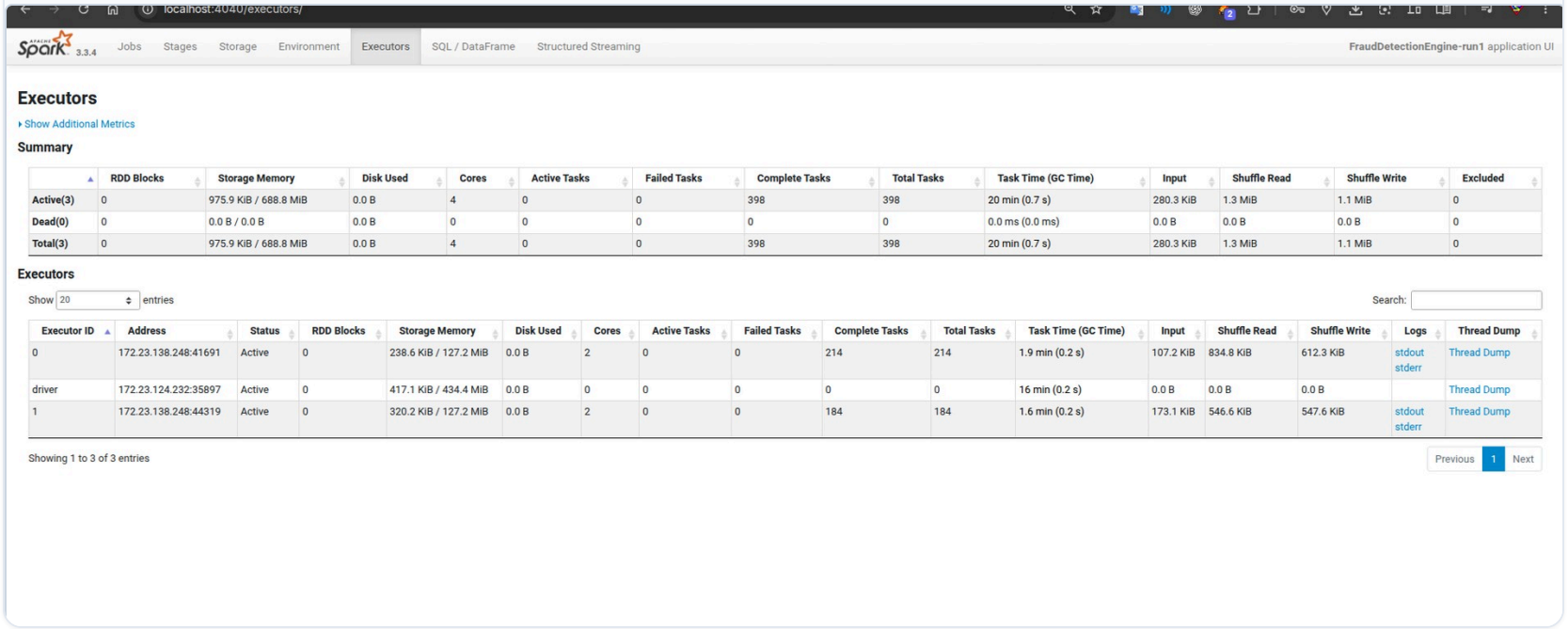
Kafka holds exactly the input, in order, before Spark is judged.

BASELINE

Run 1: Spark processes the stream in micro-batches

Fig 15 · Shashank · VM2 (Spark master + driver)





WHAT YOU ARE LOOKING AT

Driver log for run 1. Batches 1 to 6 arrive as the producer sends. The decision count grows 156, 490, 850, 1364, 1989 and finally 1990, and alerts grow 0, 0, 39, 83, 129, 129. Last batch (6) finishes 20:29:42 IST, about 77 s after the producer stopped. Executors tab: two executors, 398 completed tasks, 0 failed.

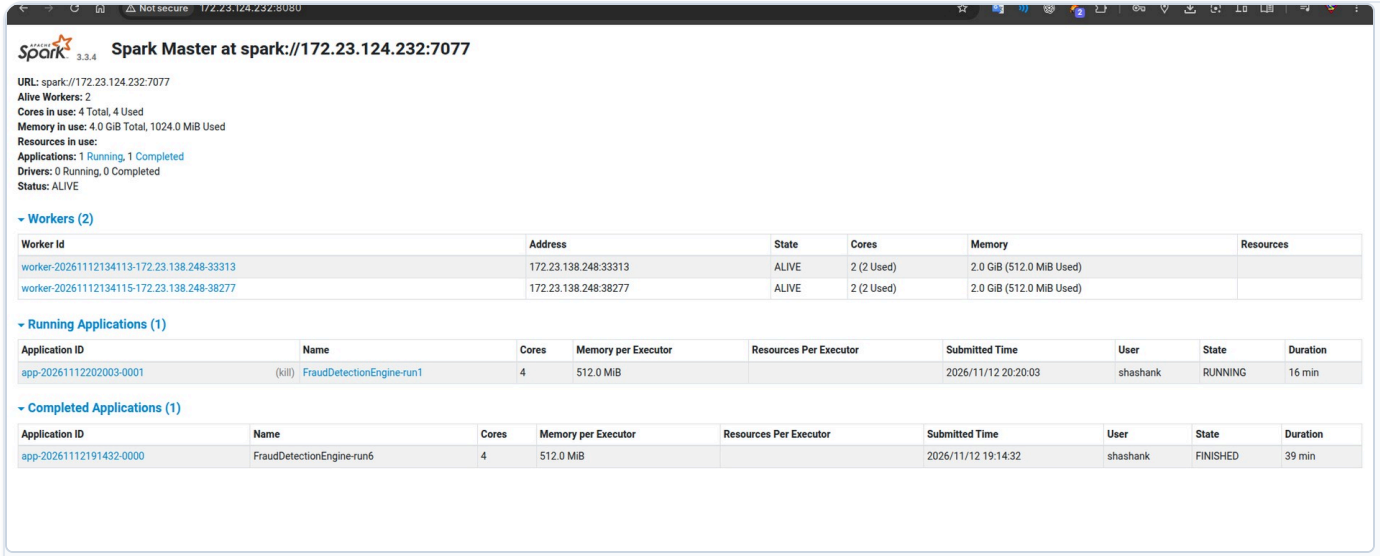
WHY IT MATTERS

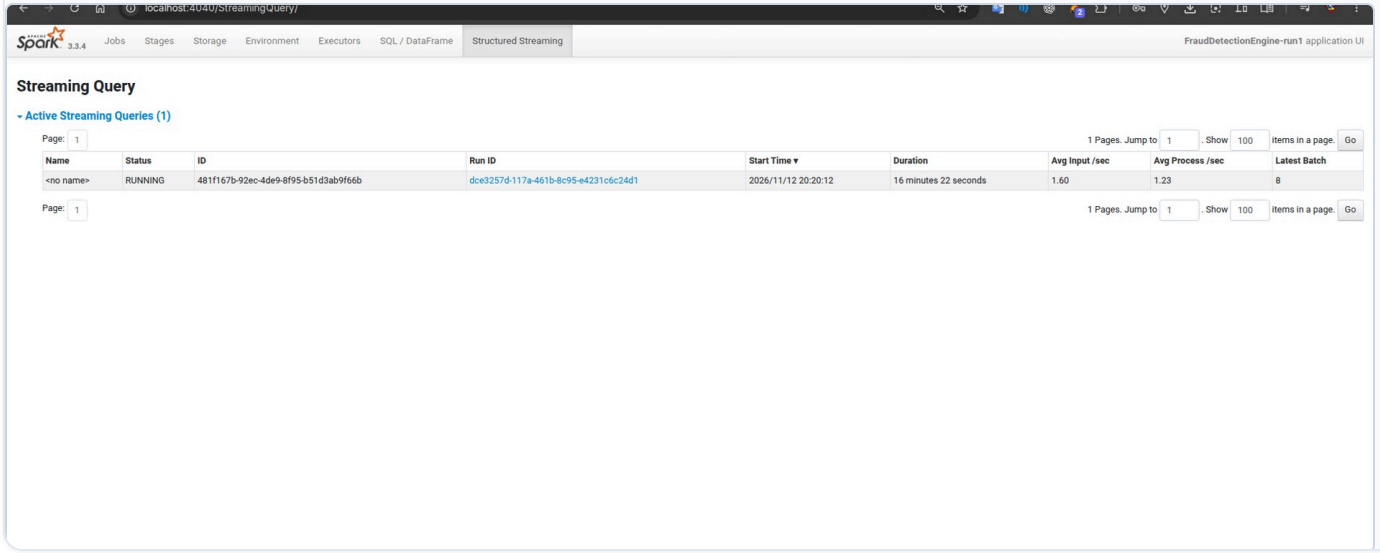
Shows real streaming behaviour: results build up as data arrives and the final state is complete (1990 decisions, 129 alerts).

BASELINE

Run 1: cluster and streaming query

Fig 16 · Shashank · VM2 (Spark master + driver)





WHAT YOU ARE LOOKING AT

Master page: application FraudDetectionEngine-run1 is running; the run 6 application is listed as completed (39 min). Streaming tab: one active query, run for 16 min 22 s.

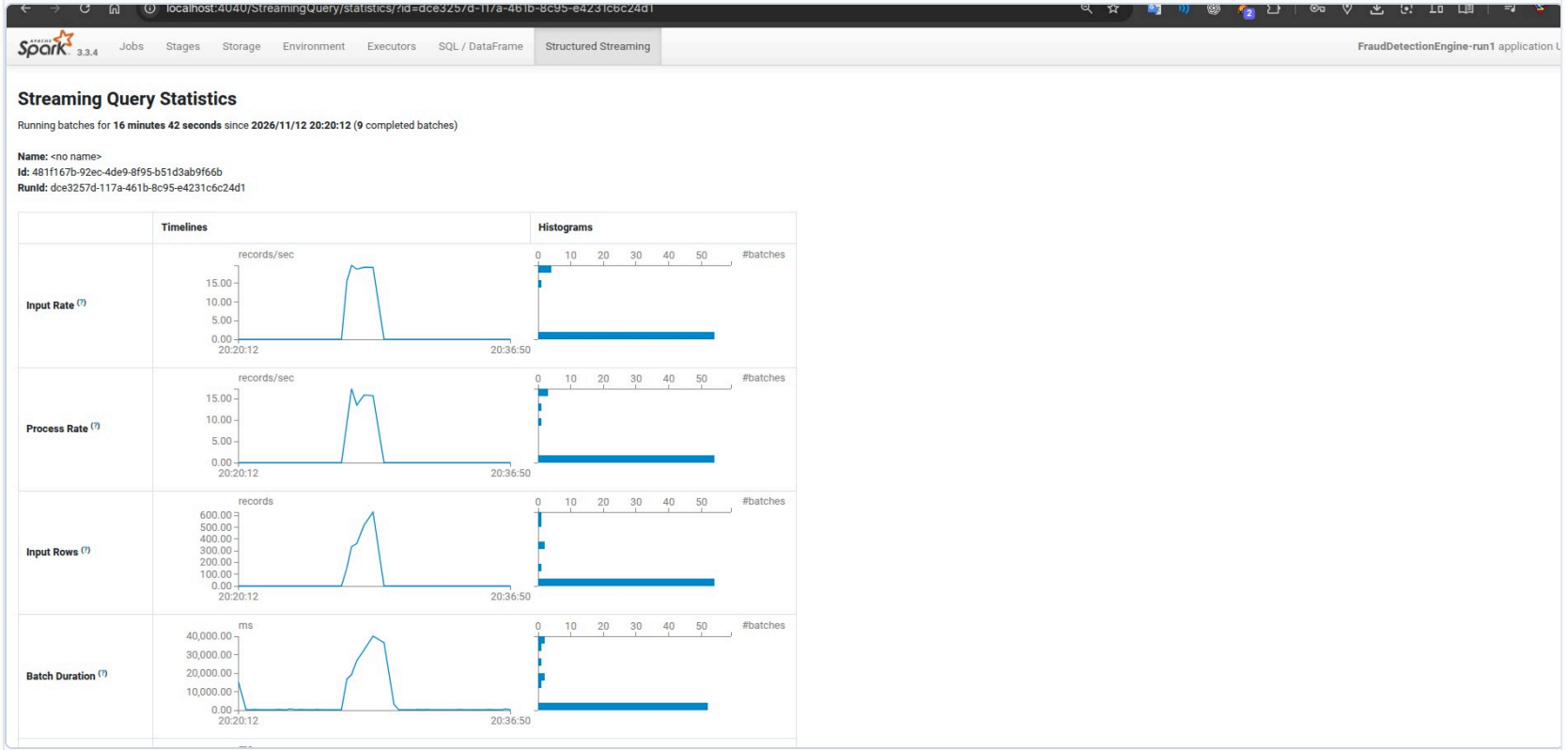
WHY IT MATTERS

One application per run, started with its own run id.

BASELINE

Run 1: Structured Streaming statistics

Fig 17 · Shashank · VM2 (Spark master + driver)



WHAT YOU ARE LOOKING AT

Input rate, process rate, input rows and batch duration over the run. Batch duration rises to about 40 s while data is flowing and returns to idle afterwards.

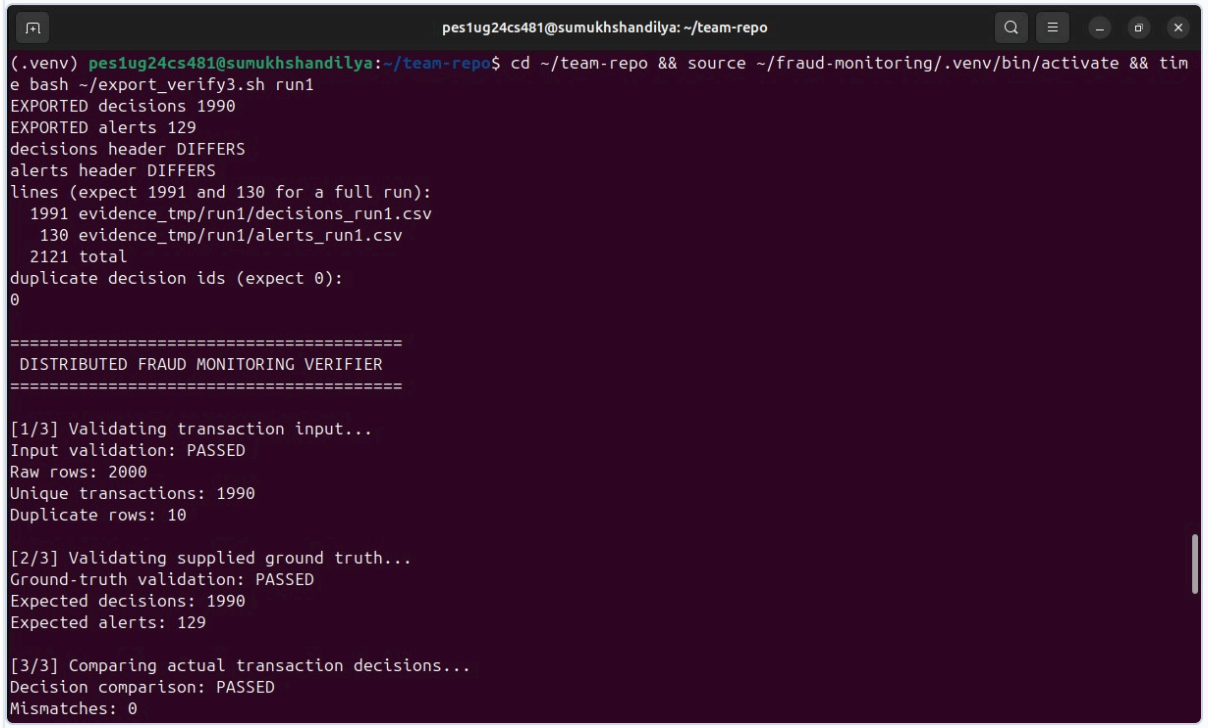
WHY IT MATTERS

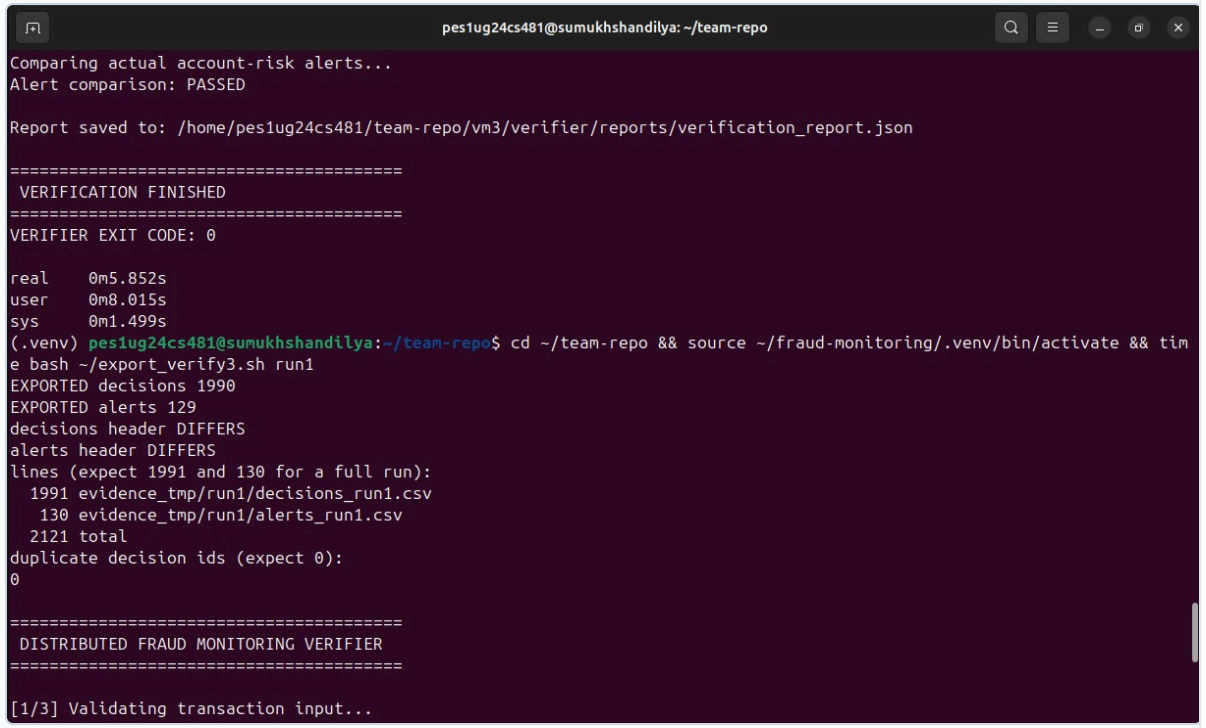
Honest performance picture. Batches take longer than the 10 s trigger because each batch re-reads and re-evaluates the stored decisions so that sliding windows stay correct.

BASELINE

Run 1: verifier PASS

Fig 18 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)





WHAT YOU ARE LOOKING AT

Input validation (2000 rows, 1990 unique, 10 duplicates), ground-truth validation (1990 decisions, 129 alerts), decision comparison PASSED with 0 mismatches, alert comparison PASSED, verifier exit code 0. The exported output has 1990 decisions and 129 alerts and 0 duplicate decision ids.

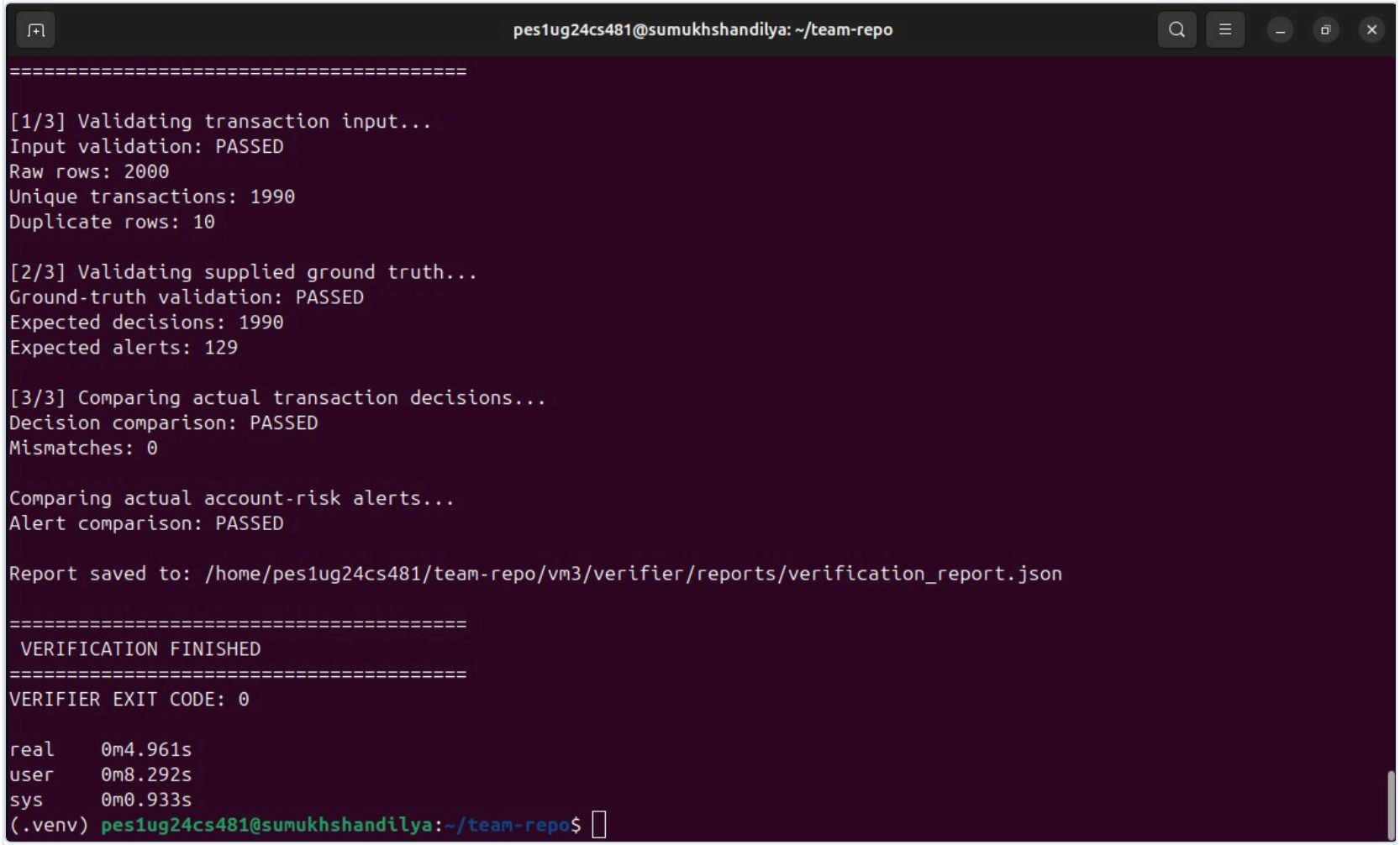
WHY IT MATTERS

The baseline passes all six verifier checks. This is the reference every experiment is compared with.

BASELINE

Run 1: verifier re-run (repeatable)

Fig 19 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)



WHAT YOU ARE LOOKING AT

The verifier is run a second time on the same export: same result, exit code 0.

WHY IT MATTERS

The result does not depend on a lucky run of the verifier.

DASHBOARD

FraudWatch dashboard

Run 1 · Demo mode

FraudWatch dashboard: what the screenshots cover

FEATURES SHOWN

Filters	Transaction decision, fraud rule, account (source or destination) and time range. Every KPI, chart and table follows them.
KPIs and charts	Totals, approved, review, blocked, rule triggers R1–R5, decision distribution, rule triggers, volume per minute.
Tables	All transactions with rule flags, alert details with search, flagged accounts.
Investigation	Pick an alert or a transaction id: why it was flagged, the transaction, the supporting transactions and the account's last 20 transactions.
Errors	If a run cannot be loaded the dashboard shows a clear message with the Spark error.
Sources	LIVE reads Iceberg through the Spark Thrift Server. DEMO reads the verified CSVs and says so on screen.

TWO VERSIONS

v1 is the first dashboard (pages first). v2 adds the filters that change every number, volume per minute and the investigation view. Both are shown so the improvement is visible.

CHECK AGAINST THE GROUND TRUTH

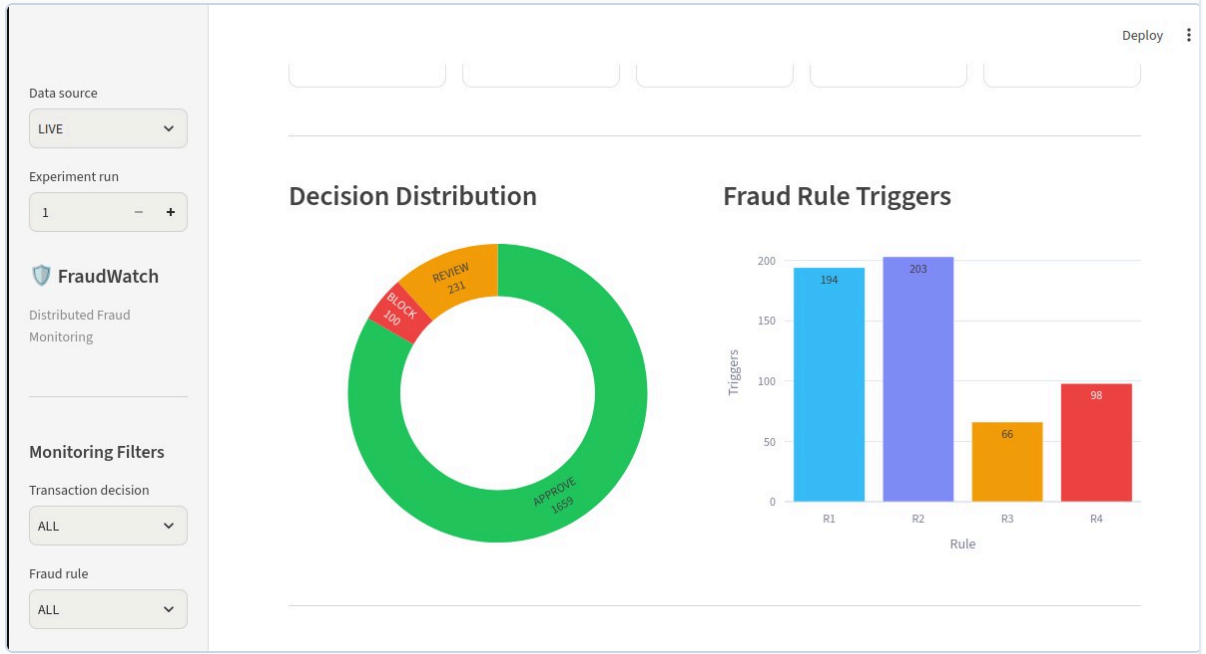
LIVE run 1 shows 1,990 / 1,659 / 231 / 100 and R1 194, R2 203, R3 66, R4 98, R5 100. BLOCK gives 100 and R2 gives 203, both equal to the verified counts.

DASHBOARD

Dashboard v1: LIVE overview (run 1)

Fig 20 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)





WHAT YOU ARE LOOKING AT

The first dashboard version in LIVE mode, experiment run 1. KPIs: 1,990 transactions, 1,659 approved, 231 under review, 100 blocked. Rule activity: R1 194, R2 203, R3 66, R4 98, R5 blocks 100. Decision distribution and rule trigger charts.

WHY IT MATTERS

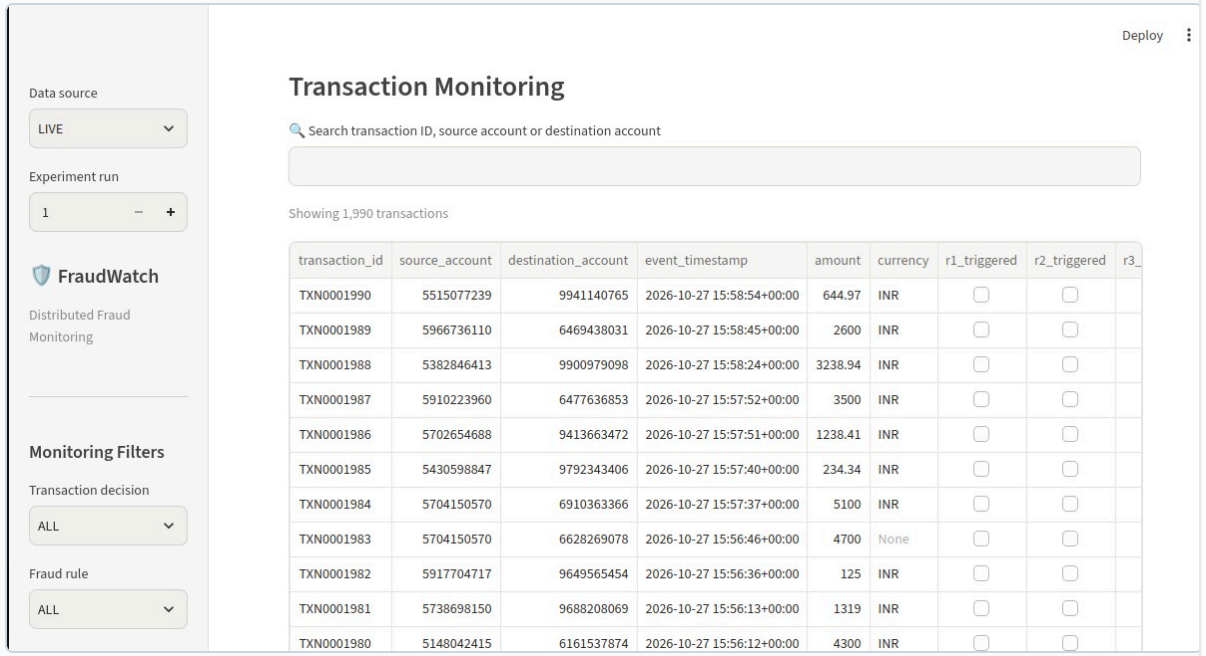
Dashboard reads the Iceberg tables over Spark Thrift Server and shows the same numbers as the verified baseline.

DASHBOARD

Dashboard v1: timeline and transactions

Fig 21 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)





WHAT YOU ARE LOOKING AT

Transaction activity timeline by decision (event time) and the transaction table (1,990 rows, rule flags per transaction).

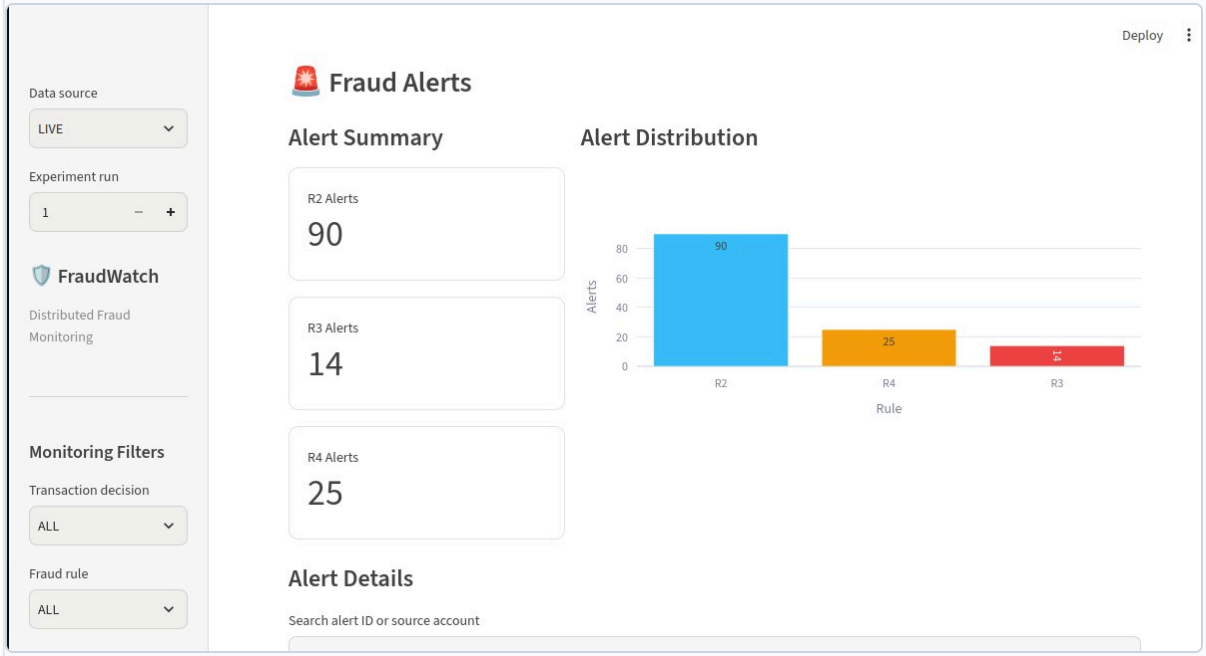
WHY IT MATTERS

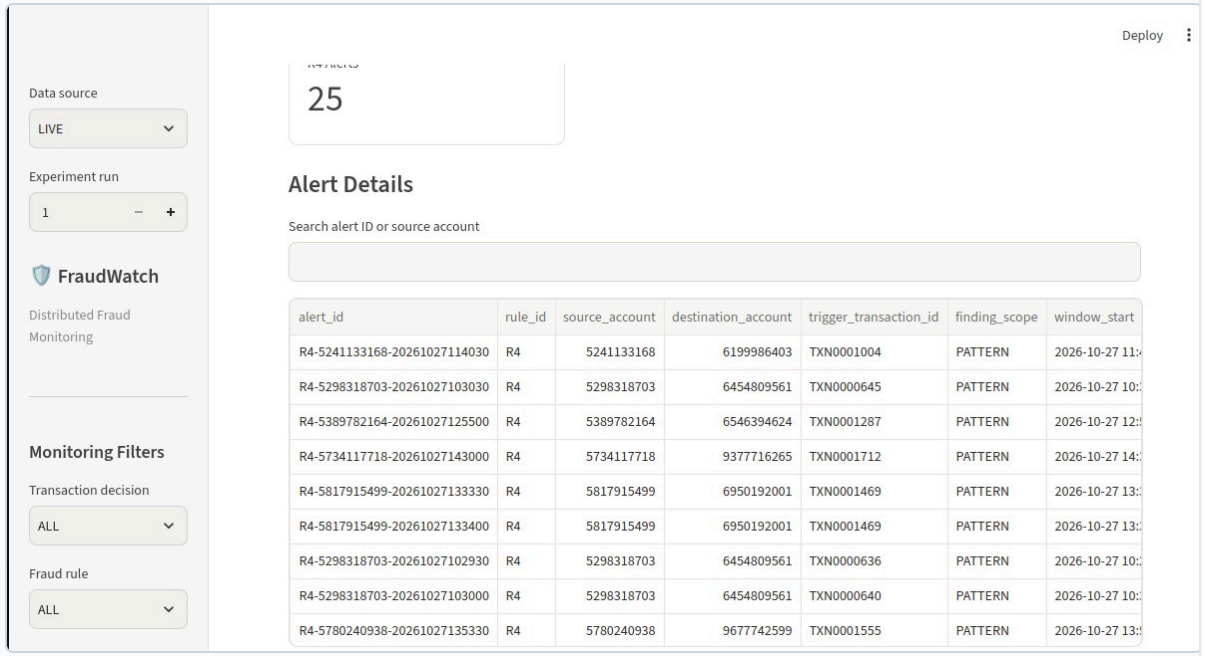
Time view and row-level detail.

DASHBOARD

Dashboard v1: alerts

Fig 22 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)





WHAT YOU ARE LOOKING AT

Alert summary R2 90, R3 14, R4 25 (129 in total), the alert distribution chart and the alert details table with search.

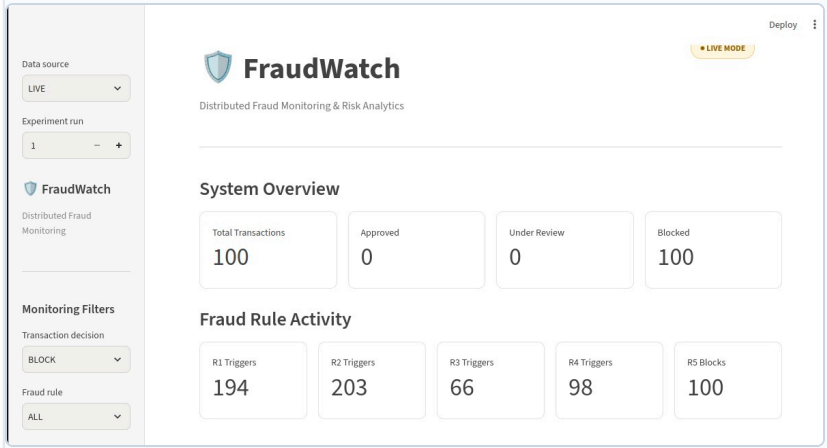
WHY IT MATTERS

Matches the 129 ground-truth alerts.

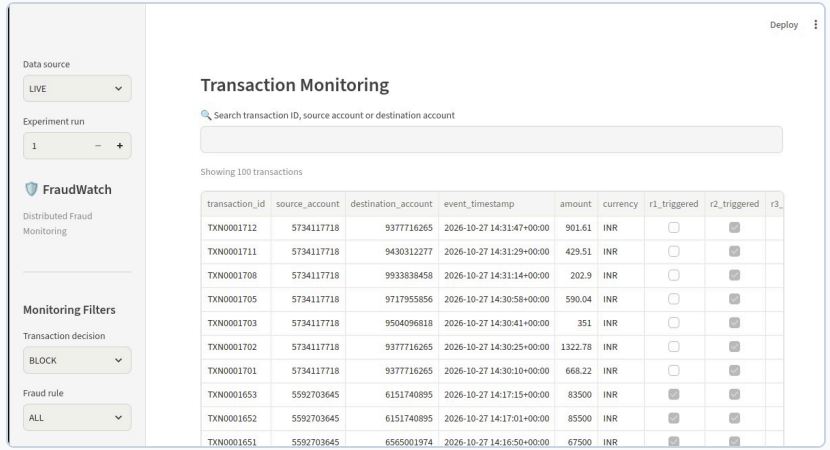
DASHBOARD

Dashboard v1: filter decision = BLOCK

Fig 23 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)







WHAT YOU ARE LOOKING AT

Decision filter BLOCK: 100 transactions. Timeline of blocked transactions and table of blocked rows.

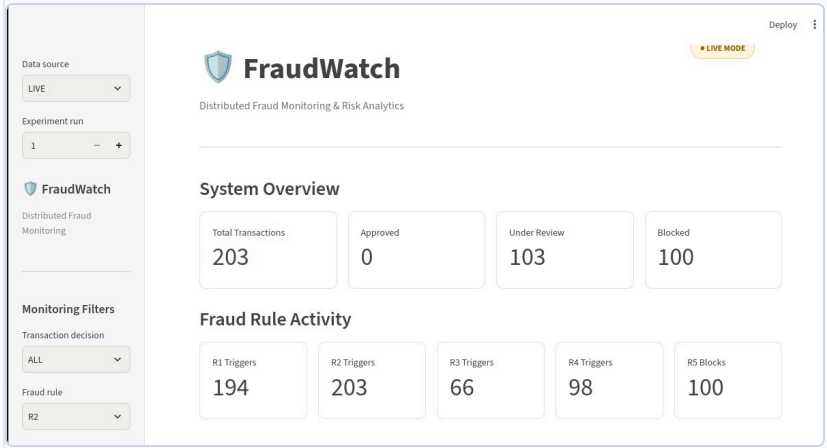
WHY IT MATTERS

The decision filter works. In this first version the rule counters still showed whole-run values; that was improved in v2.

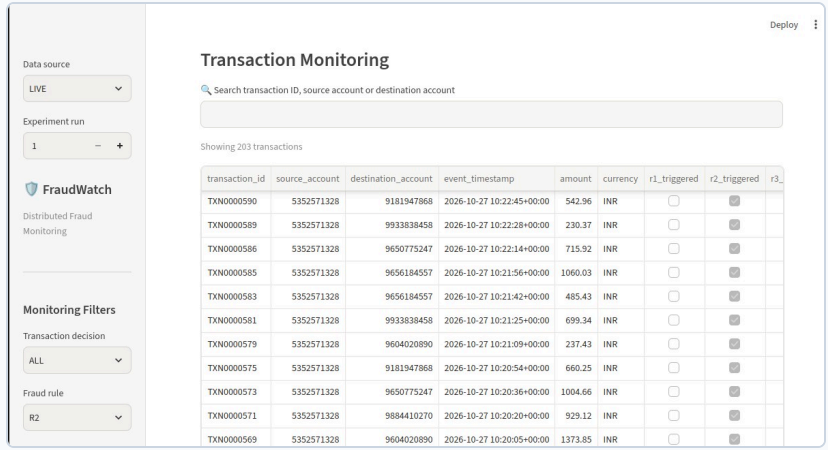
DASHBOARD

Dashboard v1: filter rule = R2

Fig 24 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)







WHAT YOU ARE LOOKING AT

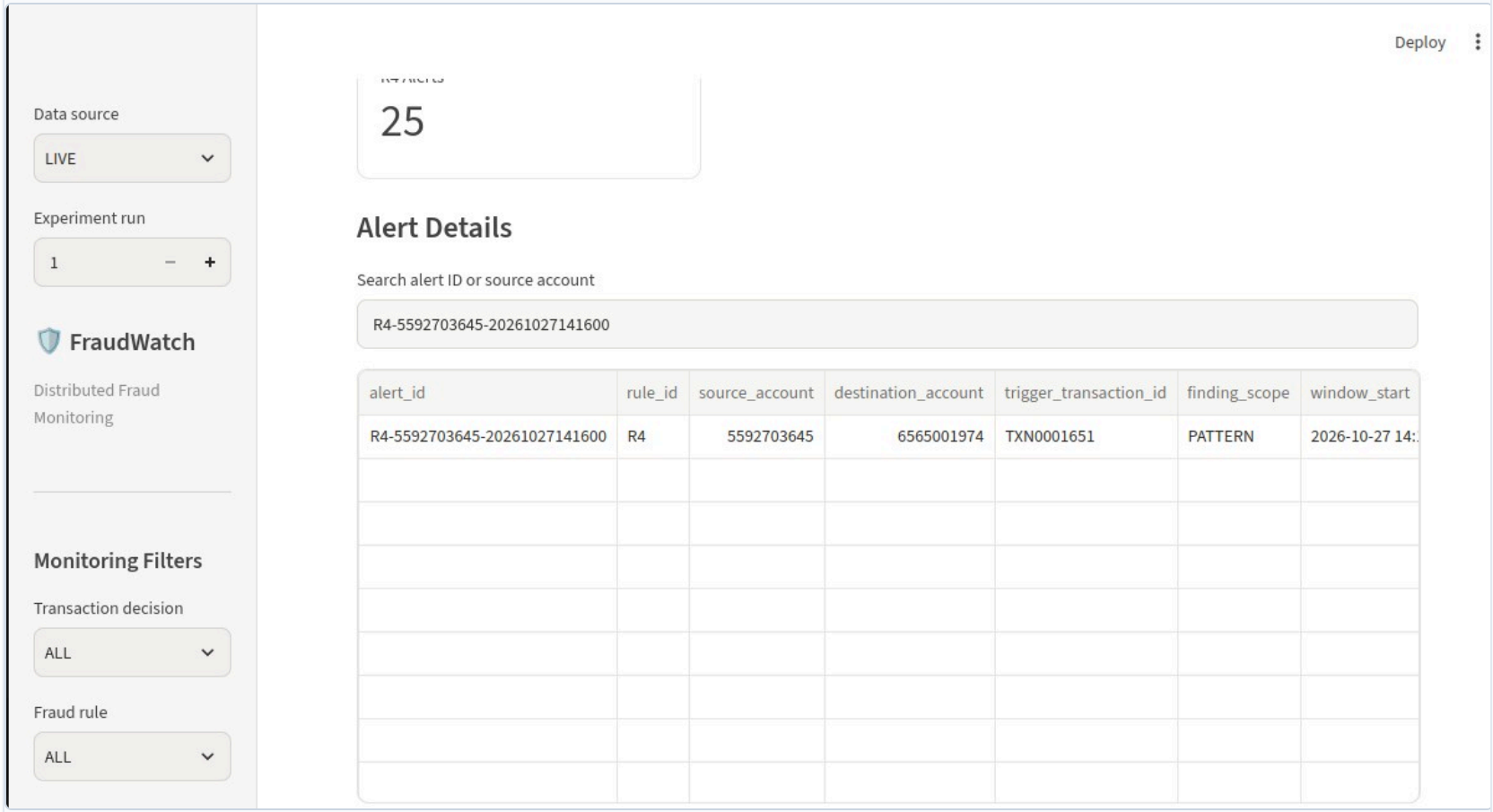
Rule filter R2: 203 transactions (103 under review, 100 blocked). Timeline and table of those rows.

WHY IT MATTERS

The rule filter works. 203 equals the ground-truth R2 trigger count.

Dashboard v1: alert search

Fig 25 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)



WHAT YOU ARE LOOKING AT

Search for alert R4-5592703645-20261027141600: one alert, trigger transaction TXN0001651, scope PATTERN.

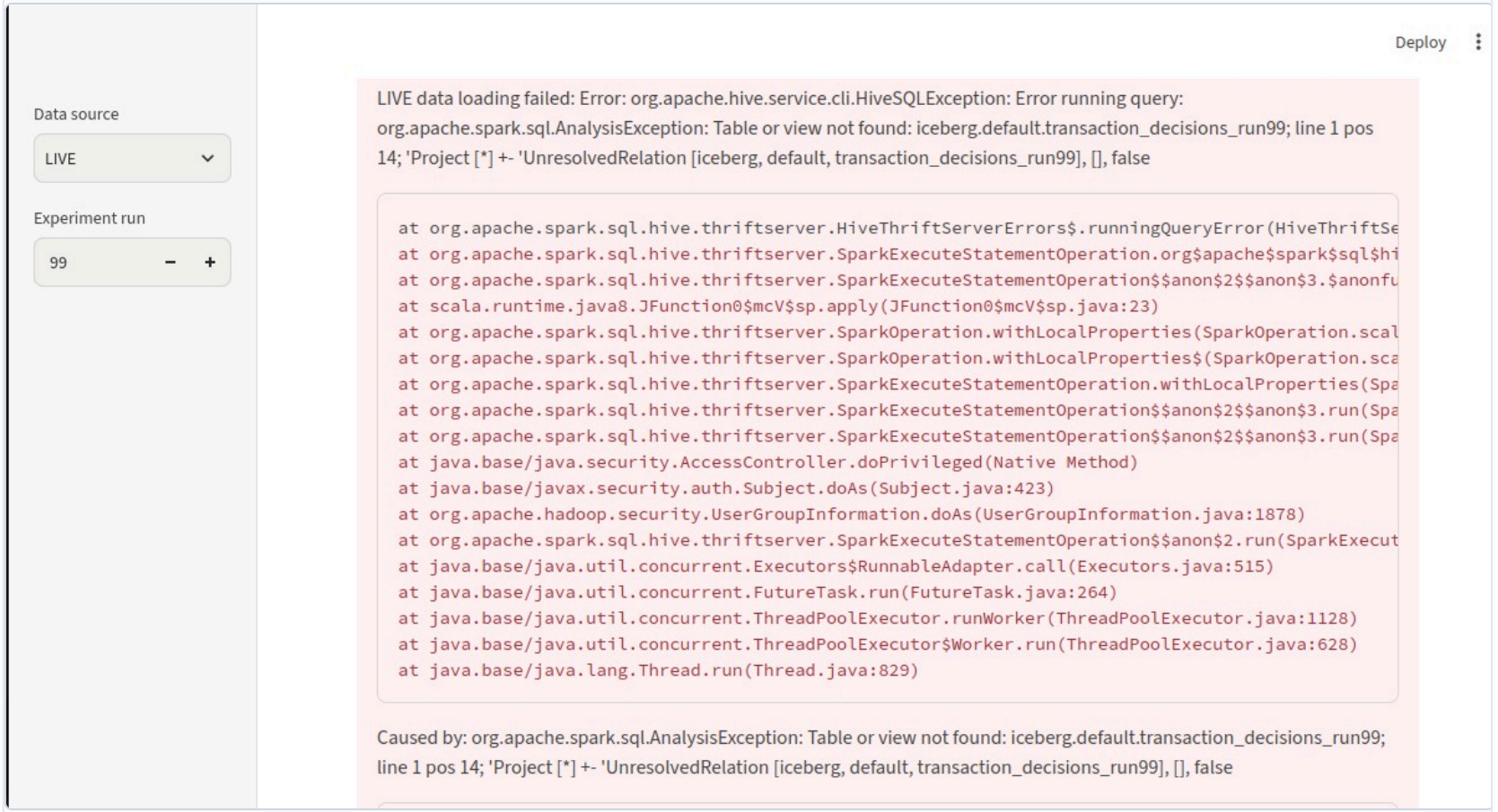
WHY IT MATTERS

Alerts can be found by id or account.

DASHBOARD

Dashboard: clear error when data cannot be loaded

Fig 26 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)



WHAT YOU ARE LOOKING AT

Experiment run 99 does not exist, so the dashboard shows LIVE data loading failed with the Spark error: Table or view not found: transaction_decisions_run99.

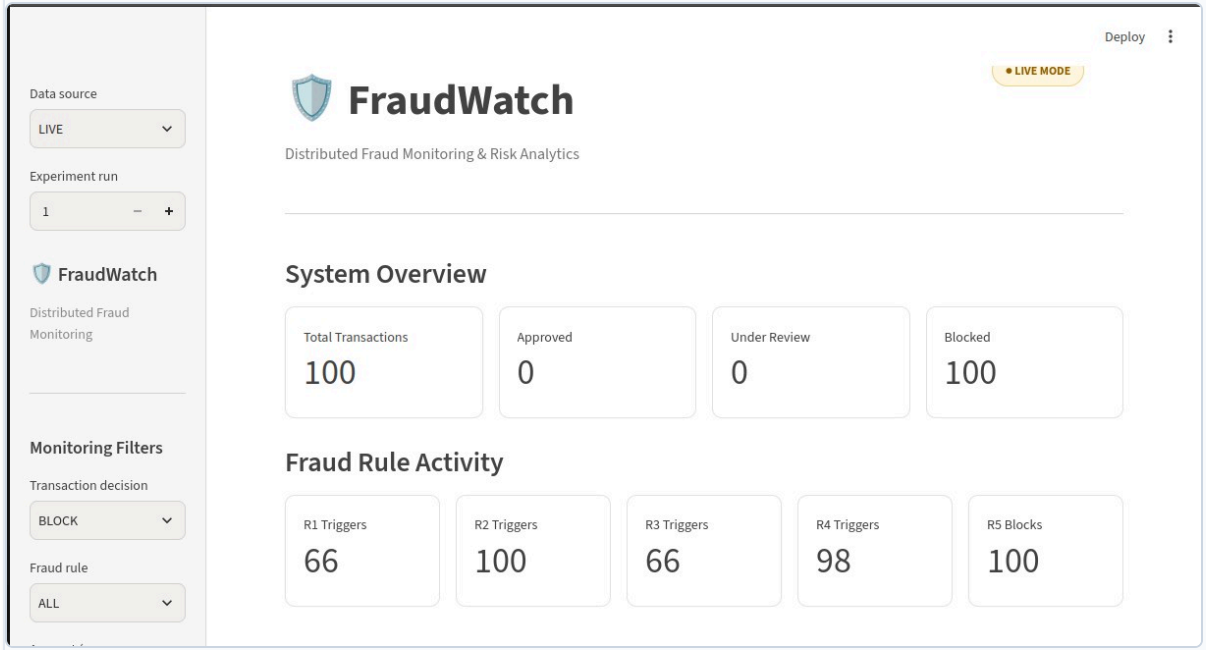
WHY IT MATTERS

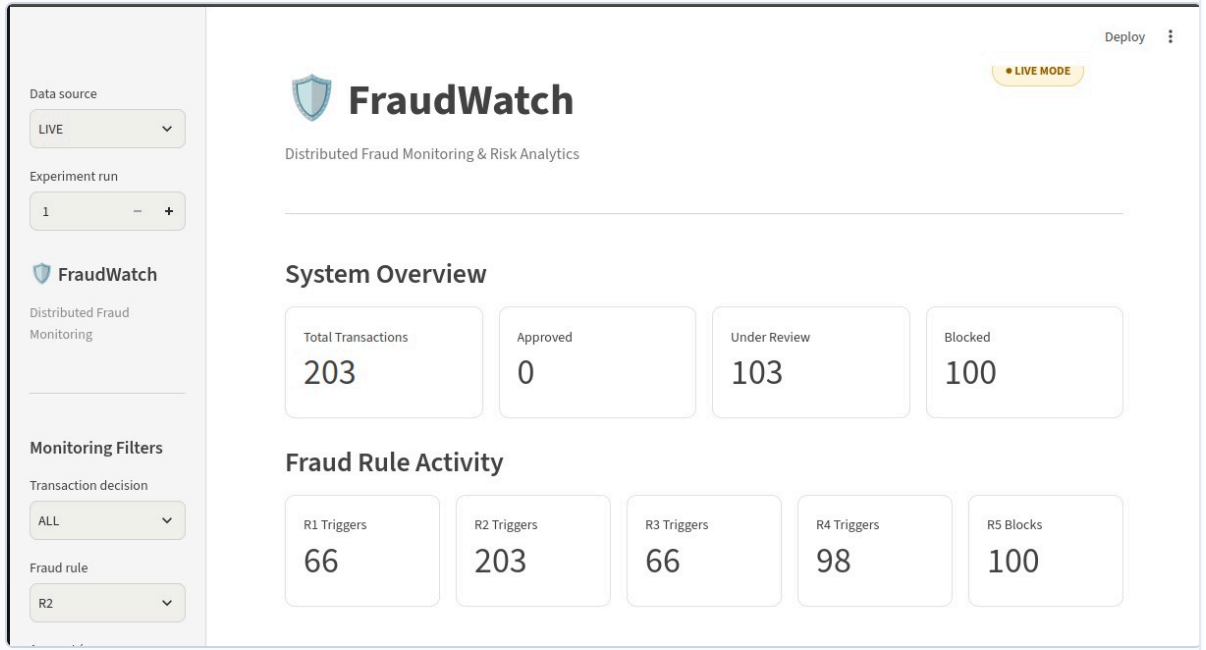
The required clear error message instead of a blank screen.

DASHBOARD

Dashboard v2: filters change every number

Fig 27 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)





WHAT YOU ARE LOOKING AT

Decision = BLOCK (100 rows) and rule = R2 (203 rows) in LIVE run 1. In v2 the rule KPIs follow the filter, for example under BLOCK the R1 counter shows 66 instead of the whole-run 194.

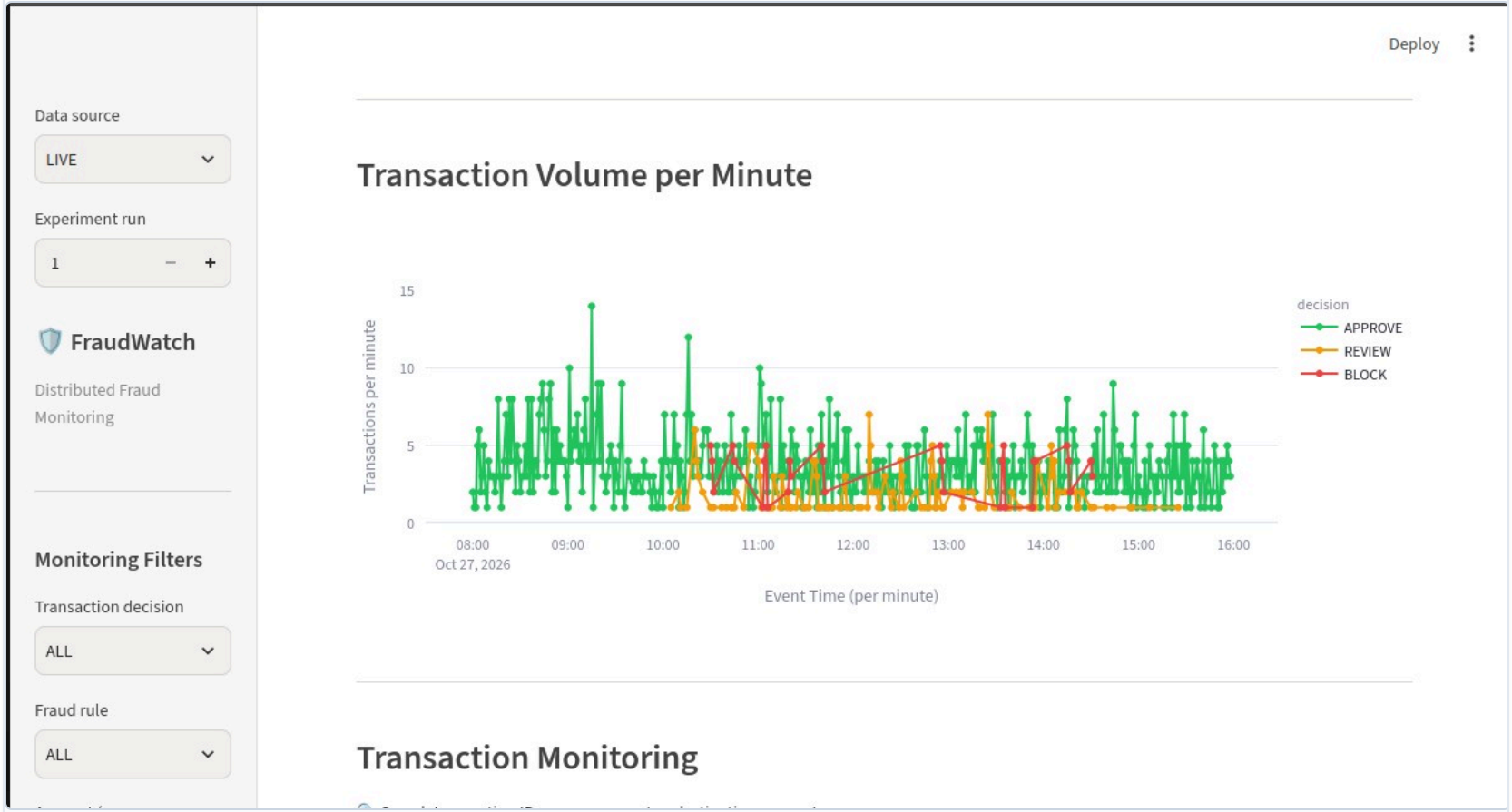
WHY IT MATTERS

Filters now apply to KPIs, charts and tables. R2 = 203 and BLOCK = 100 match the verified ground truth.

DASHBOARD

Dashboard v2: volume per minute

Fig 28 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)



WHAT YOU ARE LOOKING AT

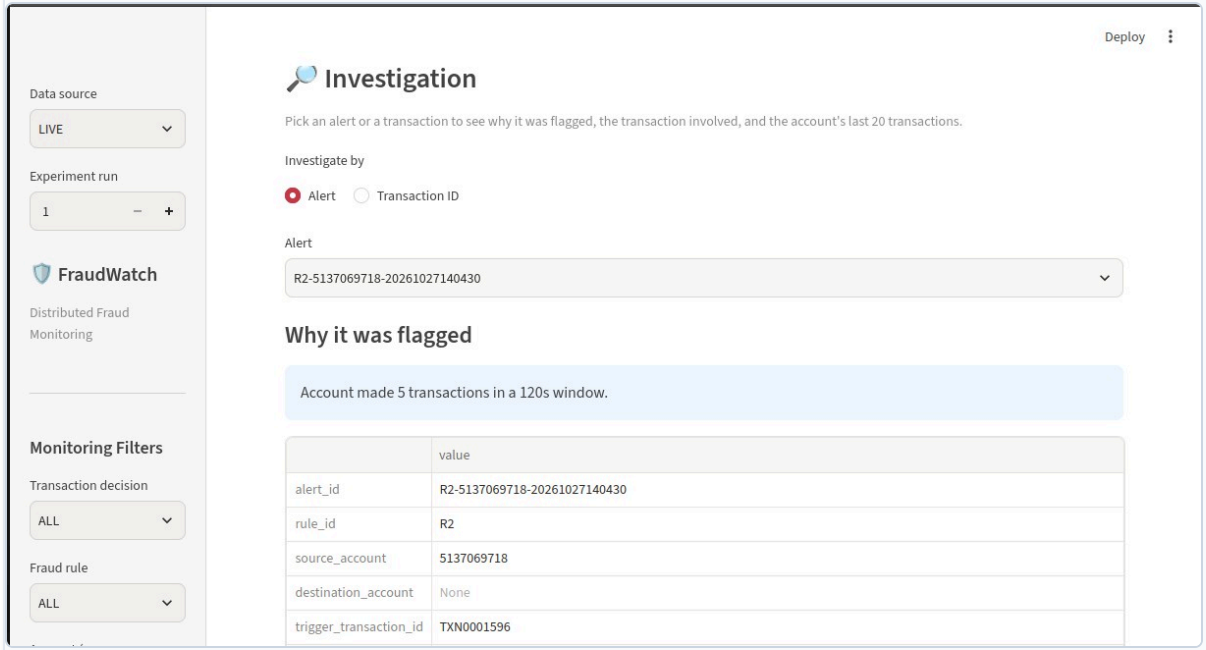
Transactions per minute by event time, split by decision (approve, review, block).

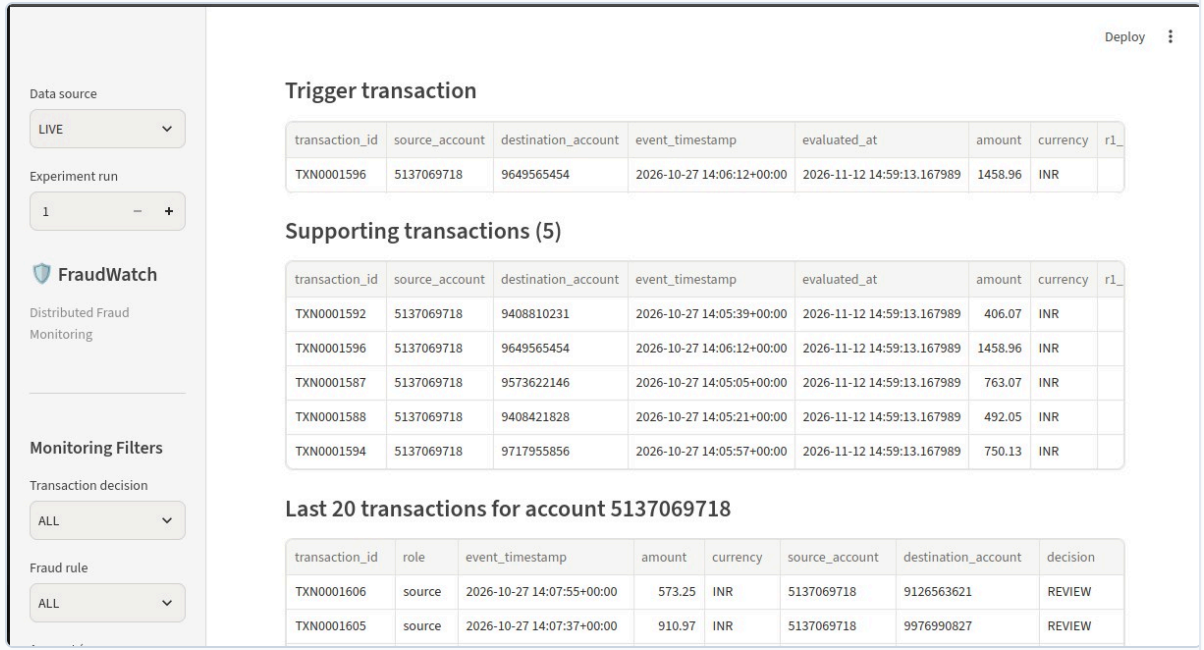
WHY IT MATTERS

The required volume per minute chart.

Dashboard v2: investigation by alert

Fig 29 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)





WHAT YOU ARE LOOKING AT

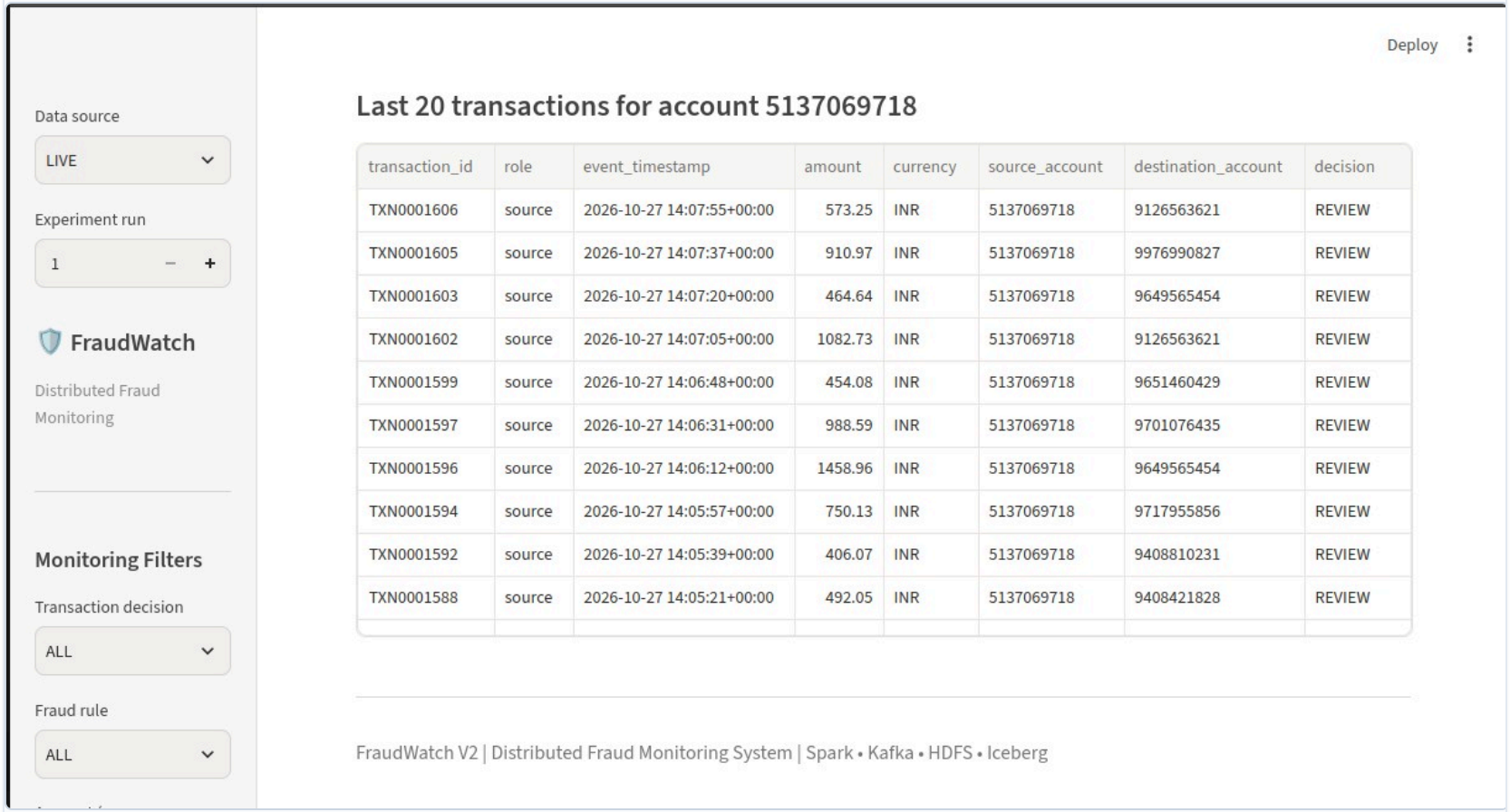
Alert R2-5137069718-20261027140430 picked. Why it was flagged: Account made 5 transactions in a 120 s window. Alert fields, the trigger transaction TXN0001596, and the 5 supporting transactions are shown.

WHY IT MATTERS

The investigation view answers why an alert exists and which transactions caused it.

Dashboard v2: last 20 transactions of the account

Fig 30 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)



transaction_id	role	event_timestamp	amount	currency	source_account	destination_account	decision
TXN0001606	source	2026-10-27 14:07:55+00:00	573.25	INR	5137069718	9126563621	REVIEW
TXN0001605	source	2026-10-27 14:07:37+00:00	910.97	INR	5137069718	9976990827	REVIEW
TXN0001603	source	2026-10-27 14:07:20+00:00	464.64	INR	5137069718	9649565454	REVIEW
TXN0001602	source	2026-10-27 14:07:05+00:00	1082.73	INR	5137069718	9126563621	REVIEW
TXN0001599	source	2026-10-27 14:06:48+00:00	454.08	INR	5137069718	9651460429	REVIEW
TXN0001597	source	2026-10-27 14:06:31+00:00	988.59	INR	5137069718	9701076435	REVIEW
TXN0001596	source	2026-10-27 14:06:12+00:00	1458.96	INR	5137069718	9649565454	REVIEW
TXN0001594	source	2026-10-27 14:05:57+00:00	750.13	INR	5137069718	9717955856	REVIEW
TXN0001592	source	2026-10-27 14:05:39+00:00	406.07	INR	5137069718	9408810231	REVIEW
TXN0001588	source	2026-10-27 14:05:21+00:00	492.05	INR	5137069718	9408421828	REVIEW

WHAT YOU ARE LOOKING AT

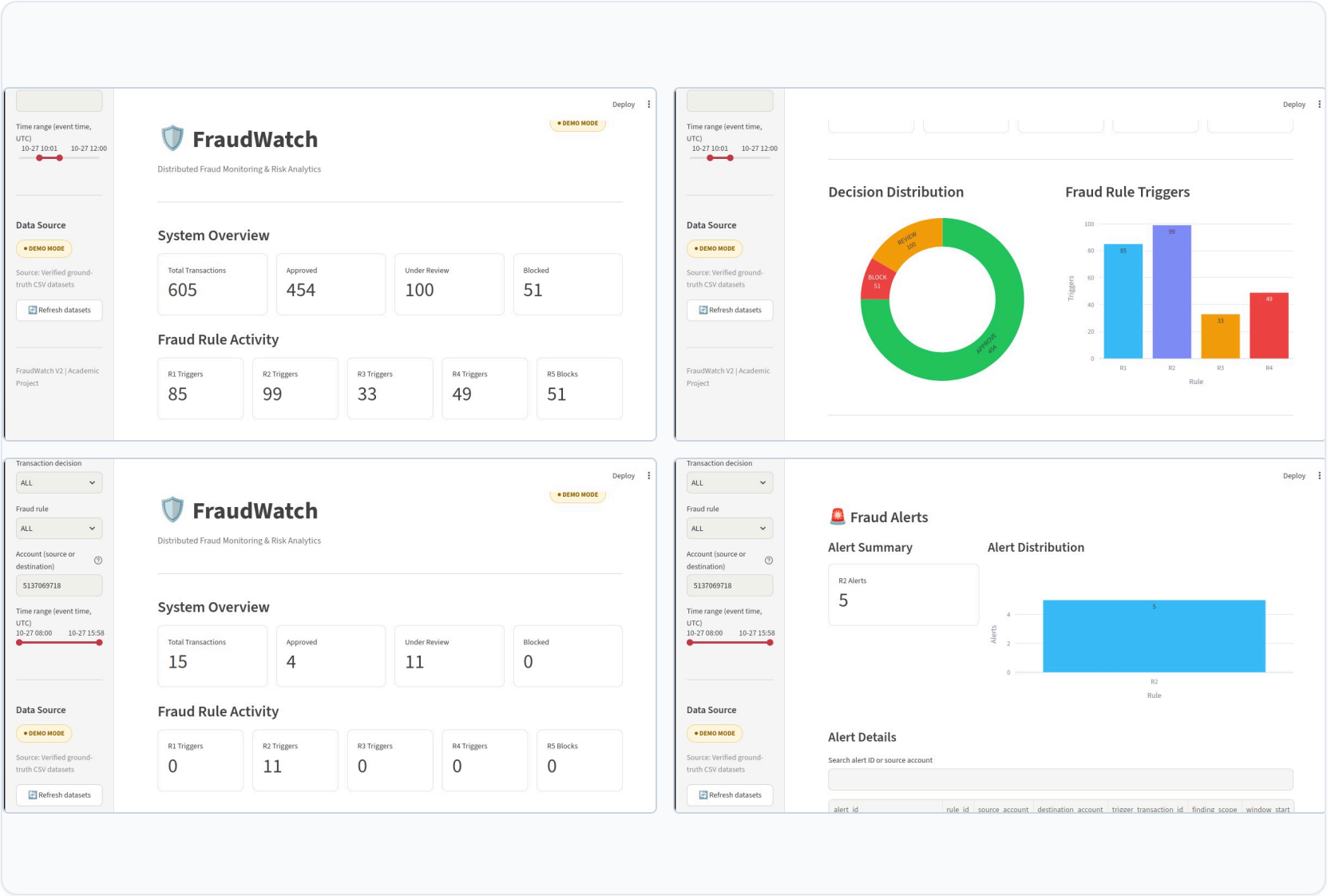
Last 20 transactions for account 5137069718 with amount, counterparty and decision (all REVIEW).

WHY IT MATTERS

The account history the project asks for.

Demo mode: time range and account filters

Fig 31 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)



WHAT YOU ARE LOOKING AT

Demo mode (the verified ground-truth CSVs, not Iceberg). Time range 10:01 to 12:00 UTC: 605 transactions. Account 5137069718: 15 transactions, 4 approved, 11 under review, R2 triggers 11, and 5 R2 alerts.

WHY IT MATTERS

Shows the time-range and account filters working. Demo mode is a fallback and is labelled as such on screen.

EXPERIMENT 1

Executor failure

Run 2

EXPERIMENT 1

Experiment 1: one executor is killed during processing

OBJECTIVE

Kill one Spark executor while the stream is running and show that the job finishes with correct results and no manual action.

WHAT HAPPENED (IST)

20:39	Application run2 starts with 2 executors (2 cores, 512 MB)
20:41	Producer starts, slow stream of 2,000 messages (0.3 s each, about 10 min)
20:44:07	kill -9 on executor process 30200. Driver logs Lost executor 0.
+2 s	Worker starts a replacement process. Spark UI shows executor 0 dead, executor 2 active.
20:44:17	Batch 11 completes normally (550 decisions)
20:51	Producer finishes
20:52:38	Batch 25 writes 1,990 decisions and 129 alerts, batch 26 is empty
after	Verifier PASS

VERDICT

PASS

KEY NUMBERS

Decisions	**1,990 / 1,990**
Alerts	**129 / 129**
Missing / extra / mismatched	**0 / 0 / 0**
Failed tasks	**0**
Manual intervention	**none**
Checkpoint commits	**0–26**

WHAT IT DEMONSTRATES

Losing an executor does not interrupt the stream or change the output. Spark replaces the executor and finishes the job.

EXPERIMENT 1

Producer: slower stream so the failure lands mid-run

Fig 32 · Saatwik · VM1 (Kafka + producer)

```
RESULT: PASS
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$ cd ~/team-repo && source .venv/bin/activate && source ~/fraud/configs/cluster.env && mkdir -p ~/fraud/logs/run2 && date +%T
python3 vm1/producer.py --mode baseline --topic transactions-run2 --run-id run2 --input data/transactions.csv --bootstrap $KAFKA_BOOTSTRAP --send-delay 0.3 --log-dir ~/fraud
/logs/run2
15:11:12
/home/pes1ug24cs509/team-repo/vm1/producer.py:132: DeprecationWarning: key_serializer does not implement kafka.serializer.Serializer
  if __name__ == '__main__': main()
/home/pes1ug24cs509/team-repo/vm1/producer.py:132: DeprecationWarning: value_serializer does not implement kafka.serializer.Serializer
  if __name__ == '__main__': main()
sent 500/2000
sent 1000/2000
sent 1500/2000
sent 2000/2000
{
  "run_id": "run2",
  "mode": "baseline",
  "topic": "transactions-run2",
  "bootstrap": "172.23.211.163:9092",
  "input": "data/transactions.csv",
  "input_sha256": "980dd52388d1c4088e4806abd9efd70cb407ed0e9421113f3b25c3095b33d1a3",
  "host": "pes1ug24cs509",
  "dry_run": false,
  "rows_in_file": 2000,
  "unique_transaction_ids": 1990,
  "messages_sent": 2000,
  "broker_acks": 2000,
  "send_errors": [],
  "n_errors": 0,
  "seed": 42,
  "fraction": 0.02,
  "target_event_lag_s": null,
  "hard_cap_s": null,
  "send_delay_s": 0.3,
  "min_wall_hold_s": 0.0,
  "injected": 0,
  "injected_actually_late": 0,
  "injected_lag_gt_60s": 0,
  "all_rows_behind_running_max": 0,
  "all_rows_lag_gt_60s": 0,
  "start_utc": "2026-11-12T15:11:13.211+00:00",
  "end_utc": "2026-11-12T15:21:17.218+00:00",
  "duration_s": 604.0
}
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$
```

```
  "start_utc": "2026-11-12T15:11:13.211+00:00",
  "end_utc": "2026-11-12T15:21:17.218+00:00",
  "duration_s": 604.0
}
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$ cd ~/team-repo && source .venv/bin/activate && source ~/fraud/configs/cluster.env
$KAFKA_HOME/bin/kafka-get-offsets.sh --bootstrap-server $KAFKA_BOOTSTRAP --topic transactions-run2
python3 vm1/verify_topic.py --topic transactions-run2 --input data/transactions.csv --expect-order
transactions-run2:0:2000
csv_rows                        2000
topic_messages                  2000
missing_from_topic              0
extra_in_topic                  0
same_content_ignoring_order     True
same_order_as_file              True
key_equals_transaction_id       True
duplicate_ids_kept              True
all_values_are_strings          True
RESULT: PASS
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$
```

WHAT YOU ARE LOOKING AT

Fresh topic transactions-run2. The producer uses --send-delay 0.3 so the stream lasts about 10 minutes (started 15:11:13 UTC, 604 s). Afterwards the topic is verified: 2000 messages, same order, RESULT: PASS.

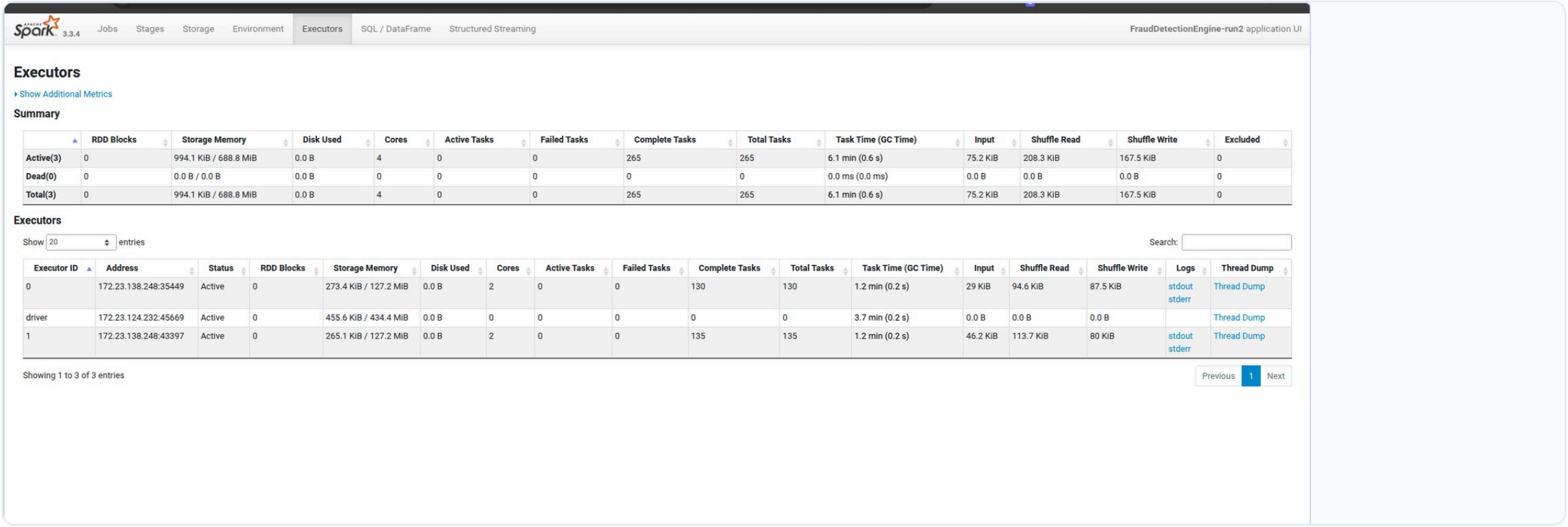
WHY IT MATTERS

A longer stream gives time to inject the fault while data is still arriving.

EXPERIMENT 1

Before the fault: two executors running

Fig 33 · Shashank · VM2 (Spark master + driver)



	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Excluded
Active(3)	0	994.1 KiB / 688.8 MiB	0.0 B	4	0	0	265	265	6.1 min (0.6 s)	75.2 KiB	208.3 KiB	167.5 KiB	0
Dead(0)	0	0.0 B / 0.0 B	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	0
Total(3)	0	994.1 KiB / 688.8 MiB	0.0 B	4	0	0	265	265	6.1 min (0.6 s)	75.2 KiB	208.3 KiB	167.5 KiB	0

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Logs	Thread Dump
0	172.23.138.248:35449	Active	0	273.4 KiB / 127.2 MiB	0.0 B	2	0	0	130	130	1.2 min (0.2 s)	29 KiB	94.6 KiB	87.5 KiB	stdout stderr	Thread Dump
driver	172.23.124.232:45669	Active	0	455.6 KiB / 434.4 MiB	0.0 B	0	0	0	0	0	3.7 min (0.2 s)	0.0 B	0.0 B	0.0 B		Thread Dump
1	172.23.138.248:43397	Active	0	265.1 KiB / 127.2 MiB	0.0 B	2	0	0	135	135	1.2 min (0.2 s)	46.2 KiB	113.7 KiB	80 KiB	stdout stderr	Thread Dump

WHAT YOU ARE LOOKING AT

Executors tab of application run2 before the kill: executor 0 and executor 1 (2 cores, 512 MB each) plus the driver, 265 tasks completed, no failures.

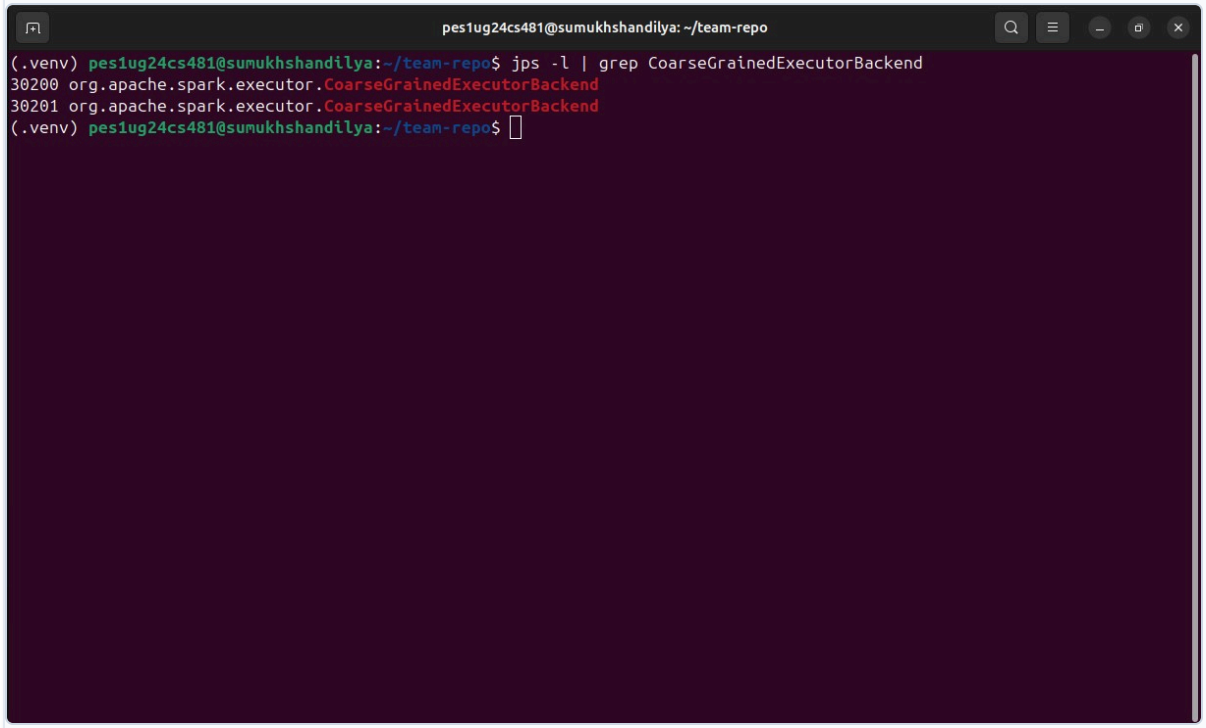
WHY IT MATTERS

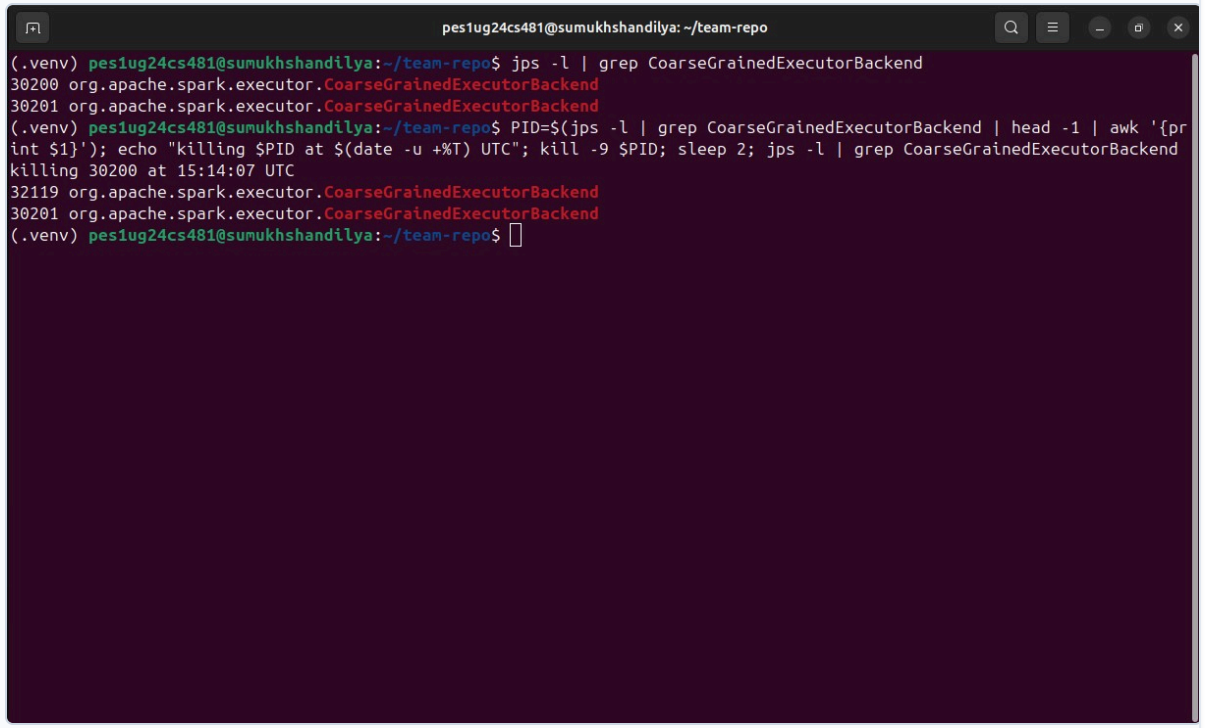
The healthy state before the failure.

EXPERIMENT 1

Failure injected: one executor is killed

Fig 34 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)





WHAT YOU ARE LOOKING AT

Top: jps lists the two executor processes (PIDs 30200 and 30201). Bottom: kill -9 on PID 30200 at 15:14:07 UTC (20:44:07 IST), about 3 min after the producer started. Two seconds later jps shows a new executor process (32119) next to the surviving one (30201).

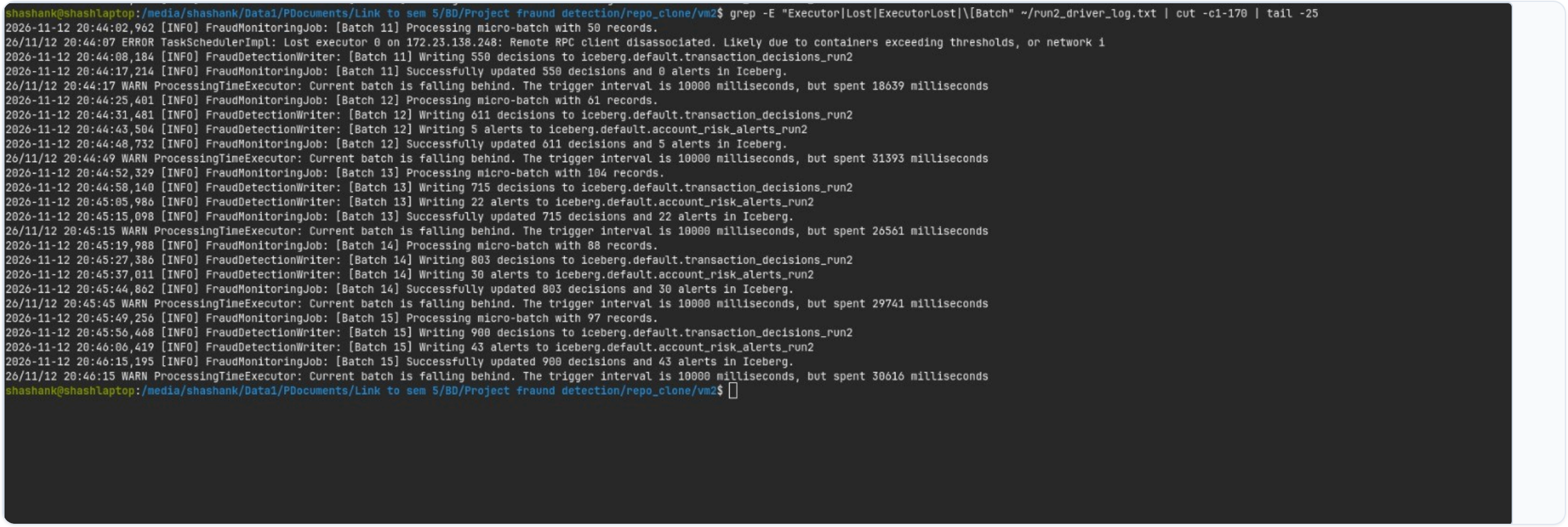
WHY IT MATTERS

A hard kill of one of the two executors while the stream is running. The worker starts a replacement process by itself.

EXPERIMENT 1

Driver log: executor lost, processing continues

Fig 35 · Shashank · VM2 (Spark master + driver)



WHAT YOU ARE LOOKING AT

At 20:44:07 the driver logs: Lost executor 0 … Remote RPC client disassociated. Batch 11 (50 records) had been processing and still completes at 20:44:17 (550 decisions). Batches 12 to 15 follow without a gap. WARN lines say the batch was slower than the 10 s trigger.

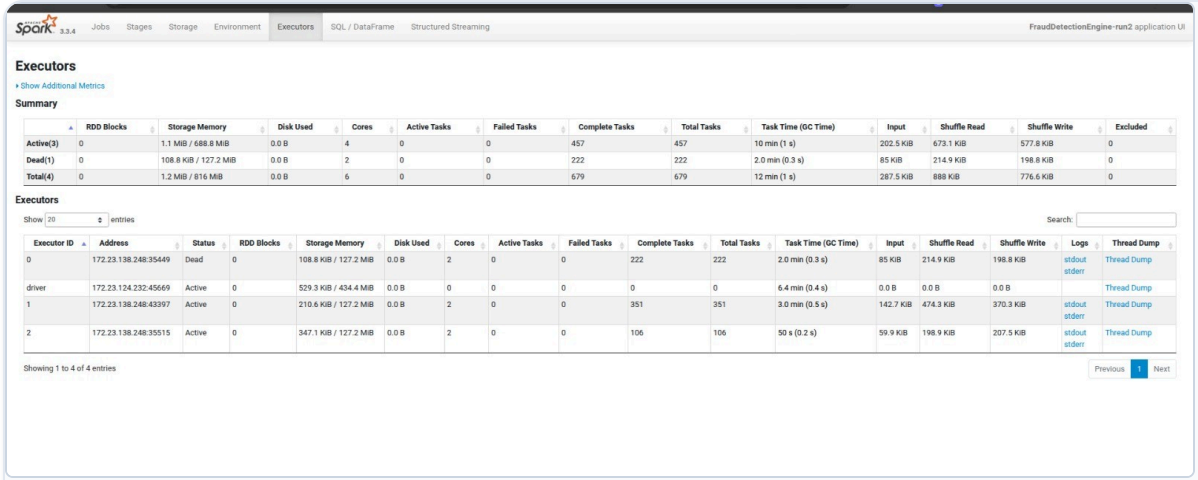
WHY IT MATTERS

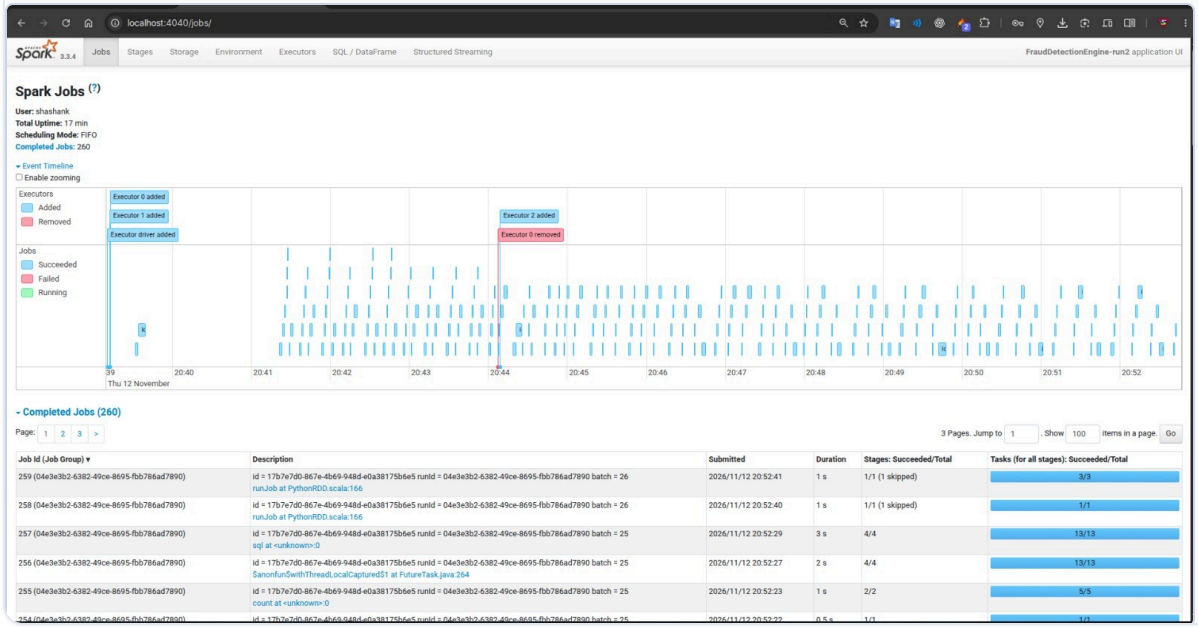
The loss is detected, no batch fails and nothing is restarted by hand.

EXPERIMENT 1

Spark UI right after the kill

Fig 36 · Shashank · VM2 (Spark master + driver)





WHAT YOU ARE LOOKING AT

Executors tab: executor 0 is Dead, a new executor 2 is Active (2 cores) and has already completed 106 tasks. Jobs timeline: a red Executor 0 removed marker and a blue Executor 2 added marker at about 20:44, and jobs keep running before and after.

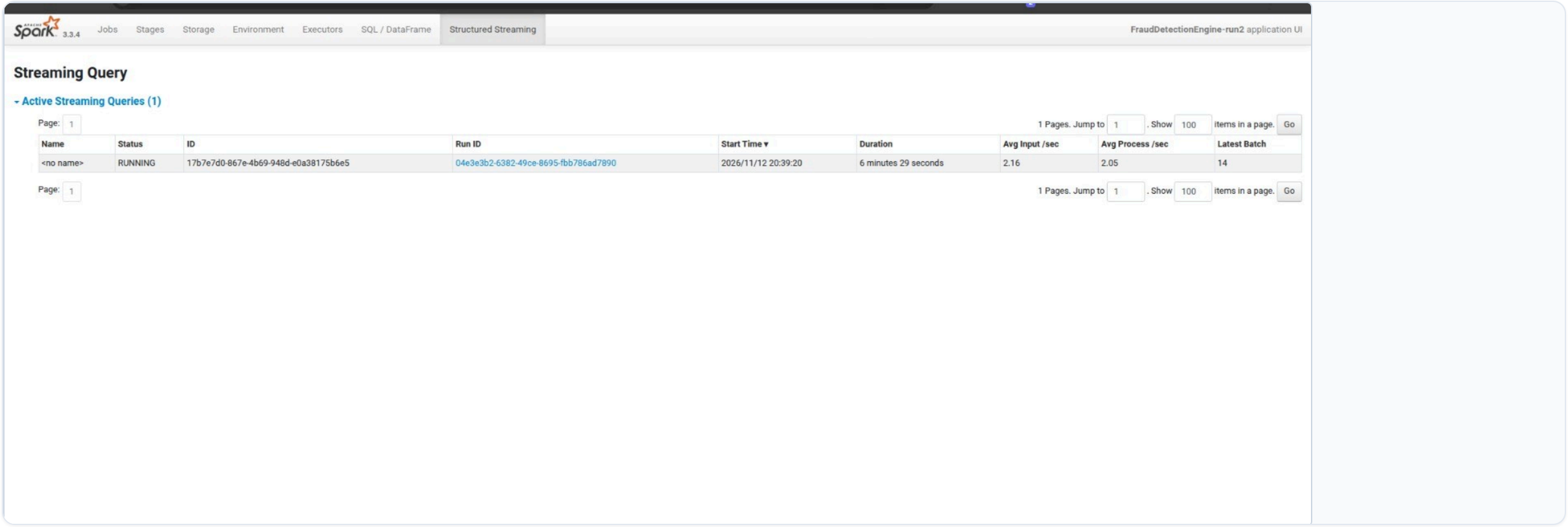
WHY IT MATTERS

Spark replaced the lost executor and rescheduled work on the survivor and the new executor.

EXPERIMENT 1

Streaming query keeps running

Fig 37 · Shashank · VM2 (Spark master + driver)



WHAT YOU ARE LOOKING AT

Structured Streaming tab: the query is still RUNNING, 6 minutes 29 seconds in, latest batch 14.

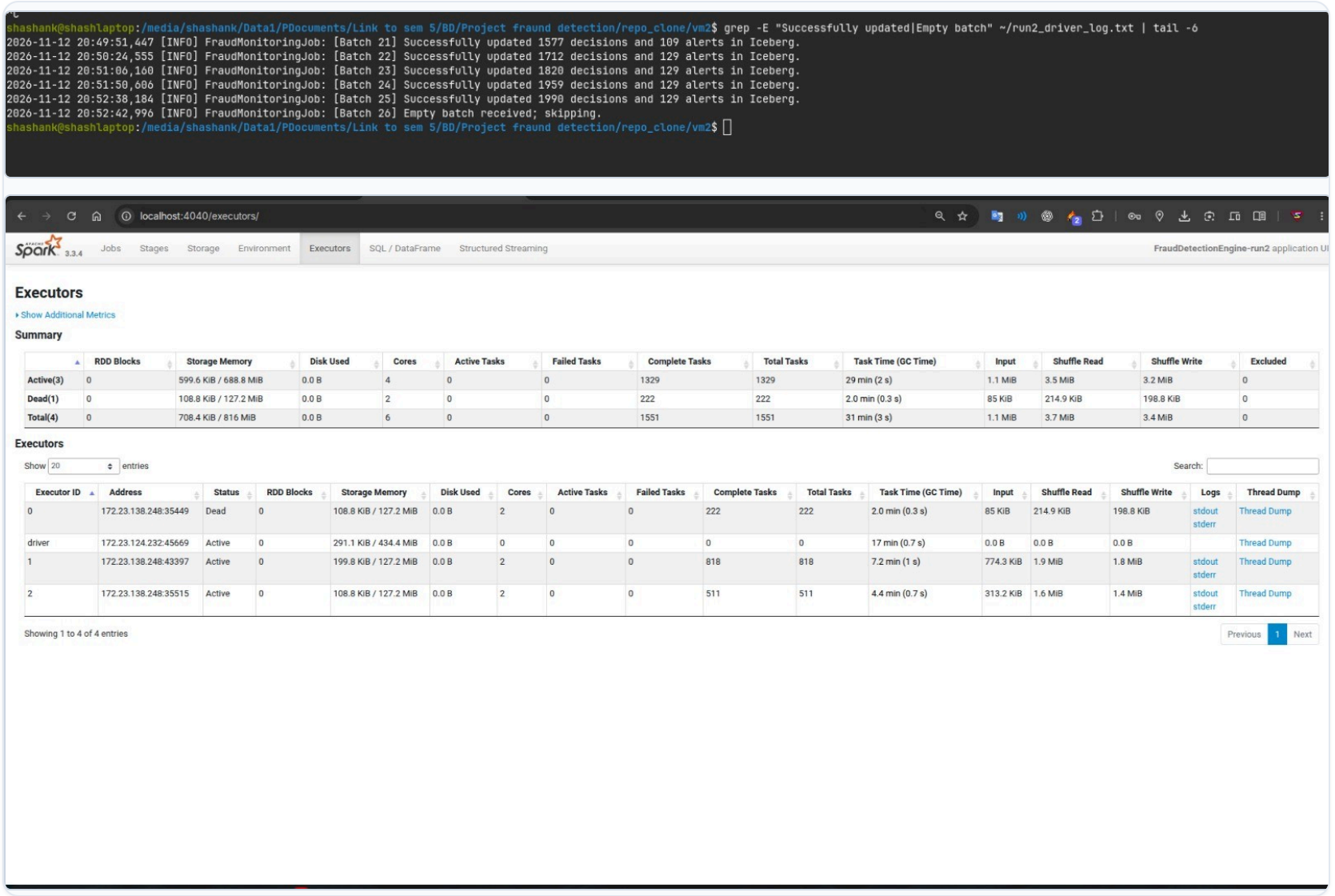
WHY IT MATTERS

The query was never restarted.

EXPERIMENT 1

End of the run: all data processed

Fig 38 · Shashank · VM2 (Spark master + driver)



WHAT YOU ARE LOOKING AT

Driver log: Batch 21 to 25 grow the output to 1990 decisions and 129 alerts (20:52:38 IST), Batch 26 is empty. Executors tab at the end: Active 3, Dead 1; executor 2 completed 511 tasks and executor 1 completed 818 tasks. The two live executors completed 1,329 tasks between them (the dead executor had completed 222 before the kill) and no task failed.

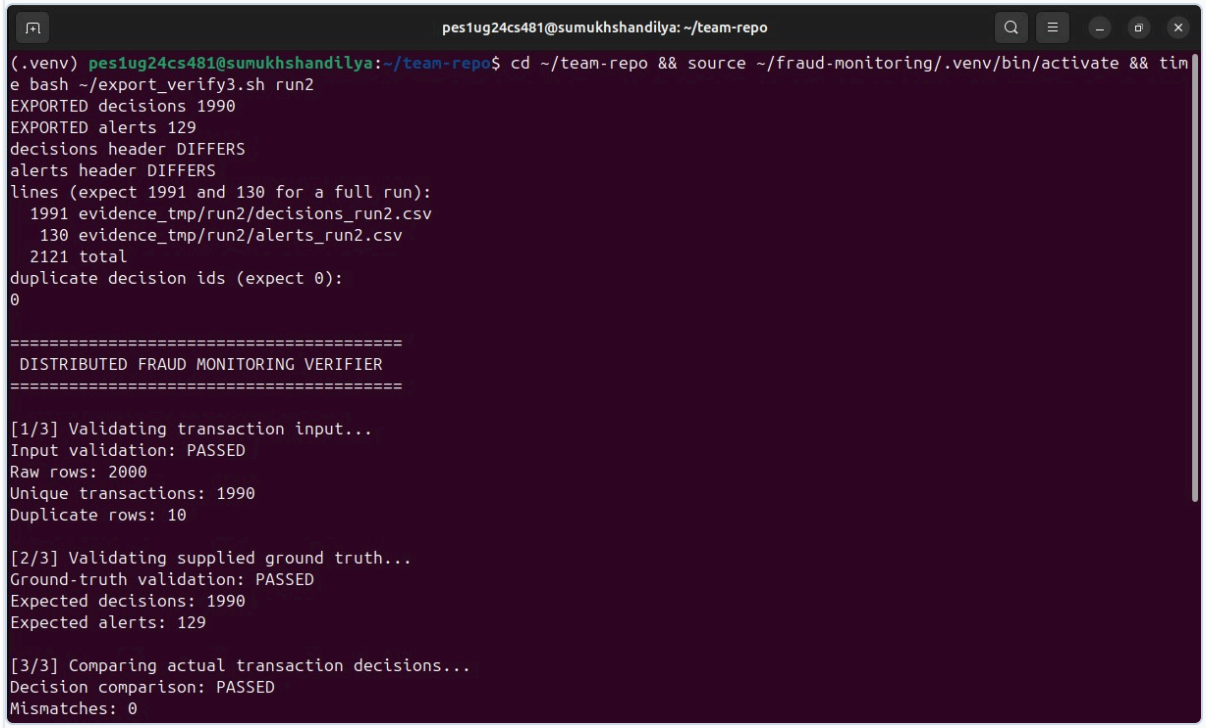
WHY IT MATTERS

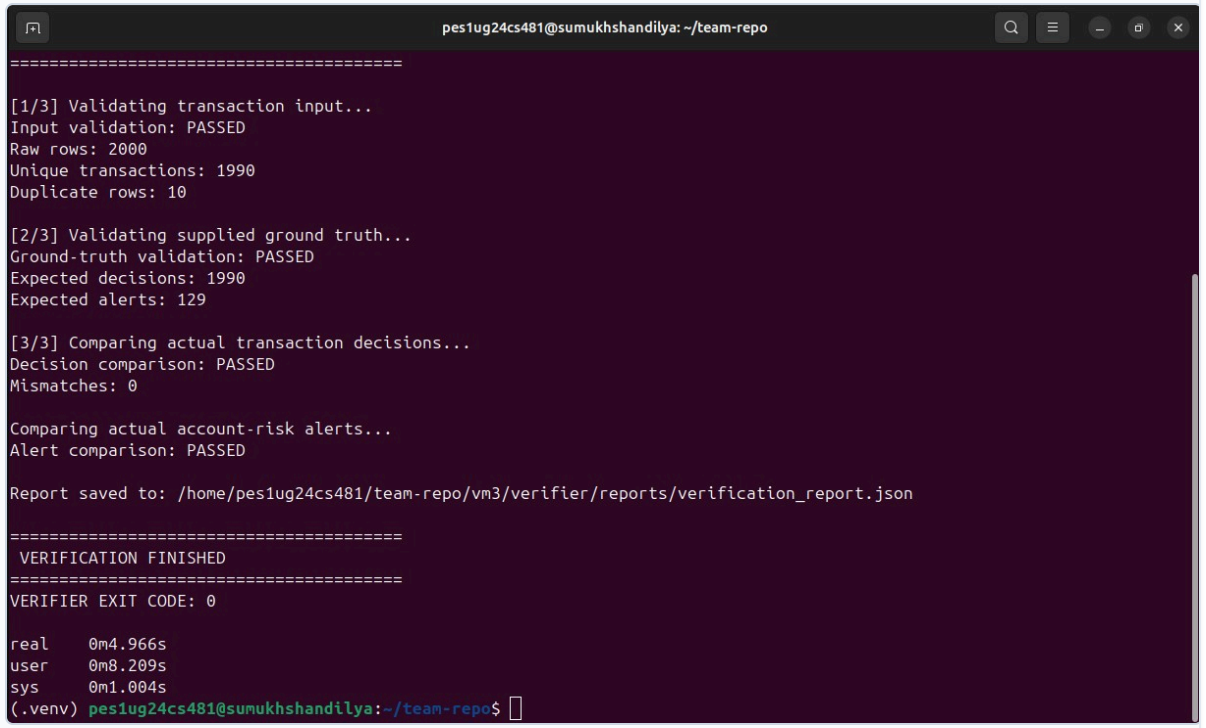
The system finished with the complete result after losing an executor.

EXPERIMENT 1

Verifier: PASS after the executor failure

Fig 39 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)





WHAT YOU ARE LOOKING AT

1990 expected decisions and 1990 actual, 0 mismatches. 129 alerts expected and 129 actual. Verifier exit code 0.

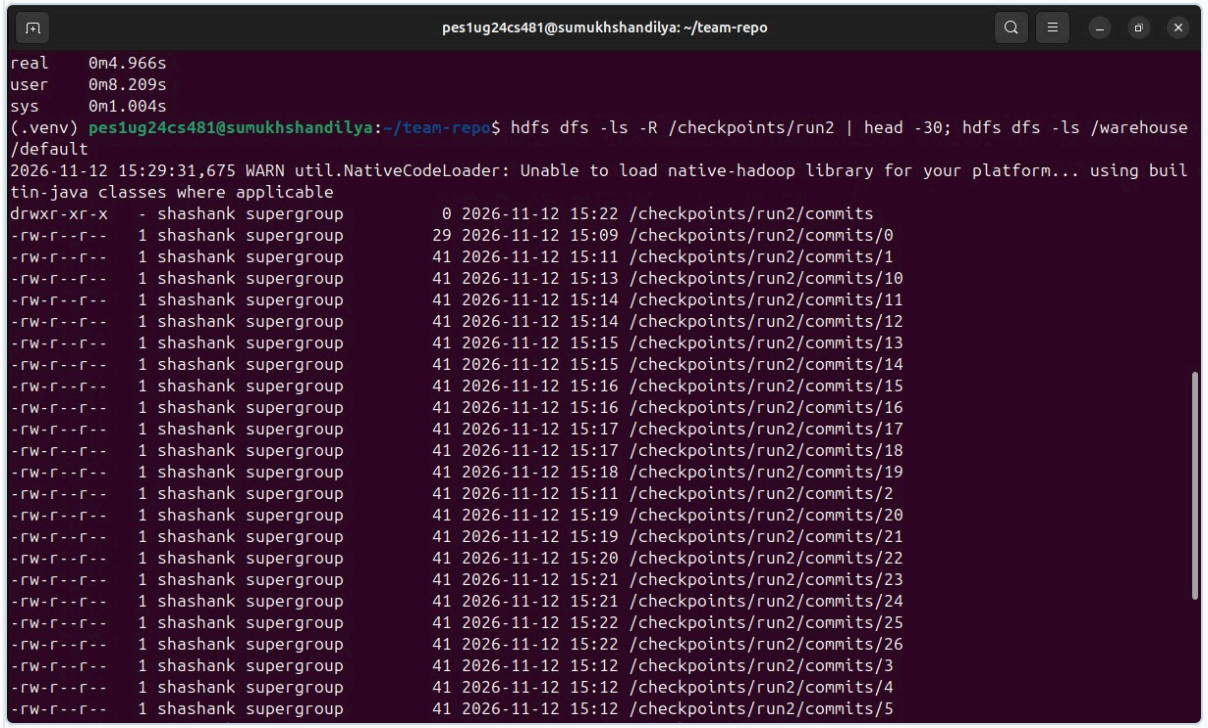
WHY IT MATTERS

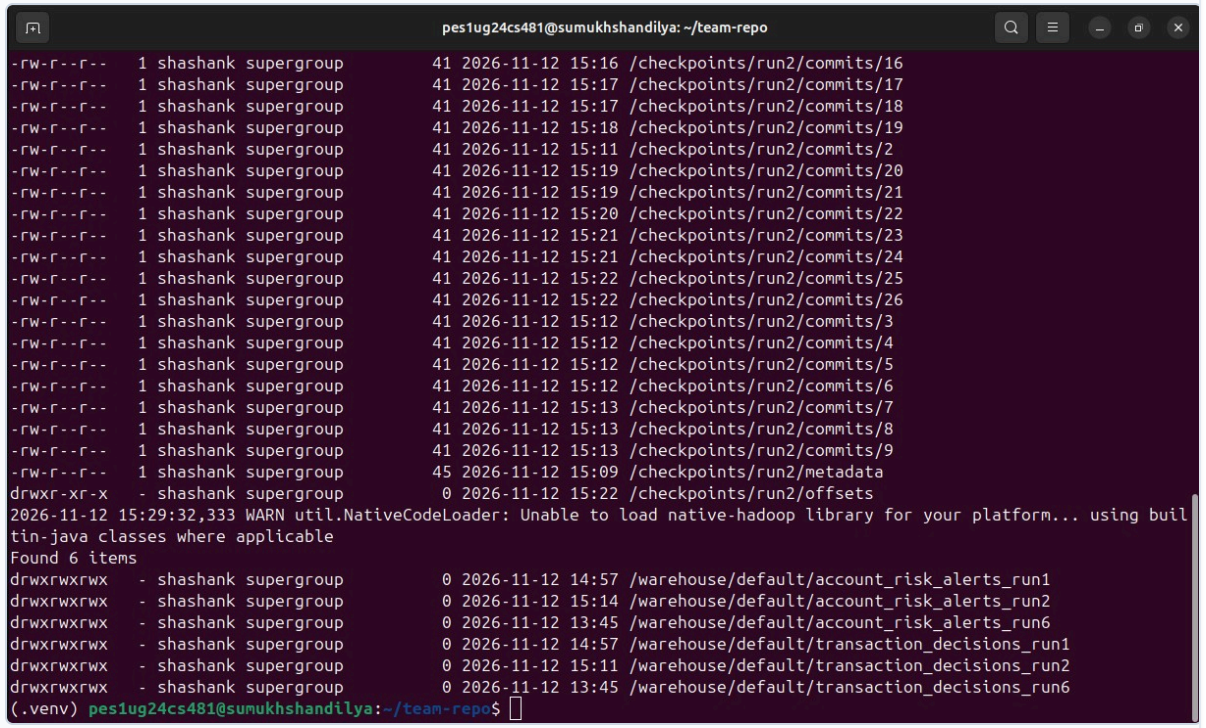
The failure did not change a single decision or alert.

EXPERIMENT 1

HDFS checkpoint after the run

Fig 40 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)





WHAT YOU ARE LOOKING AT

/checkpoints/run2/commits holds commits 0 to 26 (27 consecutive batches) and the warehouse has transaction_decisions_run2 and account_risk_alerts_run2.

WHY IT MATTERS

Every batch is committed once, in order, to the checkpoint on HDFS.

EXPERIMENT 1

Dashboard on run 2

Fig 41 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)



WHAT YOU ARE LOOKING AT

The dashboard in LIVE mode, experiment run 2: 1,990 transactions, 1,659 approved, 231 under review, 100 blocked, and rule counts R1 194, R2 203, R3 66, R4 98, R5 100.

WHY IT MATTERS

The same totals as the baseline: the user-facing view is correct after the failure.

EXPERIMENT 2

Master + driver failure

Run 3

EXPERIMENT 2

Experiment 2: master and driver are stopped mid-batch

Run 3

OBJECTIVE

Stop the Spark master and driver while a batch is being written. Show that, after a restart, the job resumes from the HDFS checkpoint and the result is complete and free of duplicates, while Kafka keeps the data.

WHAT HAPPENED (IST)

21:10	Producer starts (0.3 s per message)
21:13:05	Batch 10 committed. Batch 11 starts (66 records), 592 decisions being written at 21:13:16.
21:13:40	Master stopped. UI refuses connections. The producer keeps sending.
21:16:27	Master and job restarted by hand with the same run id (new application id)
21:16:41	Batch 11 is processed again from the checkpointed offsets and committed at 21:17:11
21:20:56	Batch 17 writes 1,990 decisions and 129 alerts. Batch 18 is empty.
after	Verifier PASS, no duplicates

VERDICT

PASS

KEY NUMBERS

Decisions	**1,990 / 1,990**
Alerts	**129 / 129**
Duplicate decision ids	**0**
Time to resubmit	**2 min 48 s**
First commit after kill	**3 min 31 s**
Checkpoint commits	**0–18 continuous**

WHAT IT DEMONSTRATES

Checkpointing plus idempotent MERGE INTO gives exactly-once results across a full restart. Recovery here is manual (restart master, resubmit job); nothing restarts the driver by itself.

Producer for run 3

Fig 42 · Saatwik · VM1 (Kafka + producer)

```
RESULT: PASS
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$ cd ~/team-repo && source .venv/bin/activate && source ~/fraud/configs/cluster.env
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server $KAFKA_BOOTSTRAP --create --topic transactions-run3 --partitions 1 --replication-factor 1
Created topic transactions-run3.
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$
```

```
Created topic transactions-run3.
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$ cd ~/team-repo && source .venv/bin/activate && source ~/fraud/configs/cluster.env && mkdir -p ~/fraud/logs/run3 && date +%T
python3 vm1/producer.py --mode baseline --topic transactions-run3 --run-id run3 --input data/transactions.csv --bootstrap $KAFKA_BOOTSTRAP --send-delay 0.3 --log-dir ~/frau
/logs/run3
15:40:07
/home/pes1ug24cs509/team-repo/vm1/producer.py:132: DeprecationWarning: key_serializer does not implement kafka.serializer.Serializer
  if __name__ == '__main__': main()
/home/pes1ug24cs509/team-repo/vm1/producer.py:132: DeprecationWarning: value_serializer does not implement kafka.serializer.Serializer
  if __name__ == '__main__': main()
sent 500/2000
```

WHAT YOU ARE LOOKING AT

Fresh topic transactions-run3 created, then the producer is started at 15:40:07 UTC with --send-delay 0.3.

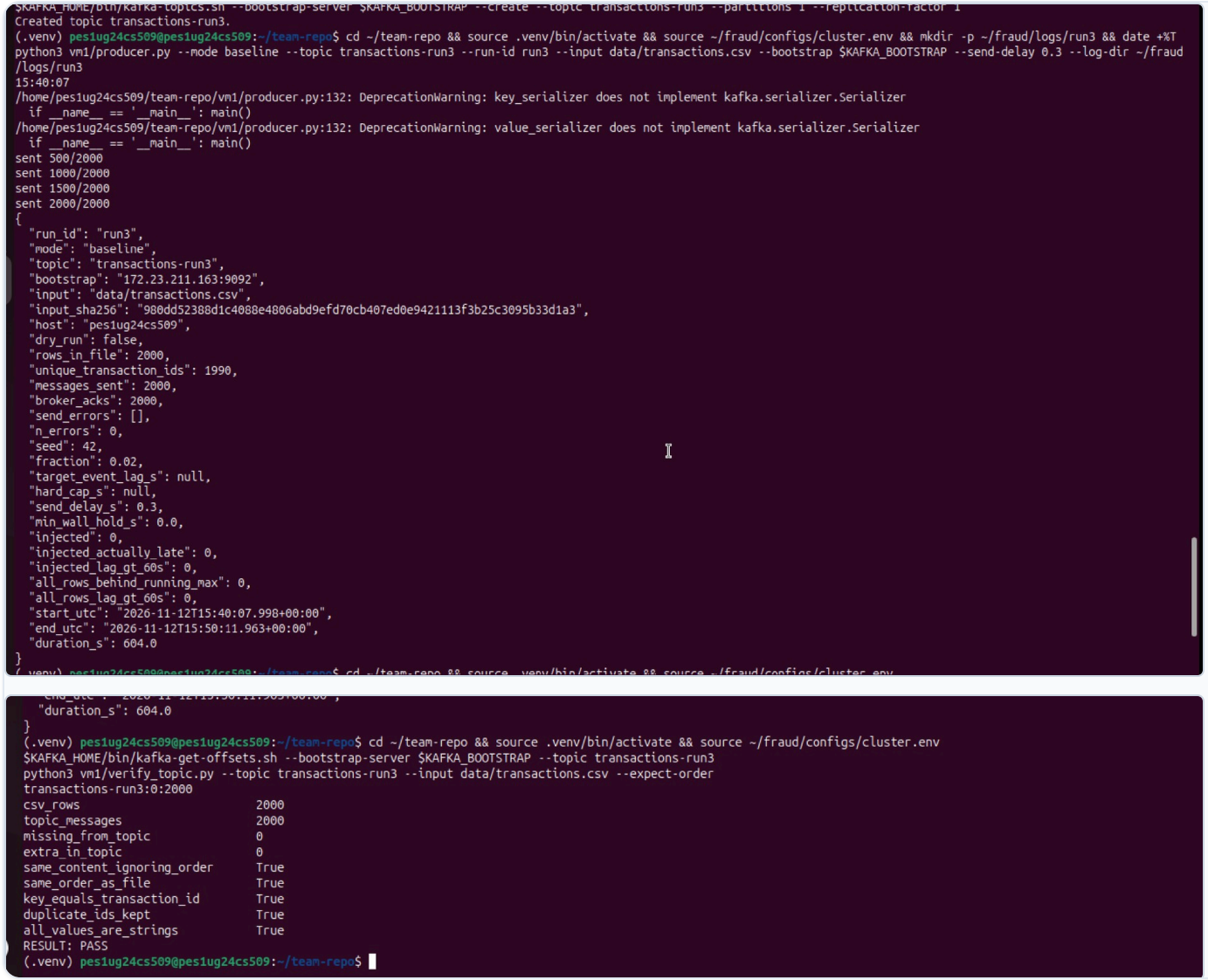
WHY IT MATTERS

Same slow stream as run 2 so the outage happens mid-stream.

EXPERIMENT 2

Producer finished, topic verified

Fig 43 · Saatwik · VM1 (Kafka + producer)



WHAT YOU ARE LOOKING AT

2000 messages sent and acknowledged, 604 s, finished 15:50:11 UTC. Topic verification: 2000 messages, same order, RESULT: PASS.

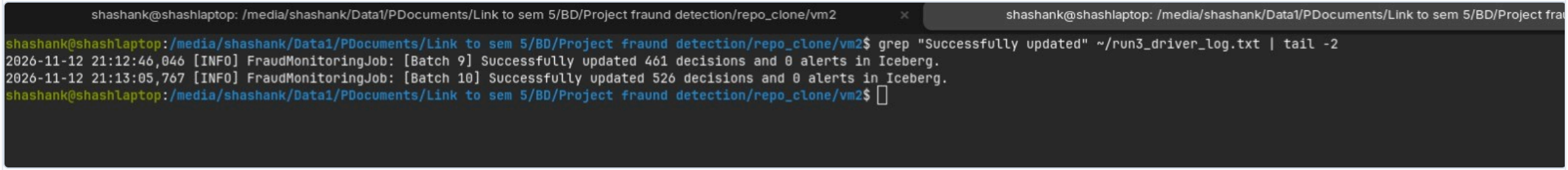
WHY IT MATTERS

The producer did not stop or re-send during the outage. Kafka simply kept the data while Spark was down.

EXPERIMENT 2

Before the failure: last committed batches

Fig 44 · Shashank · VM2 (Spark master + driver)



WHAT YOU ARE LOOKING AT

Driver log shows Batch 9 (461 decisions) and Batch 10 (526 decisions) completed at 21:12:46 and 21:13:05 IST.

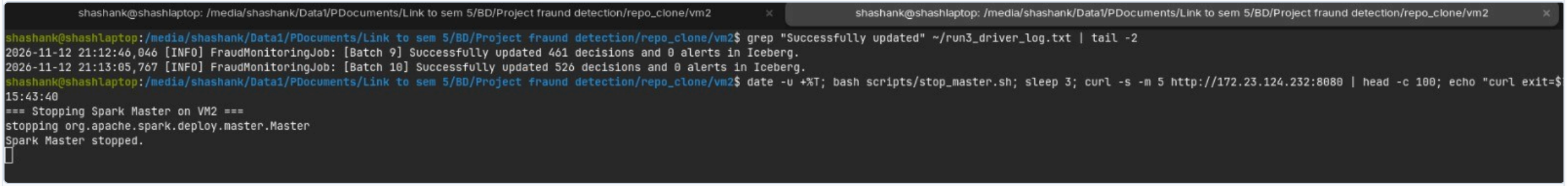
WHY IT MATTERS

These are the last batches that were fully written before the failure.

EXPERIMENT 2

Failure injected: master (and driver) stopped

Fig 45 · Shashank · VM2 (Spark master + driver)



WHAT YOU ARE LOOKING AT

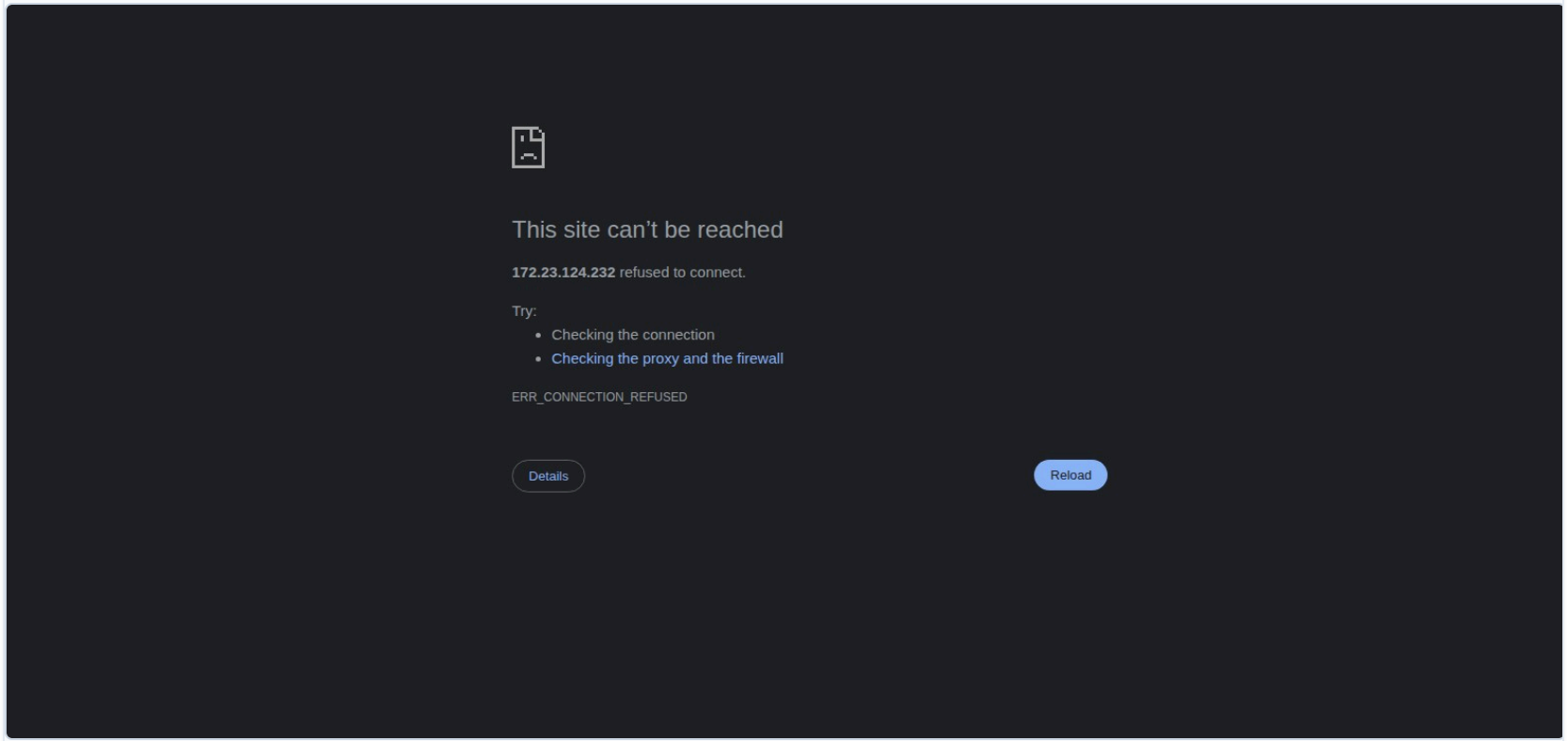
stop_master.sh is run at 15:43:40 UTC (21:13:40 IST): Spark Master stopped. The driver log ends at 21:13:16 IST while batch 11 (66 records, 592 decisions) was still being written.

WHY IT MATTERS

The control plane is gone in the middle of a batch.

Master UI unreachable

Fig 46 · Shashank · VM2 (Spark master + driver)



WHAT YOU ARE LOOKING AT

Browser: 172.23.124.232:8080 refused to connect (ERR_CONNECTION_REFUSED).

WHY IT MATTERS

Proof that the master was really down, not just slow.

EXPERIMENT 2

Recovery: master restarted, job resubmitted

Fig 47 · Shashank · VM2 (Spark master + driver)

Spark 3.3.4 **Spark Master at spark://172.23.124.232:7077**

URL: spark://172.23.124.232:7077
Alive Workers: 2
Cores in use: 4 Total, 4 Used
Memory in use: 4.0 GiB Total, 1024.0 MiB Used
Resources in use:
Applications: 1 Running, 0 Completed
Drivers: 0 Running, 0 Completed
Status: ALIVE

Workers (2)

Worker Id	Address	State	Cores	Memory	Resources
worker-20261112134113-172.23.138.248-33313	172.23.138.248:33313	ALIVE	2 (2 Used)	2.0 GiB (512.0 MiB Used)	
worker-20261112134115-172.23.138.248-38277	172.23.138.248:38277	ALIVE	2 (2 Used)	2.0 GiB (512.0 MiB Used)	

Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20261112211628-0000 (kill)	FraudDetectionEngine-run3	4	512.0 MiB		2026/11/12 21:16:28	shashank	RUNNING	2.9 min

Completed Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration

```
shashank@shashlaptop:/media/shashank/Data1/PDocuments/Link to sem 5/BD/Project fraund detection/repo_clone/vm2$ grep -E "\[Batch" ~/run3_driver_log_resume.txt | grep -v Empty | head -5 | cut -c1-170
2026-11-12 21:16:41,362 [INFO] FraudMonitoringJob: [Batch 11] Processing micro-batch with 66 records.
2026-11-12 21:16:49,931 [INFO] FraudDetectionWriter: [Batch 11] Writing 592 decisions to iceberg.default.transaction_decisions_run3
2026-11-12 21:17:03,548 [INFO] FraudDetectionWriter: [Batch 11] Writing 5 alerts to iceberg.default.account_risk_alerts_run3
2026-11-12 21:17:11,060 [INFO] FraudMonitoringJob: [Batch 11] Successfully updated 592 decisions and 5 alerts in Iceberg.
shashank@shashlaptop:/media/shashank/Data1/PDocuments/Link to sem 5/BD/Project fraund detection/repo_clone/vm2$
```

WHAT YOU ARE LOOKING AT

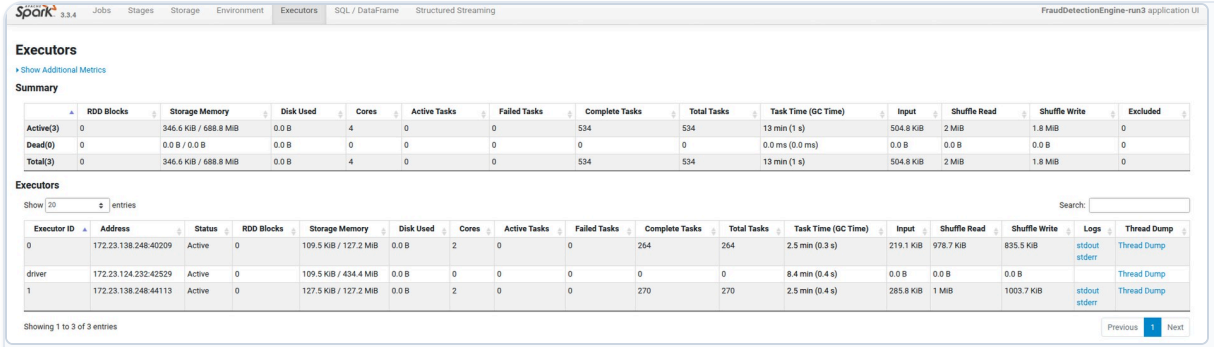
Master page: Alive Workers 2, new application app-...-0000 FraudDetectionEngine-run3 (the application counter reset because the master restarted). The new driver log starts at 21:16:27 with the same run id, topic and HDFS checkpoint. Batch 11 is processed again (66 records, 592 decisions, 5 alerts) and committed at 21:17:11 IST.

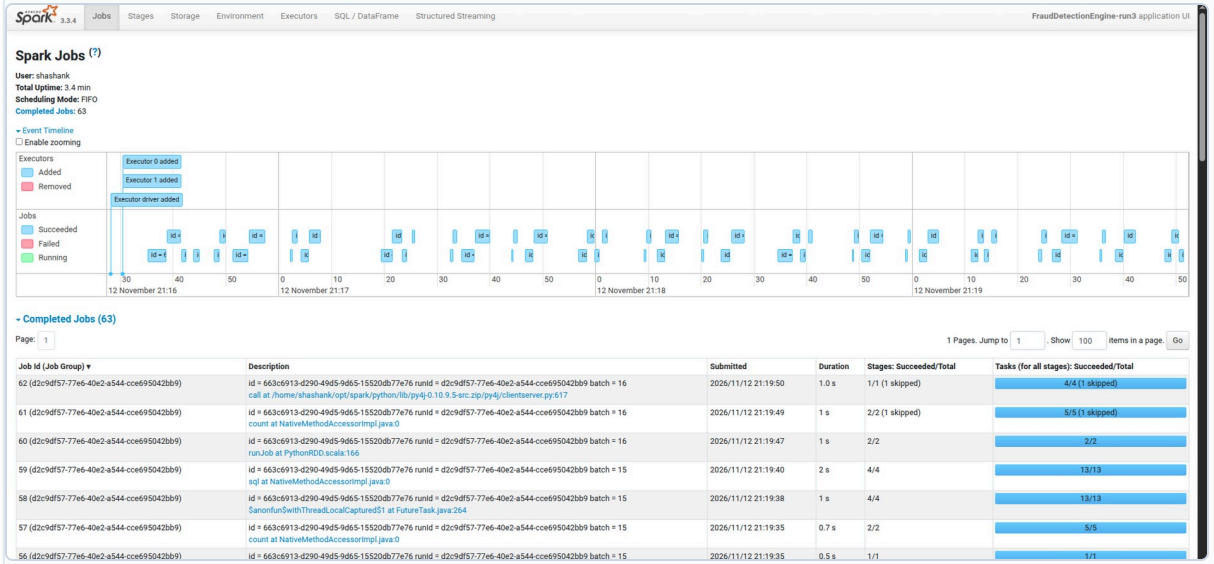
WHY IT MATTERS

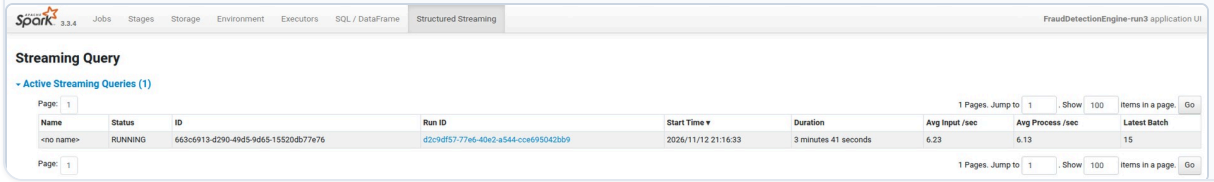
The workers re-registered on their own. The job was resubmitted by hand (about 2 min 48 s after the master stopped) and resumed from the checkpoint at the unfinished batch 11, not from the start. MERGE INTO makes the replay safe.

Recovered cluster: executors, jobs and stream

Fig 48 · Shashank · VM2 (Spark master + driver)







WHAT YOU ARE LOOKING AT

Executors: two active executors (534 tasks completed, 0 failed). Jobs: the resumed application runs jobs continuously. Streaming tab: query RUNNING, latest batch 15 after 3 min 41 s.

WHY IT MATTERS

Normal operation after recovery.

EXPERIMENT 2

Recovery finished: complete output

Fig 49 · Shashank · VM2 (Spark master + driver)



WHAT YOU ARE LOOKING AT

Batch 17 writes 1990 decisions and 129 alerts at 21:20:56 IST, Batch 18 is empty at 21:21:00.

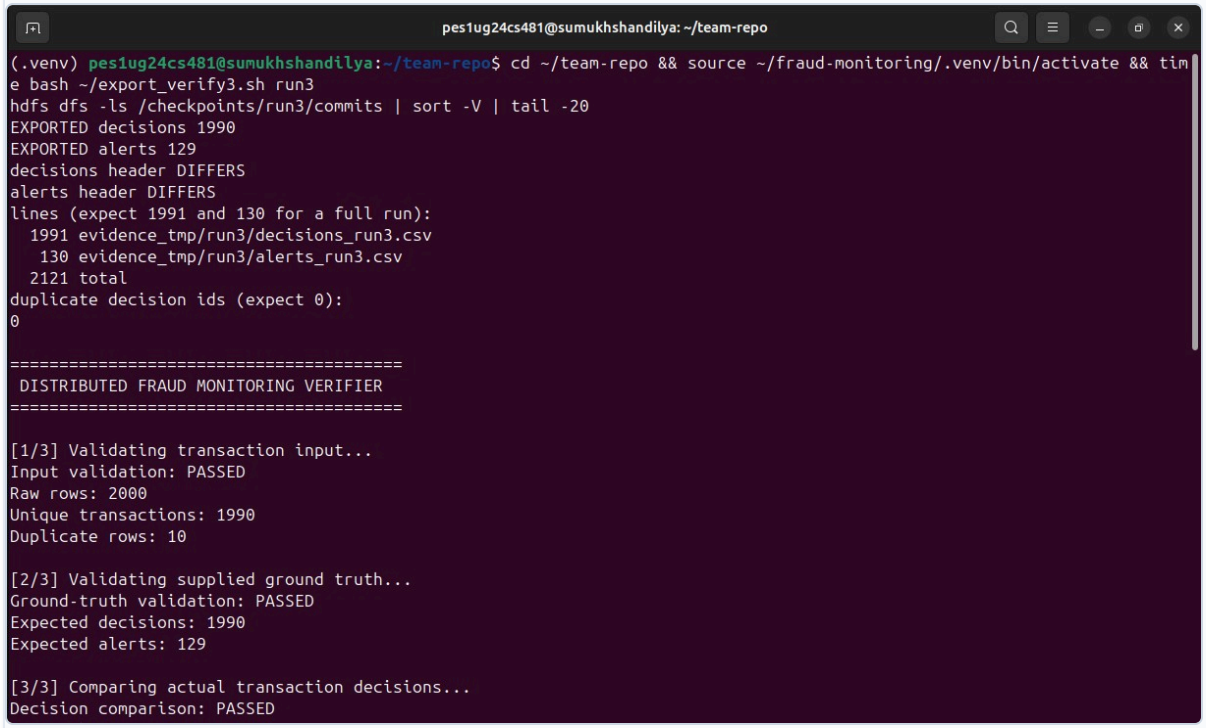
WHY IT MATTERS

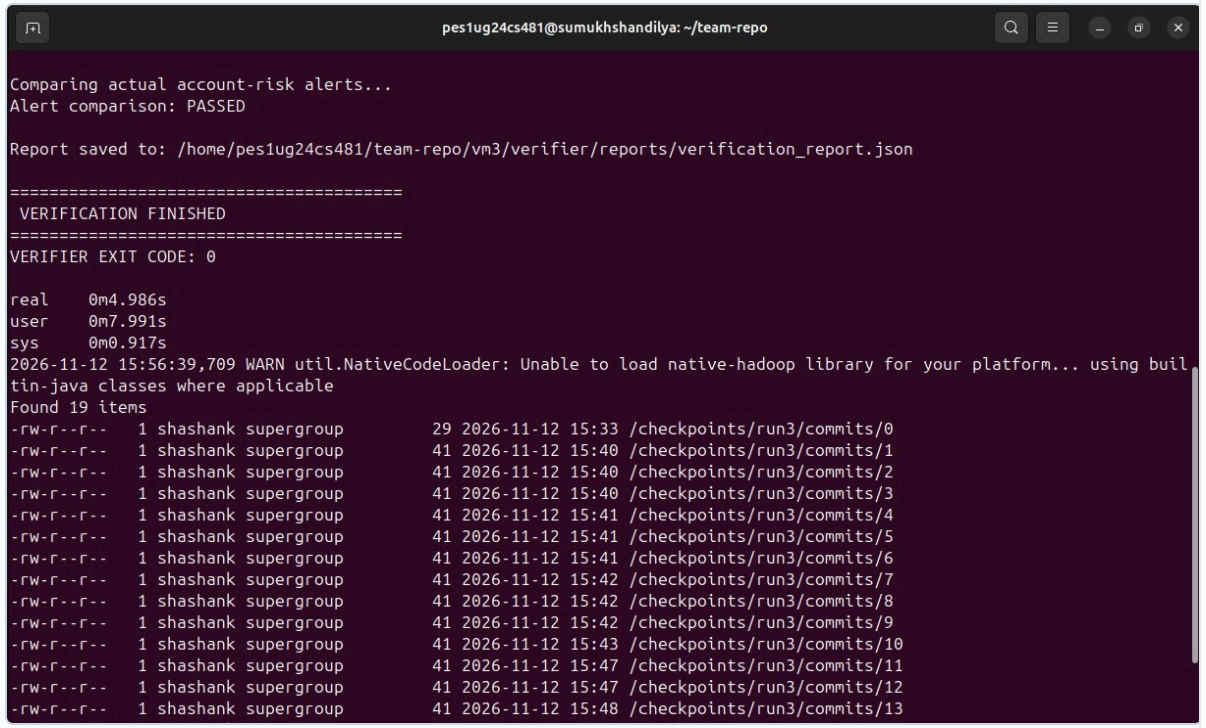
The backlog that built up during the outage was processed and the final state is complete.

EXPERIMENT 2

Verifier: PASS, no duplicates

Fig 50 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)





WHAT YOU ARE LOOKING AT

Decision comparison PASSED (0 mismatches) and alert comparison PASSED. Verifier exit code 0. Exported output: 1990 decisions, 129 alerts, 0 duplicate decision ids.

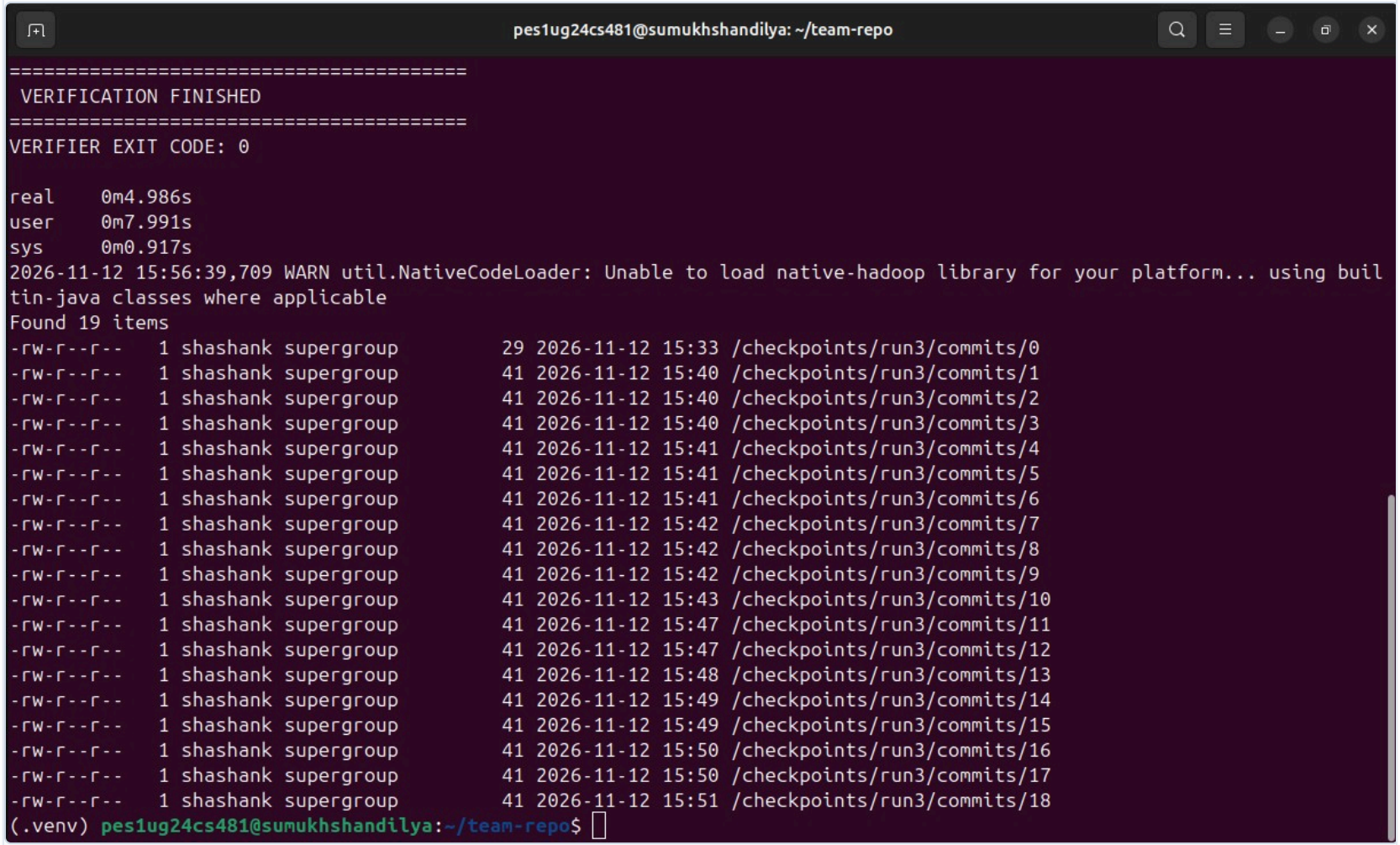
WHY IT MATTERS

Replaying batch 11 after the restart did not create duplicates. This is the idempotent-write guarantee.

EXPERIMENT 2

HDFS checkpoint reused across the restart

Fig 51 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)



WHAT YOU ARE LOOKING AT

/checkpoints/run3/commits: 19 consecutive commits, 0 to 18, written before and after the restart.

WHY IT MATTERS

One checkpoint, one continuous batch sequence. The restarted job continued the old query.

Dashboard on run 3

Fig 52 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)



WHAT YOU ARE LOOKING AT

LIVE mode, experiment run 3: same totals as the baseline (1,990 / 1,659 / 231 / 100; R1 194, R2 203, R3 66, R4 98, R5 100).

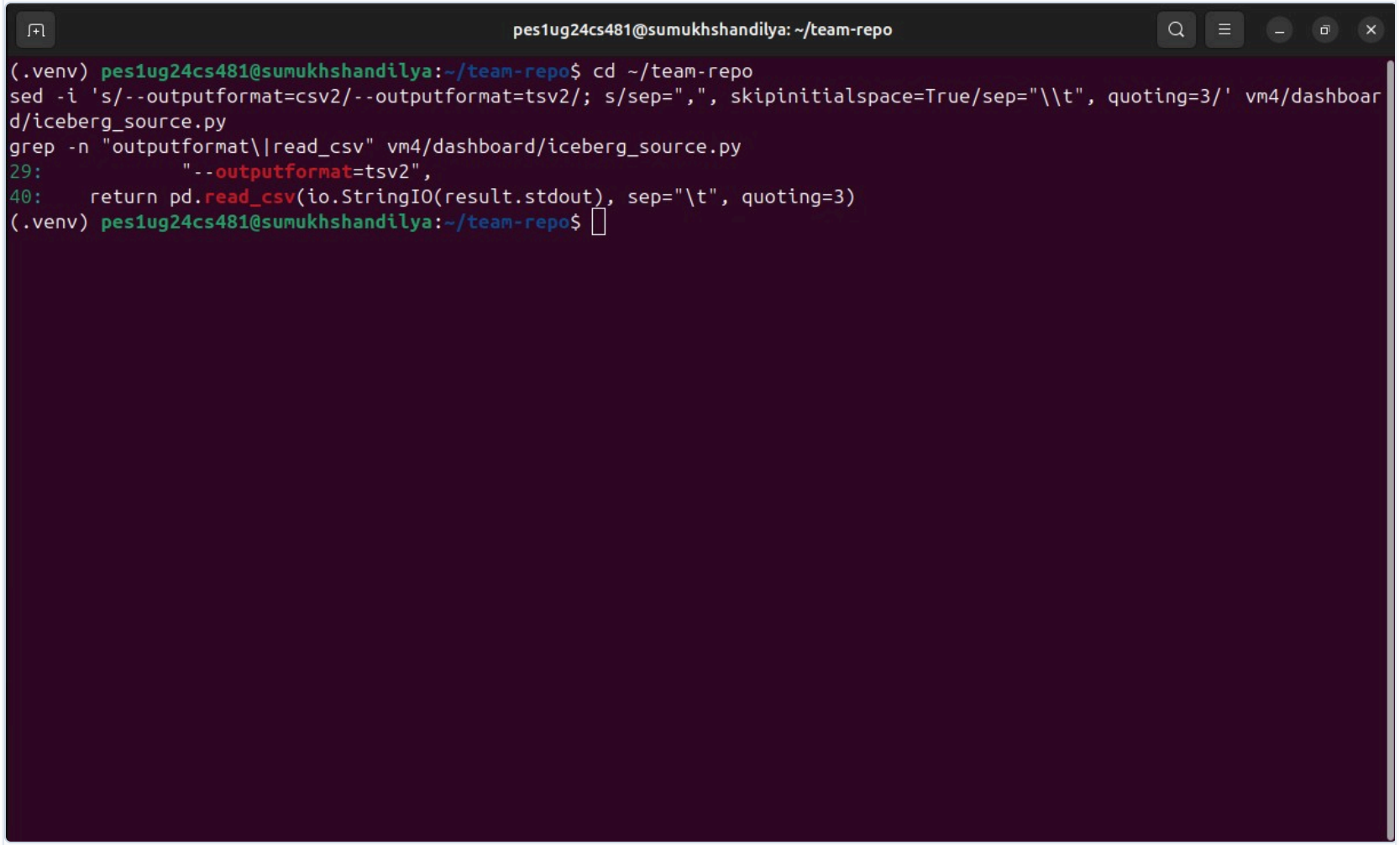
WHY IT MATTERS

The data shown to users is correct after the recovery.

EXPERIMENT 2

Side fix during the outage: dashboard parser

Fig 53 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)



WHAT YOU ARE LOOKING AT

beeline output format changed from csv2 to tsv2 and the reader changed to a tab separator (vm4/dashboard/iceberg_source.py).

WHY IT MATTERS

The first LIVE dashboard failed with Expected 15 fields, saw 16, because values with commas broke CSV parsing. This was a dashboard bug, not part of the experiment, and is fixed.

EXPERIMENT 3

Late data under 1 minute

Run 4

Experiment 3: late data under one minute

OBJECTIVE

Send events that arrive up to a minute after newer events and show that they are still accepted and every result is correct.

WHAT HAPPENED (IST)

21:38	Application run4 starts
21:40:46	Producer starts in late_lt60 mode. 7 rows are held and sent 63 s later.
during	The held rows arrive with an event-time lag of 30 to 48 s, under the 60 s watermark
21:49:50	Producer finishes (544.6 s)
21:50:38	Batch 22 writes 1,990 decisions and 129 alerts
after	Verifier PASS

VERDICT

PASS

KEY NUMBERS

Late rows injected	**7**
Event-time lag	**30–48 s**
Decisions	**1,990 / 1,990**
Alerts	**129 / 129**
Missing / extra / mismatched	**0 / 0 / 0**
Longest batch	**46.5 s**

WHAT IT DEMONSTRATES

Events up to a minute late are accepted: out-of-order arrival does not change any decision or alert. A mistyped run id earlier in the experiment was caught and cleaned up (see the appendix); it did not touch the result.

EXPERIMENT 3

Producer for run 4: inject lateness under 60 s

Fig 54 · Saatwik · VM1 (Kafka + producer)

```
  "injected_actually_late": 7,
  "injected_lag_gt_60s": 0,
  "duration_s": 0.0
Error while executing topic command : Topic 'transactions-run4' already exists.
[2026-11-12 16:06:47,760] ERROR org.apache.kafka.common.errors.TopicExistsException: Topic 'transactions-run4' already exists.
 (org.apache.kafka.tools.TopicCommand)
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$ $KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server $KAFKA_BOOTSTRAP --describe --topic transactions-run4
Topic: transactions-run4        TopicId: UP1zrVTRTDCcIKHa0ebZAQ PartitionCount: 1       ReplicationFactor: 1    Configs:
        Topic: transactions-run4        Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$ cd ~/team-repo && source .venv/bin/activate && source ~/fraud/configs/cluster.env && mkdir -p ~/fraud/logs/run4 && date +%T
python3 vm1/producer.py --mode late_lt60 --topic transactions-run4 --run-id run4 --input data/transactions.csv --bootstrap $KAFKA_BOOTSTRAP --fraction 0.005 --min-wall-hold
63 --log-dir ~/fraud/logs/run4
16:10:45
/home/pes1ug24cs509/team-repo/vm1/producer.py:132: DeprecationWarning: key_serializer does not implement kafka.serializer.Serializer
  if __name__ == '__main__': main()
/home/pes1ug24cs509/team-repo/vm1/producer.py:132: DeprecationWarning: value_serializer does not implement kafka.serializer.Serializer
  if __name__ == '__main__': main()
sent 500/2000
sent 1000/2000
```

WHAT YOU ARE LOOKING AT

topic transactions-run4 is described, then the producer starts in late_lt60 mode: --fraction 0.005 --min-wall-hold 63. The earlier line Topic already exists is from re-running the create command.

WHY IT MATTERS

Selected rows are held back and sent later, so they arrive after newer events. This tests lateness inside the 1-minute watermark.

EXPERIMENT 3

Producer summary: 7 late rows, all under 60 s

Fig 55 · Saatwik · VM1 (Kafka + producer)

```
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$ cd ~/team-repo && source .venv/bin/activate && source ~/fraud/configs/cluster.env && mkdir -p ~/fraud/logs/run4 && date +%T
python3 vm1/producer.py --mode late_lt60 --topic transactions-run4 --run-id run4 --input data/transactions.csv --bootstrap $KAFKA_BOOTSTRAP --fraction 0.005 --min-wall-hold
63 --log-dir ~/fraud/logs/run4
16:10:45
/home/pes1ug24cs509/team-repo/vm1/producer.py:132: DeprecationWarning: key_serializer does not implement kafka.serializer.Serializer
  if __name__ == '__main__': main()
/home/pes1ug24cs509/team-repo/vm1/producer.py:132: DeprecationWarning: value_serializer does not implement kafka.serializer.Serializer
  if __name__ == '__main__': main()
sent 500/2000
sent 1000/2000
sent 1500/2000
sent 2000/2000
{
  "run_id": "run4",
  "mode": "late_lt60",
  "topic": "transactions-run4",
  "bootstrap": "172.23.211.163:9092",
  "input": "data/transactions.csv",
  "input_sha256": "980dd52388d1c4088e4806abd9efd70cb407ed0e9421113f3b25c3095b33d1a3",
  "host": "pes1ug24cs509",
  "dry_run": false,
  "rows_in_file": 2000,
  "unique_transaction_ids": 1990,
  "messages_sent": 2000,
  "broker_acks": 2000,
  "send_errors": [],
  "n_errors": 0,
  "seed": 42,
  "fraction": 0.005,
  "target_event_lag_s": 30.0,
  "hard_cap_s": 59.0,
  "send_delay_s": 0.05,
  "min_wall_hold_s": 63.0,
  "injected": 7,
  "injected_actually_late": 7,
  "injected_lag_gt_60s": 0,
  "all_rows_behind_running_max": 7,
  "all_rows_lag_gt_60s": 0,
  "start_utc": "2026-11-12T16:10:46.163+00:00",
  "end_utc": "2026-11-12T16:19:50.766+00:00",
  "duration_s": 544.6
}
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$
```

WHAT YOU ARE LOOKING AT

late_lt60 mode, target lag 30 s, hard cap 59 s. injected: 7, injected_actually_late: 7, injected_lag_gt_60s: 0. 2000 sent, 2000 acknowledged, 0 errors, 544.6 s (16:10:46 to 16:19:50 UTC).

WHY IT MATTERS

The producer log proves exactly 7 events arrived late and none of them by more than 60 s.

Which rows were late

Fig 56 · Saatwik · VM1 (Kafka + producer)

```
  "duration_s": 544.6
}
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$ ls ~/fraud/logs/run4
head -1 ~/fraud/logs/run4/*arrival*.csv
awk -F, 'NR==1 || $0 ~ /True|true/' ~/fraud/logs/run4/*arrival*.csv | head -12
run4_late_lt60_arrival.csv  run4_late_lt60_summary.json
send_order,file_row,transaction_id,event_id,event_timestamp,injected_delay,arrival_utc,scanned_utc,wall_delay_s,max_event_time_sent_before,event_lag_s
send_order,file_row,transaction_id,event_id,event_timestamp,injected_delay,arrival_utc,scanned_utc,wall_delay_s,max_event_time_sent_before,event_lag_s
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$
```

```
RESULT: PASS
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$ awk -F, 'NR==1 || $9+0>30' ~/fraud/logs/run4/run4_late_lt60_arrival.csv | head -12
send_order,file_row,transaction_id,event_id,event_timestamp,injected_delay,arrival_utc,scanned_utc,wall_delay_s,max_event_time_sent_before,event_lag_s
126,124,TXN0000125,E0000125,2026-10-27T08:35:07Z,1,2026-11-12T16:11:55.770+00:00,2026-11-12T16:10:52.767+00:00,63.002,2026-10-27T08:35:37.000+00:00,30.0
127,127,TXN0000128,E0000128,2026-10-27T08:35:39Z,0,2026-11-12T16:11:55.826+00:00,2026-11-12T16:10:52.873+00:00,62.953,2026-10-27T08:35:37.000+00:00,-2.0
293,290,TXN0000291,E0000291,2026-10-27T09:08:10Z,1,2026-11-12T16:13:07.365+00:00,2026-11-12T16:12:04.359+00:00,63.006,2026-10-27T09:08:43.000+00:00,33.0
294,294,TXN0000295,E0000295,2026-10-27T09:09:16Z,0,2026-11-12T16:13:07.424+00:00,2026-11-12T16:12:04.514+00:00,62.911,2026-10-27T09:08:43.000+00:00,-33.0
371,368,TXN0000369,E0000369,2026-10-27T09:21:42Z,1,2026-11-12T16:14:14.262+00:00,2026-11-12T16:13:11.258+00:00,63.005,2026-10-27T09:22:15.000+00:00,33.0
372,372,TXN0000373,E0000373,2026-10-27T09:22:15Z,0,2026-11-12T16:14:14.317+00:00,2026-11-12T16:13:11.412+00:00,62.905,2026-10-27T09:22:15.000+00:00,0.0
395,394,TXN0000395,E0000395,2026-10-27T09:28:58Z,1,2026-11-12T16:15:18.486+00:00,2026-11-12T16:14:15.485+00:00,63.001,2026-10-27T09:29:44.000+00:00,46.0
396,396,TXN0000397,E0000397,2026-10-27T09:30:11Z,0,2026-11-12T16:15:18.538+00:00,2026-11-12T16:14:15.536+00:00,63.002,2026-10-27T09:29:44.000+00:00,-27.0
689,684,TXN0000684,E0000685,2026-10-27T10:43:14Z,1,2026-11-12T16:16:36.538+00:00,2026-11-12T16:15:33.530+00:00,63.008,2026-10-27T10:43:50.000+00:00,36.0
690,690,TXN0000690,E0000691,2026-10-27T10:43:50Z,0,2026-11-12T16:16:36.598+00:00,2026-11-12T16:15:33.791+00:00,62.807,2026-10-27T10:43:50.000+00:00,0.0
1030,1026,TXN0001023,E0001027,2026-10-27T11:45:38Z,1,2026-11-12T16:17:57.164+00:00,2026-11-12T16:16:54.155+00:00,63.01,2026-10-27T11:46:23.000+00:00,45.0
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$
```

WHAT YOU ARE LOOKING AT

The arrival log is filtered on wall_delay_s > 30. Seven rows: TXN0000125, 291, 369, 395, 684, 1023 and 1953, held 63 s of wall time. Their event times lag the newest event already sent by 30, 33, 33, 46, 36, 45 and 48 s.

WHY IT MATTERS

A concrete list of late events, so the dashboard and verifier can be checked against it.

EXPERIMENT 3

Kafka topic after the late injection

Fig 57 · Saatwik · VM1 (Kafka + producer)

```
send_order,file_row,transaction_id,event_id,event_timestamp,injected_delay,arrival_utc,scanned_utc,wall_delay_s,max_event_time_sent_before,event_lag_s
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$ source ~/fraud/configs/cluster.env
$KAFKA_HOME/bin/kafka-get-offsets.sh --bootstrap-server $KAFKA_BOOTSTRAP --topic transactions-run4
python3 vm1/verify_topic.py --topic transactions-run4 --input data/transactions.csv
transactions-run4:0:2000
csv_rows                        2000
topic_messages                  2000
missing_from_topic              0
extra_in_topic                  0
same_content_ignoring_order     True
same_order_as_file              False
key_equals_transaction_id       True
duplicate_ids_kept              True
all_values_are_strings          True
RESULT: PASS
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$
```

WHAT YOU ARE LOOKING AT

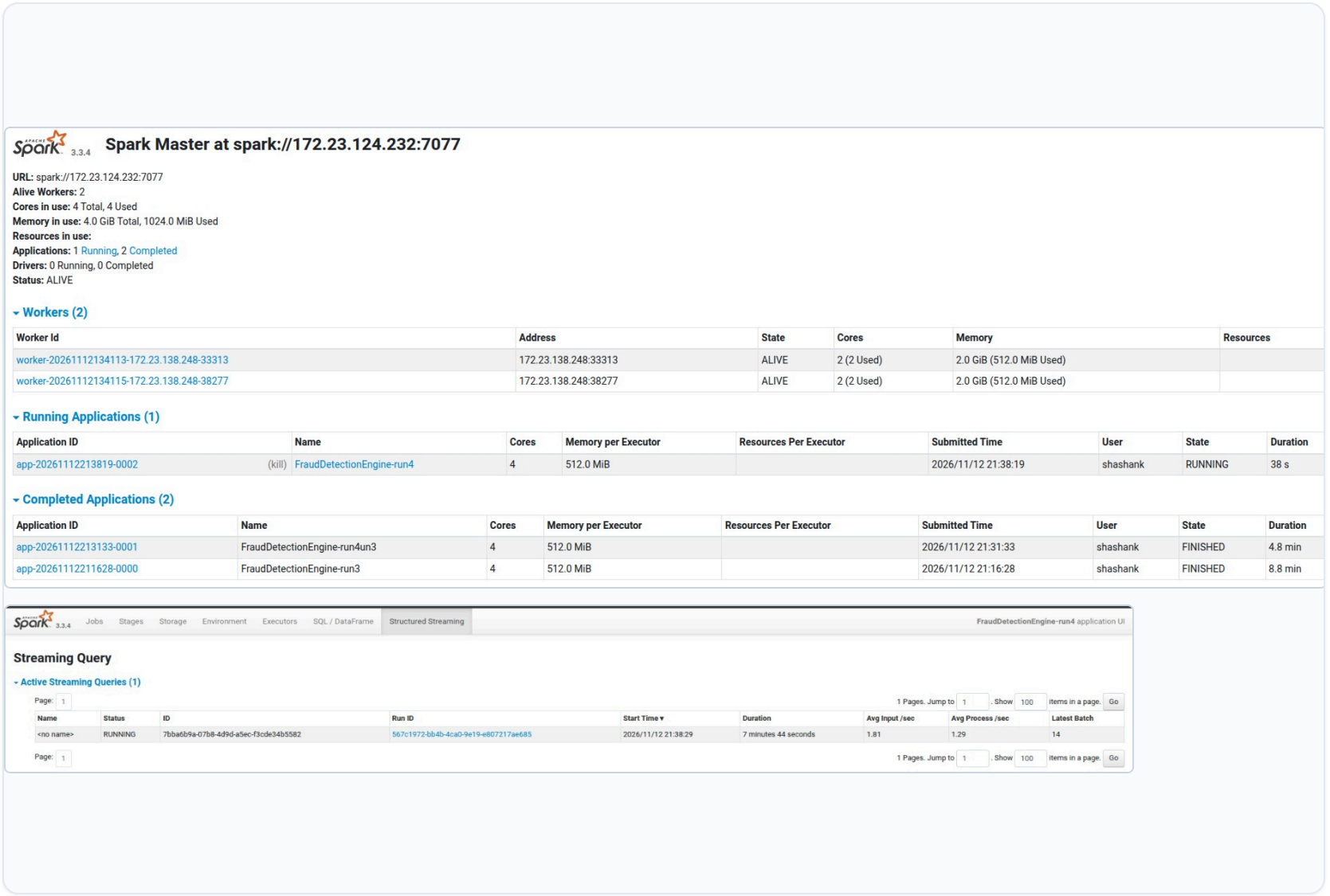
Topic verification without --expect-order: 2000 rows = 2000 messages, same content ignoring order = True, same order as file = False, duplicates kept. RESULT: PASS.

WHY IT MATTERS

Nothing was lost. Only the order changed, which is exactly what the experiment wanted.

Spark running run 4

Fig 58 · Shashank · VM2 (Spark master + driver)



Spark 3.3.4 **Spark Master at spark://172.23.124.232:7077**

URL: spark://172.23.124.232:7077
Alive Workers: 2
Cores in use: 4 Total, 4 Used
Memory in use: 4.0 GiB Total, 1024.0 MiB Used
Resources in use:
Applications: 1 Running, 2 Completed
Drivers: 0 Running, 0 Completed
Status: ALIVE

Workers (2)

Worker Id	Address	State	Cores	Memory	Resources
worker-20261112134113-172.23.138.248-33313	172.23.138.248:33313	ALIVE	2 (2 Used)	2.0 GiB (512.0 MiB Used)	
worker-20261112134115-172.23.138.248-38277	172.23.138.248:38277	ALIVE	2 (2 Used)	2.0 GiB (512.0 MiB Used)	

Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20261112213819-0002 (kill)	FraudDetectionEngine-run4	4	512.0 MiB		2026/11/12 21:38:19	shashank	RUNNING	38 s

Completed Applications (2)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20261112213133-0001	FraudDetectionEngine-run4un3	4	512.0 MiB		2026/11/12 21:31:33	shashank	FINISHED	4.8 min
app-20261112211628-0000	FraudDetectionEngine-run3	4	512.0 MiB		2026/11/12 21:16:28	shashank	FINISHED	8.8 min

Spark 3.3.4 Jobs Stages Storage Environment Executors SQL / DataFrame Structured Streaming — FraudDetectionEngine-run4 application UI

Streaming Query

Active Streaming Queries (1)

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

Name	Status	ID	Run ID	Start Time ▾	Duration	Avg Input /sec	Avg Process /sec	Latest Batch
<no name>	RUNNING	7bba6b9a-07b8-4d9d-a5ec-f3cde34b5582	567c1972-bb4b-4ca0-9e19-e807217ae685	2026/11/12 21:38:29	7 minutes 44 seconds	1.81	1.29	14

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

WHAT YOU ARE LOOKING AT

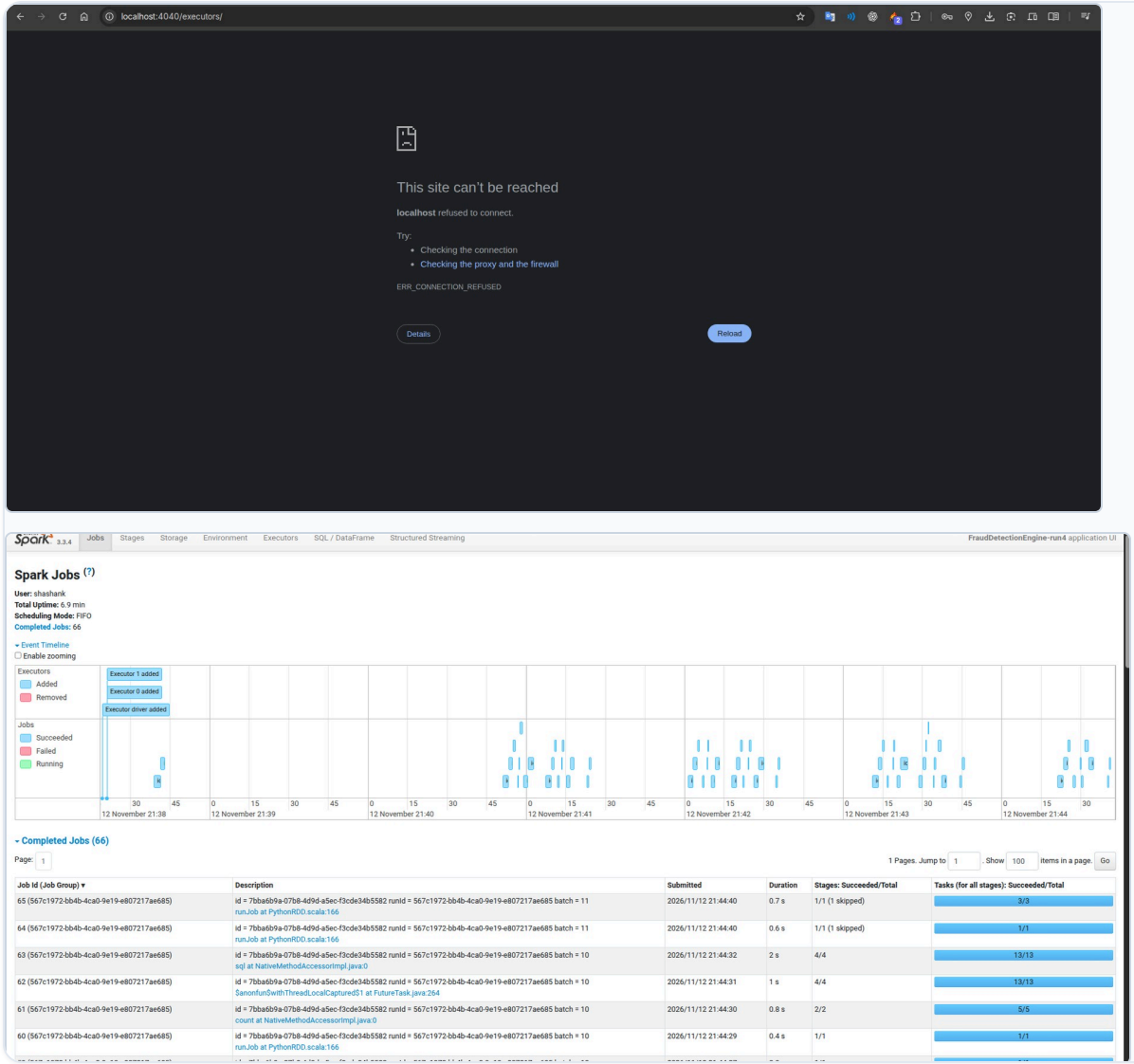
Master: application FraudDetectionEngine-run4 running with 2 workers alive (the stopped run3 application and a stray run4un3 application are listed as completed, see appendix). Streaming tab: one active query, 7 min 44 s, latest batch 14.

WHY IT MATTERS

The job is the same job as before; only the input differs.

Spark jobs during the run

Fig 59 · Shashank · VM2 (Spark master + driver)



WHAT YOU ARE LOOKING AT

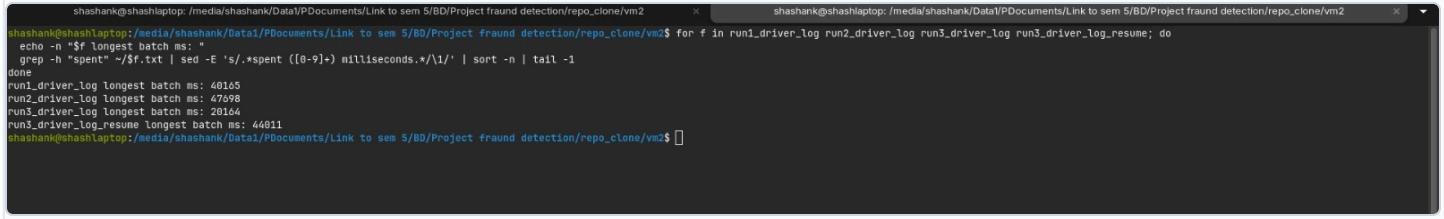
Jobs tab of run 4: 66 completed jobs over about 7 minutes. After the application stopped, its UI at localhost:4040 is no longer reachable, which is normal.

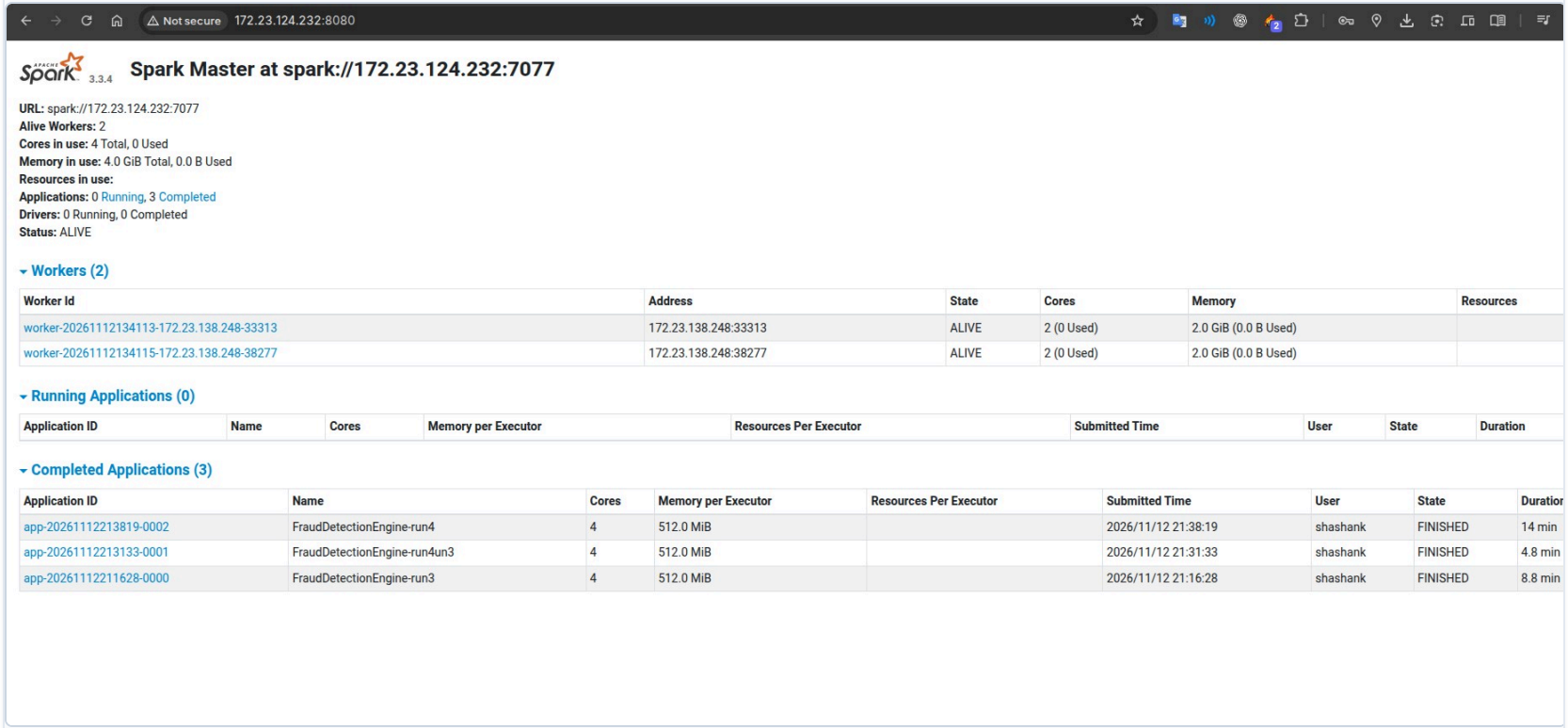
WHY IT MATTERS

Evidence that the job processed the late data like any other batch.

Batch time summary

Fig 60 · Shashank · VM2 (Spark master + driver)





WHAT YOU ARE LOOKING AT

Longest micro-batch per run: run 1 40.2 s, run 2 47.7 s, run 3 20.2 s (before the kill) and 44.0 s (after the resume). Bottom: master page after the run, no running application; run 4 finished after 14 min.

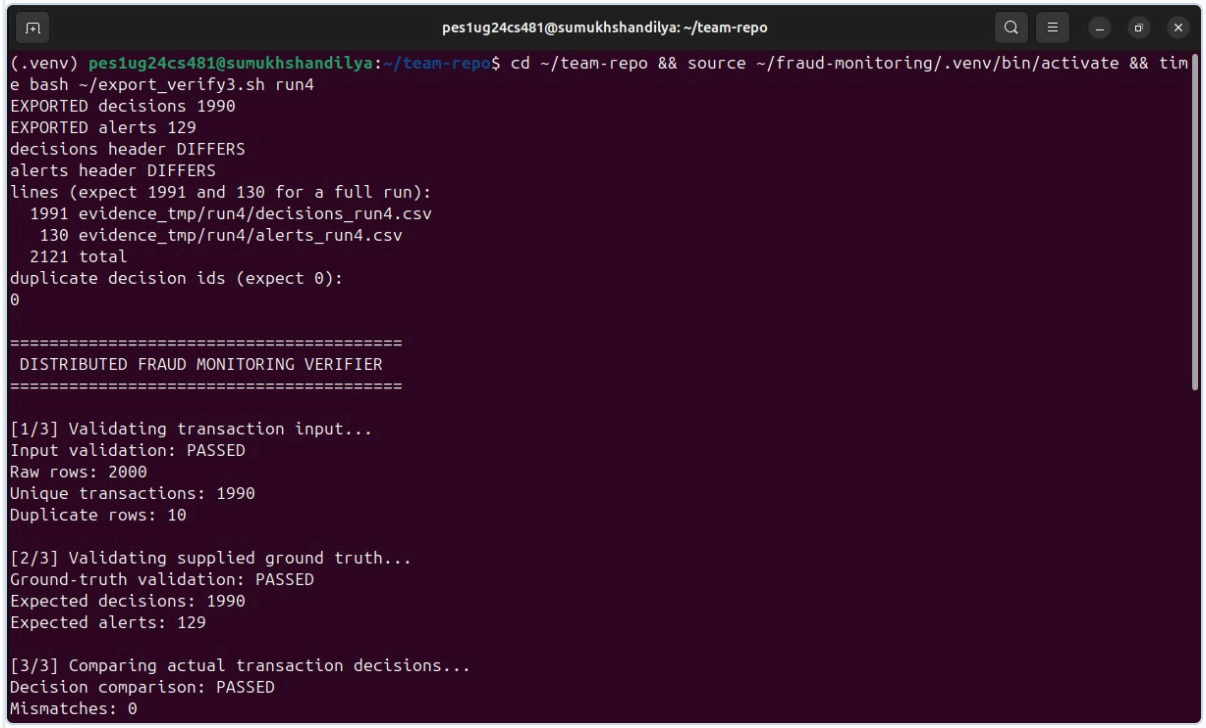
WHY IT MATTERS

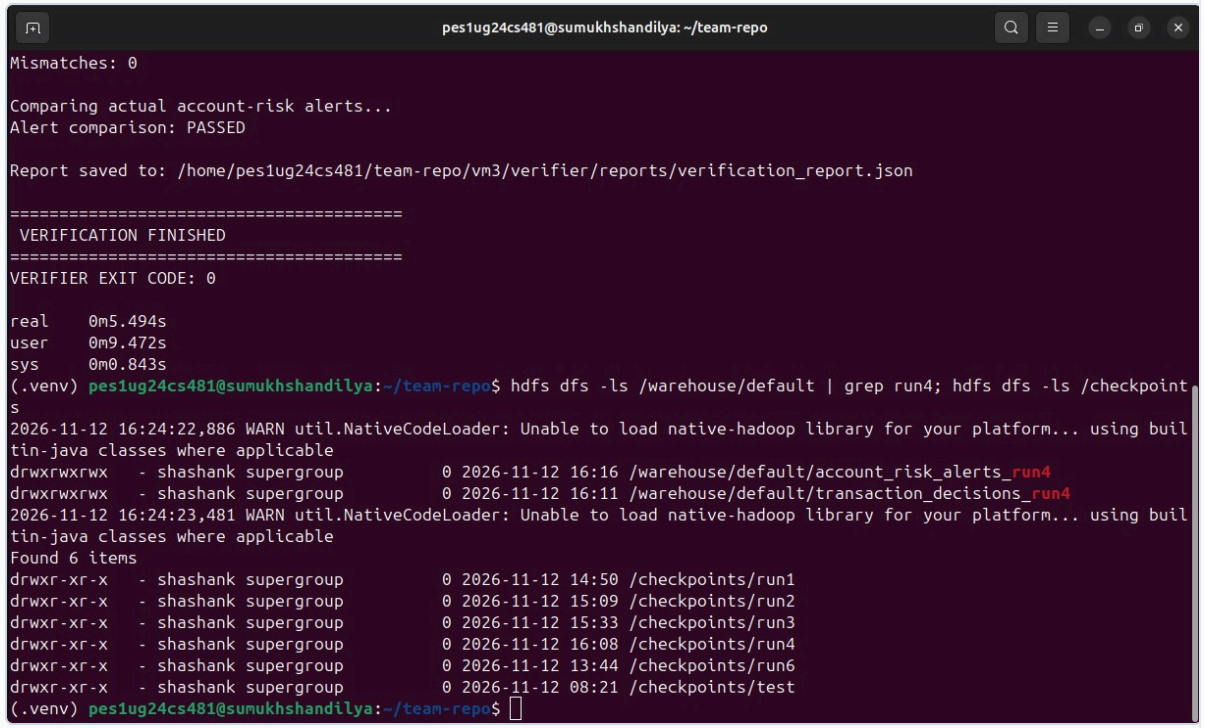
The stream always caught up, but batches are 2 to 5 times longer than the 10 s trigger.

EXPERIMENT 3

Verifier: PASS with late data

Fig 61 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)





WHAT YOU ARE LOOKING AT

All six checks pass: 1990 decisions, 0 mismatches, 129 alerts, exit code 0. The HDFS listing shows transaction_decisions_run4 and account_risk_alerts_run4 and checkpoints for runs 1, 2, 3, 4 and 6.

WHY IT MATTERS

Events up to 48 s late were accepted and every result is correct.

Dashboard on run 4

Fig 62 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)



WHAT YOU ARE LOOKING AT

LIVE mode, experiment run 4: 1,990 transactions, same decision and rule counts as the baseline.

WHY IT MATTERS

The user-facing totals match the baseline.

EXPERIMENT 4

Late data beyond the watermark

Run 5

EXPERIMENT 4

Experiment 4: late data beyond the watermark

OBJECTIVE

Send events that arrive more than a minute late and show exactly what the system does with them.

WHAT HAPPENED (IST)

22:31:02	Application run5 starts
22:31:40	Producer starts in late_gt60 mode. 7 rows held and sent 63 s later.
during	The held rows arrive 93 to 129 s behind the newest event, over the 60 s watermark
22:40:43	Producer finishes (543 s). Kafka holds all 2,000 messages.
22:41:25	Batch 21 writes 1,983 decisions and 129 alerts
after	Verifier: decisions FAIL (7 missing), alerts PASS. Missing ids equal the late ids.

VERDICT

FAIL (expected)

KEY NUMBERS

Late rows injected	**7 (93–129 s)**
Decisions	**1,983 / 1,990**
Missing / extra / mismatched	**7 / 0 / 0**
Alerts	**129 / 129**
Missing ids = late ids	**IDENTICAL**
Dashboard approved	**1,659 → 1,652**

WHAT IT DEMONSTRATES

Rows later than the 1-minute watermark may be dropped, and here all 7 were. The verifier found them exactly and the dashboard reports them as not found. This is a measured limit of the design, from a single run, not a defect.

EXPERIMENT 4

Dry run and fresh topic for run 5

Fig 63 · Saatwik · VM1 (Kafka + producer)

```
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$ cd ~/team-repo && source .venv/bin/activate && source ~/fraud/configs/cluster.env
python3 vm1/producer.py --mode late_gt60 --topic transactions-run5 --run-id run5 --input data/transactions.csv --bootstrap $KAFKA_BOOTSTRAP --fraction 0.005 --log-dir ~/frau
d/logs/dry5 --dry-run 2>/dev/null | grep -E "injected|fraction|target_event|duration"
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server $KAFKA_BOOTSTRAP --create --topic transactions-run5 --partitions 1 --replication-factor 1
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server $KAFKA_BOOTSTRAP --describe --topic transactions-run5
  "fraction": 0.005,
  "target_event_lag_s": 90.0,
  "injected": 7,
  "injected_actually_late": 7,
  "injected_lag_gt_60s": 7,
  "duration_s": 0.0
Created topic transactions-run5.
Topic: transactions-run5        TopicId: nCLgQGyjQa6r0ynxuFsC-A PartitionCount: 1       ReplicationFactor: 1    Configs:
        Topic: transactions-run5        Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
```

WHAT YOU ARE LOOKING AT

A dry run (no Kafka, no sleeping) of the late_gt60 mode predicts: fraction 0.005, target lag 90 s, injected 7, injected_lag_gt_60s 7. Then topic transactions-run5 is created and described.

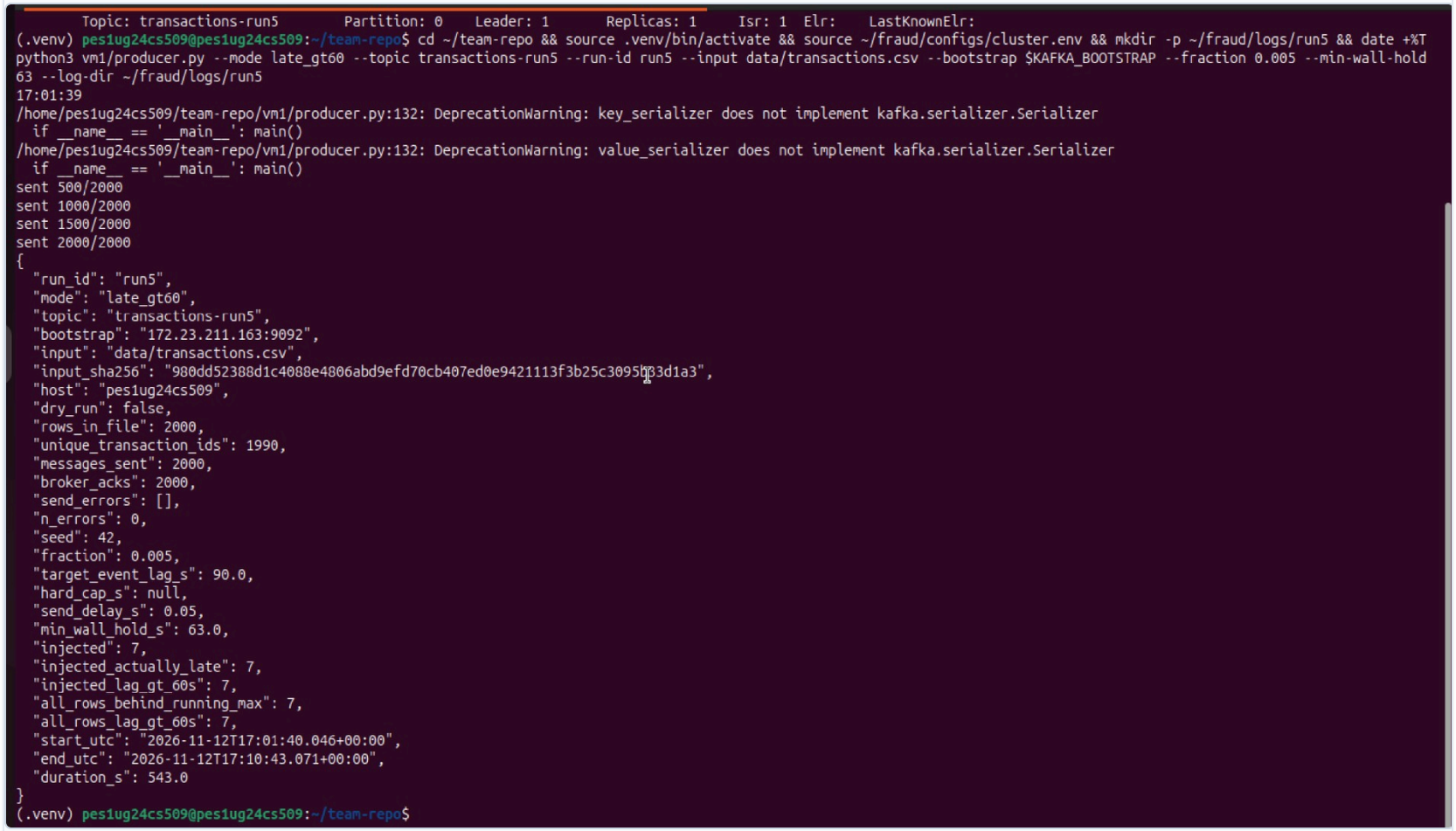
WHY IT MATTERS

The lateness plan was checked before the real run.

EXPERIMENT 4

Producer: late rows beyond the watermark

Fig 64 · Saatwik · VM1 (Kafka + producer)



WHAT YOU ARE LOOKING AT

late_gt60 mode, target lag 90 s. injected: 7, injected_actually_late: 7, injected_lag_gt_60s: 7. 2000 sent, 2000 acknowledged, 0 errors, 543 s, started 17:01:40 UTC.

WHY IT MATTERS

Seven events were sent with an event-time lag larger than the 60 s watermark.

The seven late events

Fig 65 · Saatwik · VM1 (Kafka + producer)

```
}
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$ ls ~/fraud/logs/run5
awk -F, 'NR==1 || $9+0>30' ~/fraud/logs/run5/run5_late_gt60_arrival.csv | head -20
run5_late_gt60_arrival.csv  run5_late_gt60_summary.json
send_order,file_row,transaction_id,event_id,event_timestamp,injected_delay,arrival_utc,scanned_utc,wall_delay_s,max_event_time_sent_before,event_lag_s
131,124,TXN0000125,E0000125,2026-10-27T08:35:07Z,1,2026-11-12T17:02:49.587+00:00,2026-11-12T17:01:46.586+00:00,63.001,2026-10-27T08:36:52.000+00:00,105.0
132,132,TXN0000133,E0000133,2026-10-27T08:37:03Z,0,2026-11-12T17:02:49.638+00:00,2026-11-12T17:01:46.959+00:00,62.679,2026-10-27T08:36:52.000+00:00,-11.0
296,290,TXN0000291,E0000291,2026-10-27T09:08:10Z,1,2026-11-12T17:04:00.892+00:00,2026-11-12T17:02:57.890+00:00,63.002,2026-10-27T09:10:19.000+00:00,129.0
297,297,TXN0000298,E0000298,2026-10-27T09:10:30Z,0,2026-11-12T17:04:00.948+00:00,2026-11-12T17:02:58.200+00:00,62.748,2026-10-27T09:10:19.000+00:00,-11.0
374,368,TXN0000369,E0000369,2026-10-27T09:21:42Z,1,2026-11-12T17:05:07.725+00:00,2026-11-12T17:04:04.718+00:00,63.007,2026-10-27T09:23:40.000+00:00,118.0
375,375,TXN0000376,E0000376,2026-10-27T09:23:52Z,0,2026-11-12T17:05:07.778+00:00,2026-11-12T17:04:05.041+00:00,62.738,2026-10-27T09:23:40.000+00:00,-12.0
399,394,TXN0000395,E0000395,2026-10-27T09:28:58Z,1,2026-11-12T17:06:11.790+00:00,2026-11-12T17:05:08.778+00:00,63.011,2026-10-27T09:30:58.000+00:00,120.0
400,400,TXN0000401,E0000401,2026-10-27T09:31:07Z,0,2026-11-12T17:06:11.866+00:00,2026-11-12T17:05:09.044+00:00,62.822,2026-10-27T09:30:58.000+00:00,-9.0
696,684,TXN0000684,E0000685,2026-10-27T10:43:14Z,1,2026-11-12T17:07:29.766+00:00,2026-11-12T17:06:26.756+00:00,63.01,2026-10-27T10:44:47.000+00:00,93.0
697,697,TXN0000697,E0000698,2026-10-27T10:44:49Z,0,2026-11-12T17:07:29.823+00:00,2026-11-12T17:06:27.381+00:00,62.441,2026-10-27T10:44:47.000+00:00,-2.0
1033,1026,TXN0001023,E0001027,2026-10-27T11:45:38Z,1,2026-11-12T17:08:50.003+00:00,2026-11-12T17:07:46.994+00:00,63.009,2026-10-27T11:47:12.000+00:00,94.0
1034,1034,TXN0001031,E0001035,2026-10-27T11:48:24Z,0,2026-11-12T17:08:50.058+00:00,2026-11-12T17:07:47.356+00:00,62.702,2026-10-27T11:47:12.000+00:00,-72.0
1967,1961,TXN0001953,E0001962,2026-10-27T15:47:19Z,1,2026-11-12T17:10:41.391+00:00,2026-11-12T17:09:38.389+00:00,63.001,2026-10-27T15:48:53.000+00:00,94.0
1968,1968,TXN0001960,E0001969,2026-10-27T15:49:09Z,0,2026-11-12T17:10:41.442+00:00,2026-11-12T17:09:38.692+00:00,62.75,2026-10-27T15:48:53.000+00:00,-16.0
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$
```

```
RESULT: PASS
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$ grep -E "injected_delay|transaction_id" ~/fraud/logs/run5/run5_late_gt60_arrival.csv | head -1
awk -F, '$6==1 {print $3}' ~/fraud/logs/run5/run5_late_gt60_arrival.csv > ~/fraud/logs/run5/late_ids_run5.txt; wc -l ~/fraud/logs/run5/late_ids_run5.txt; cat ~/fraud/logs/ru
n5/late_ids_run5.txt
send_order,file_row,transaction_id,event_id,event_timestamp,injected_delay,arrival_utc,scanned_utc,wall_delay_s,max_event_time_sent_before,event_lag_s
7 /home/pes1ug24cs509/fraud/logs/run5/late_ids_run5.txt
TXN0000125
TXN0000291
TXN0000369
TXN0000395
TXN0000684
TXN0001023
TXN0001953
(.venv) pes1ug24cs509@pes1ug24cs509:~/team-repo$
```

WHAT YOU ARE LOOKING AT

Arrival log rows with delay and the extracted list of late ids saved for later comparison: TXN0000125, 291, 369, 395, 684, 1023, 1953. Event-time lags: 105, 129, 118, 120, 93, 94 and 94 s.

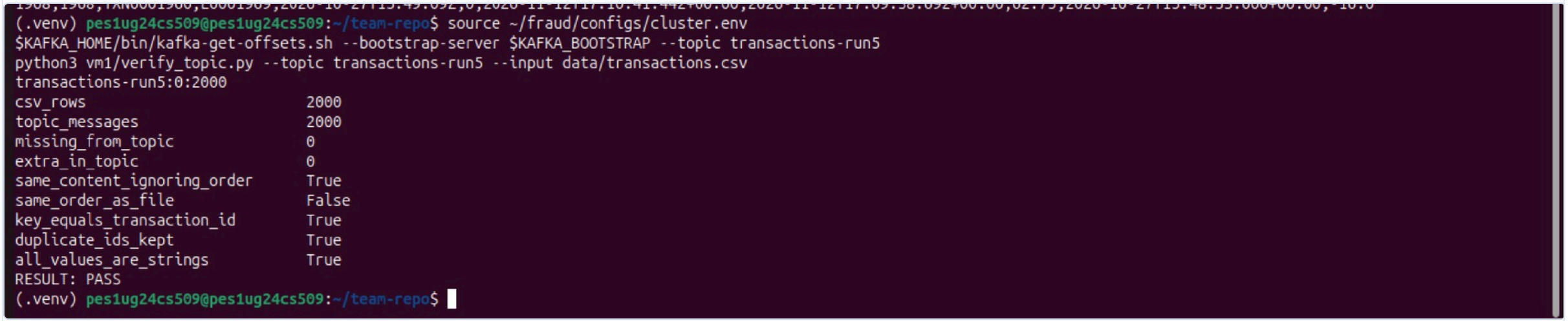
WHY IT MATTERS

The late list is created from the producer log, independently of Spark and of the verifier.

EXPERIMENT 4

Kafka still holds all 2000 messages

Fig 66 · Saatwik · VM1 (Kafka + producer)



WHAT YOU ARE LOOKING AT

verify_topic.py: 2000 rows = 2000 messages, same content ignoring order, same order False. RESULT: PASS.

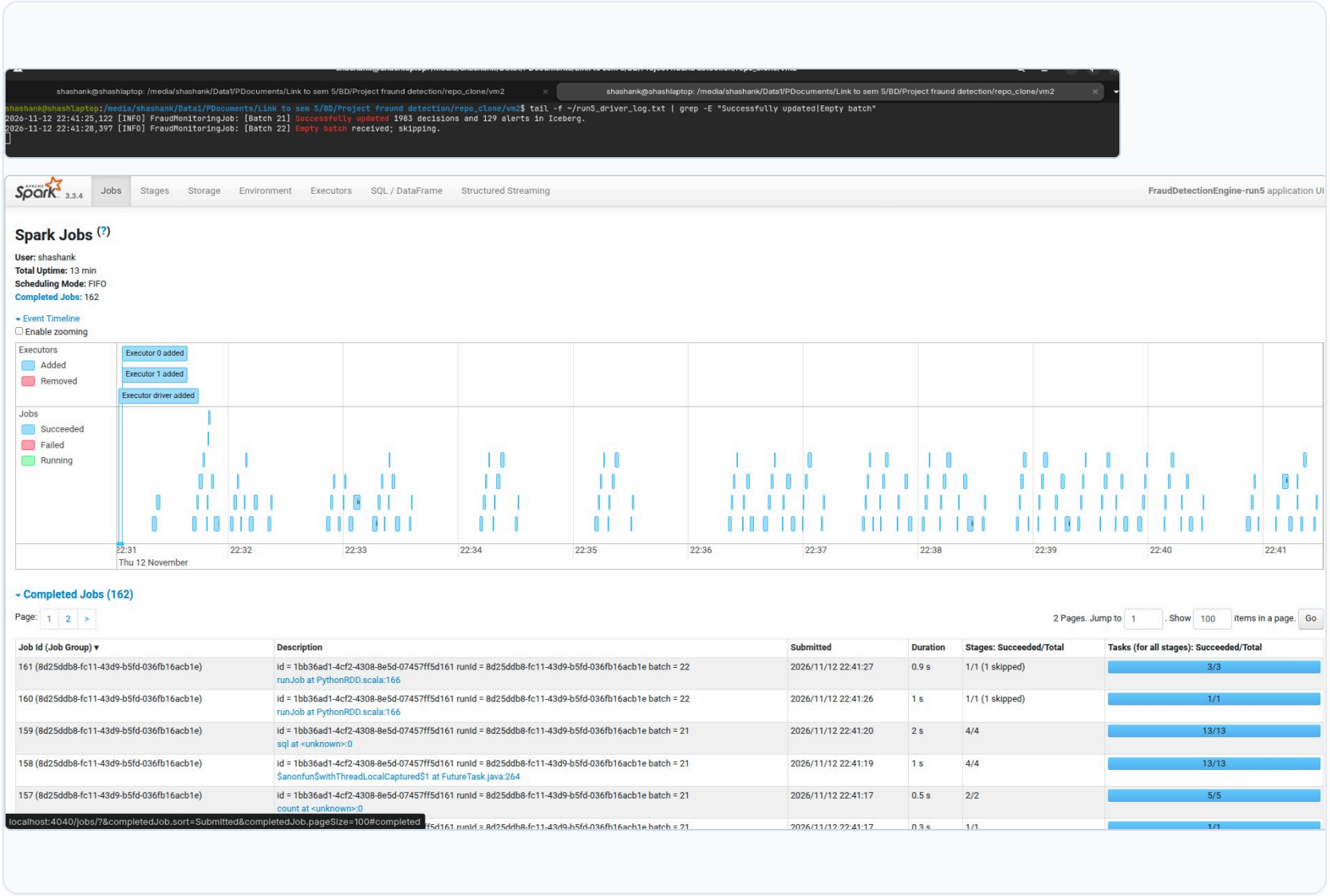
WHY IT MATTERS

The data reached Kafka. Any loss must therefore happen inside Spark.

EXPERIMENT 4

Spark run 5 finishes

Fig 67 · Shashank · VM2 (Spark master + driver)



WHAT YOU ARE LOOKING AT

Driver log: Batch 21 writes 1983 decisions and 129 alerts (22:41:25 IST), Batch 22 is empty. Jobs tab: 162 completed jobs.

WHY IT MATTERS

Spark committed 1983 decisions, which is 7 fewer than the 1990 unique input rows.

EXPERIMENT 4

Verifier: FAIL as expected

Fig 68 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)

```
(.venv) pes1ug24cs481@sumukhshandilya:~/team-repo$ cd ~/team-repo && source ~/fraud-monitoring/.venv/bin/activate && tim
e bash ~/export_verify3.sh run5
EXPORTED decisions 1983
EXPORTED alerts 129
decisions header DIFFERS
alerts header DIFFERS
lines (expect 1991 and 130 for a full run):
  1984 evidence_tmp/run5/decisions_run5.csv
   130 evidence_tmp/run5/alerts_run5.csv
  2114 total
duplicate decision ids (expect 0):
0

========================================
 DISTRIBUTED FRAUD MONITORING VERIFIER
========================================

[1/3] Validating transaction input...
Input validation: PASSED
Raw rows: 2000
Unique transactions: 1990
Duplicate rows: 10

[2/3] Validating supplied ground truth...
Ground-truth validation: PASSED
Expected decisions: 1990
Expected alerts: 129

[3/3] Comparing actual transaction decisions...
Decision comparison: FAILED
Mismatches: 0
```

```
========================================

[1/3] Validating transaction input...
Input validation: PASSED
Raw rows: 2000
Unique transactions: 1990
Duplicate rows: 10

[2/3] Validating supplied ground truth...
Ground-truth validation: PASSED
Expected decisions: 1990
Expected alerts: 129

[3/3] Comparing actual transaction decisions...
Decision comparison: FAILED
Mismatches: 0

Comparing actual account-risk alerts...
Alert comparison: PASSED

Report saved to: /home/pes1ug24cs481/team-repo/vm3/verifier/reports/verification_report.json

========================================
 VERIFICATION FINISHED
========================================
VERIFIER EXIT CODE: 1

real    0m5.190s
user    0m8.527s
sys     0m0.966s
(.venv) pes1ug24cs481@sumukhshandilya:~/team-repo$
```

WHAT YOU ARE LOOKING AT

Input and ground-truth validation pass. Decision comparison FAILED (0 value mismatches). Alert comparison PASSED. Verifier exit code 1. The exported output has 1983 decisions and 129 alerts.

WHY IT MATTERS

The verifier detects the loss. No decision is wrong; some are missing.

EXPERIMENT 4

Exactly the late events are missing

Fig 69 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)

```
(.venv) pes1ug24cs481@sumukhshandilya:~/team-repo$ cd ~/team-repo
cut -d, -f1 evidence_tmp/run5/decisions_run5.csv | tail -n +2 | tr -d '\r"' | sort > /tmp/actual_ids.txt
cut -d, -f1 data/ground_truth_transaction_decisions.csv | tail -n +2 | tr -d '\r"' | sort > /tmp/gt_ids.txt
echo "MISSING from run5 (in ground truth, not in actual):"; comm -13 /tmp/actual_ids.txt /tmp/gt_ids.txt | tee /tmp/miss
ing_ids.txt
echo "COUNT:"; wc -l < /tmp/missing_ids.txt
printf 'TXN0000125\nTXN0000291\nTXN0000369\nTXN0000395\nTXN0000684\nTXN0001023\nTXN0001953\n' | sort > /tmp/late_ids.txt

echo "DIFFERENCE vs producer's late list (empty = identical):"; diff /tmp/missing_ids.txt /tmp/late_ids.txt && echo "IDE
NTICAL"
MISSING from run5 (in ground truth, not in actual):
TXN0000125
TXN0000291
TXN0000369
TXN0000395
TXN0000684
TXN0001023
TXN0001953
COUNT:
7
DIFFERENCE vs producer's late list (empty = identical):
IDENTICAL
(.venv) pes1ug24cs481@sumukhshandilya:~/team-repo$ ls evidence_tmp/run5/
python3 -c "import json; d=json.load(open('vm3/verifier/reports/verification_report.json')); print(json.dumps(d, indent=
1)[:3000])"
alerts_run5.csv  decisions_run5.csv  verification_report_run5.json  verification_run5.txt
{
 "input_validation": {
  "status": "PASSED",
  "total_input_rows": 2000,
  "unique_transactions": 1990,
```

WHAT YOU ARE LOOKING AT

ids in the ground truth but not in the output: TXN0000125, 291, 369, 395, 684, 1023 and 1953 (count 7). A diff against the producer's late list prints IDENTICAL.

WHY IT MATTERS

This links cause and effect: the 7 missing decisions are the 7 events that arrived more than 60 s late.

Verification report (JSON), part 1

Fig 70 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)


```
  "duplicate_rows": 10,
  "duplicate_transaction_ids": [
   "TXN0000606",
   "TXN0000791",
   "TXN0000968",
   "TXN0000991",
   "TXN0001050",
   "TXN0001078",
   "TXN0001775",
   "TXN0001817",
   "TXN0001858",
   "TXN0001987"
  ],
  "duplicate_conflicts": [],
  "invalid_amounts": [],
  "invalid_timestamps": [],
  "currency_distribution": {
   "INR": 1986,
   "USD": 2,
   "": 2
  },
  "errors": [],
  "warnings": []
 },
 "ground_truth_validation": {
  "status": "PASSED",
  "decision_count": 1990,
  "alert_count": 129,
  "decision_distribution": {
   "APPROVE": 1659,
   "REVIEW": 231,
  },
  "rule_trigger_distribution": {
   "r1": 194,
   "r2": 203,
   "r3": 66,
   "r4": 98
  },
  "alert_rule_distribution": {
   "R2": 90,
   "R4": 25,
   "R3": 14
  },
  "alert_scope_distribution": {
   "WINDOW": 104,
   "PATTERN": 25
  },
  "missing_decisions": [],
  "extra_decisions": [],
  "duplicate_decisions": [],
  "r5_consistency_errors": [],
  "missing_alert_supporting_ids": [],
  "errors": []
 },
 "decision_comparison": {
  "expected_count": 1990,
  "actual_count": 1983,
  "missing_count": 7,
  "extra_count": 0,
  "missing_ids": [
   "TXN0000125",
   "TXN0000291",
```


WHAT YOU ARE LOOKING AT

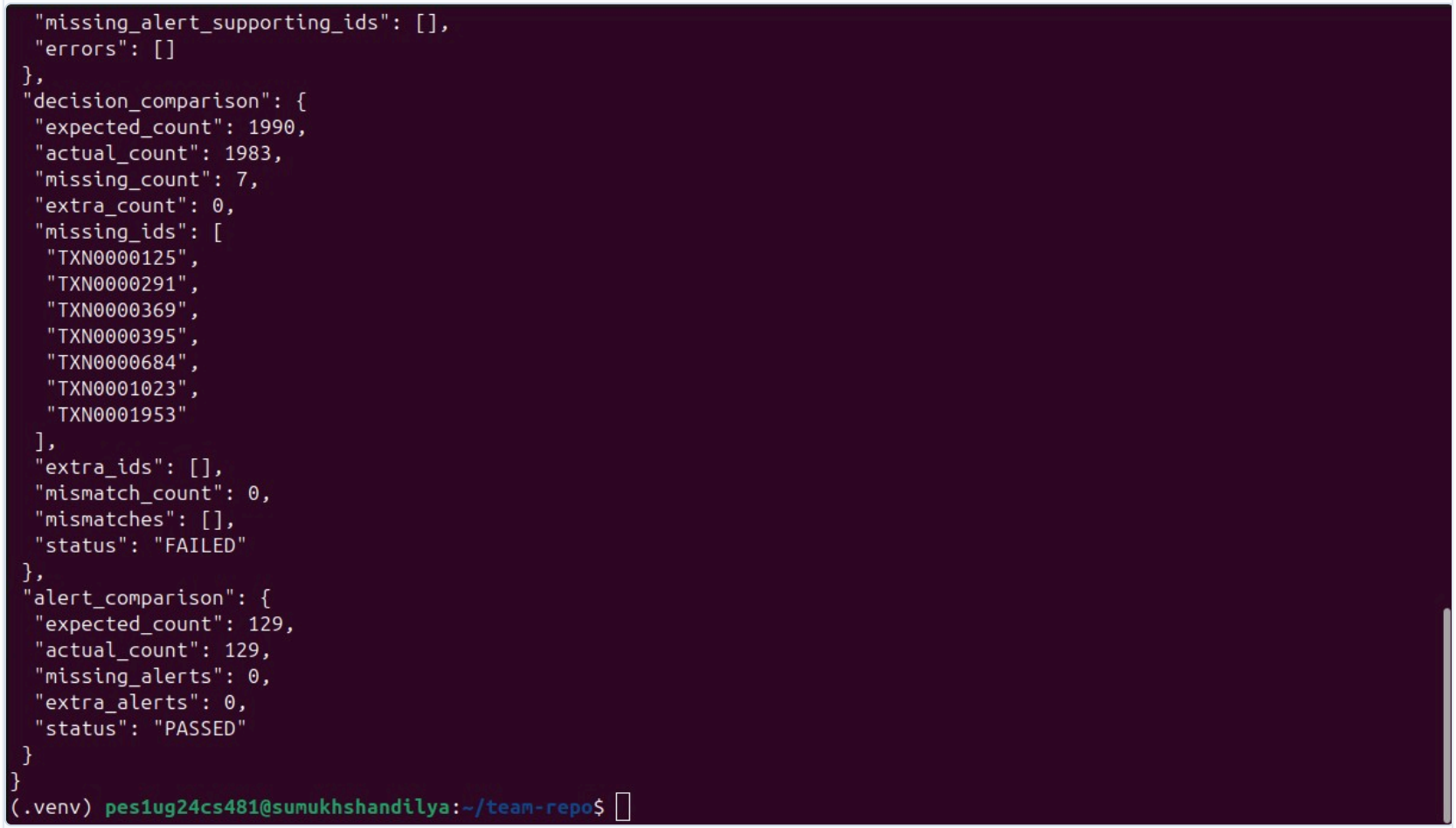
Input validation PASSED (2000 rows, 1990 unique, 10 duplicate ids listed, no conflicts, currency INR 1986, USD 2, blank 2). Ground truth: 1990 decisions (APPROVE 1659, REVIEW 231, BLOCK 100), triggers R1 194, R2 203, R3 66, R4 98, alert rules R2 90, R4 25, R3 14.

WHY IT MATTERS

The reference the output is compared with.

Verification report (JSON), part 2

Fig 71 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)



```
  "missing_alert_supporting_ids": [],
  "errors": []
 },
 "decision_comparison": {
  "expected_count": 1990,
  "actual_count": 1983,
  "missing_count": 7,
  "extra_count": 0,
  "missing_ids": [
   "TXN0000125",
   "TXN0000291",
   "TXN0000369",
   "TXN0000395",
   "TXN0000684",
   "TXN0001023",
   "TXN0001953"
  ],
  "extra_ids": [],
  "mismatch_count": 0,
  "mismatches": [],
  "status": "FAILED"
 },
 "alert_comparison": {
  "expected_count": 129,
  "actual_count": 129,
  "missing_alerts": 0,
  "extra_alerts": 0,
  "status": "PASSED"
 }
}
(.venv) pes1ug24cs481@sumukhshandilya:~/team-repo$
```

WHAT YOU ARE LOOKING AT

decision_comparison: expected 1990, actual 1983, missing_count 7, extra_count 0, mismatch_count 0, FAILED. alert_comparison: expected 129, actual 129, missing 0, extra 0, PASSED.

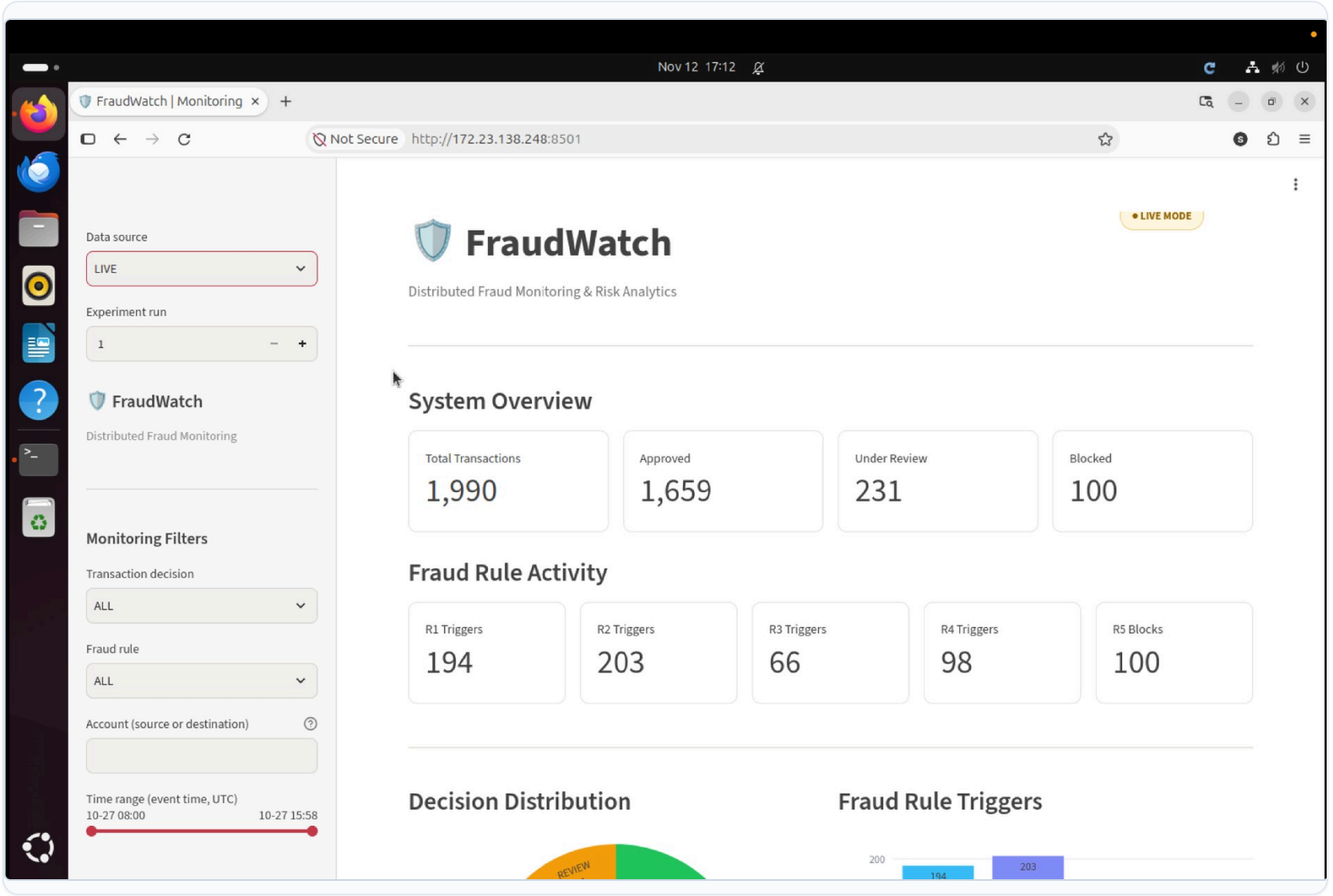
WHY IT MATTERS

All the information needed to explain the failure is in the machine-readable report.

EXPERIMENT 4

Reference: dashboard on run 1 (before)

Fig 72 · Saatwik · VM1 (Kafka + producer)



WHAT YOU ARE LOOKING AT

Dashboard in LIVE mode, experiment run 1: 1,990 transactions, 1,659 approved, 231 under review, 100 blocked.

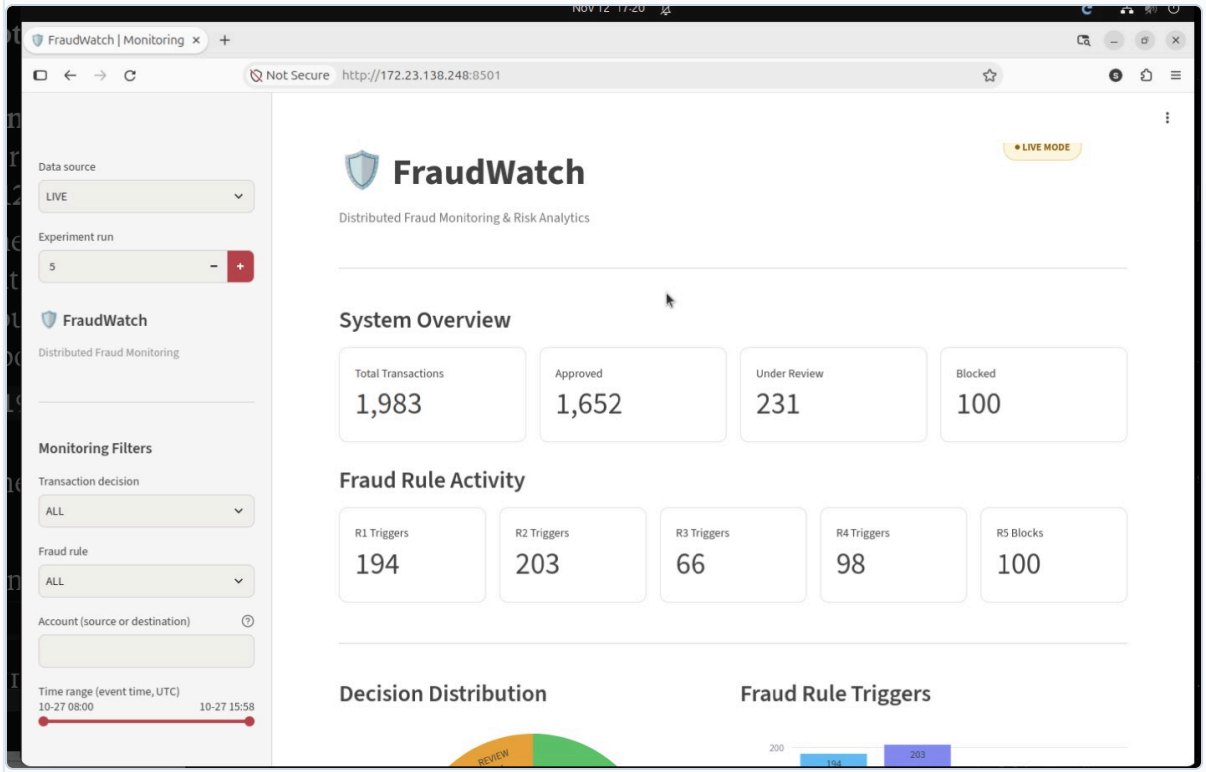
WHY IT MATTERS

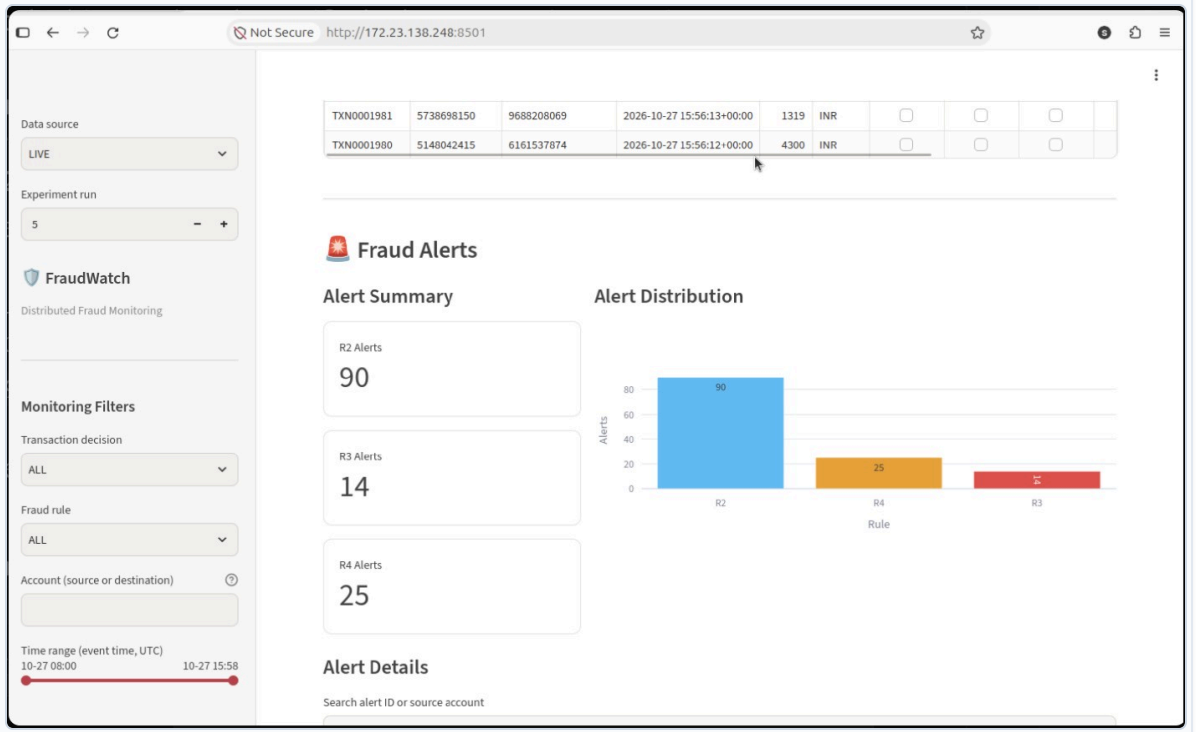
Baseline totals taken just before opening run 5.

EXPERIMENT 4

Dashboard on run 5

Fig 73 · Saatwik · VM1 (Kafka + producer)





WHAT YOU ARE LOOKING AT

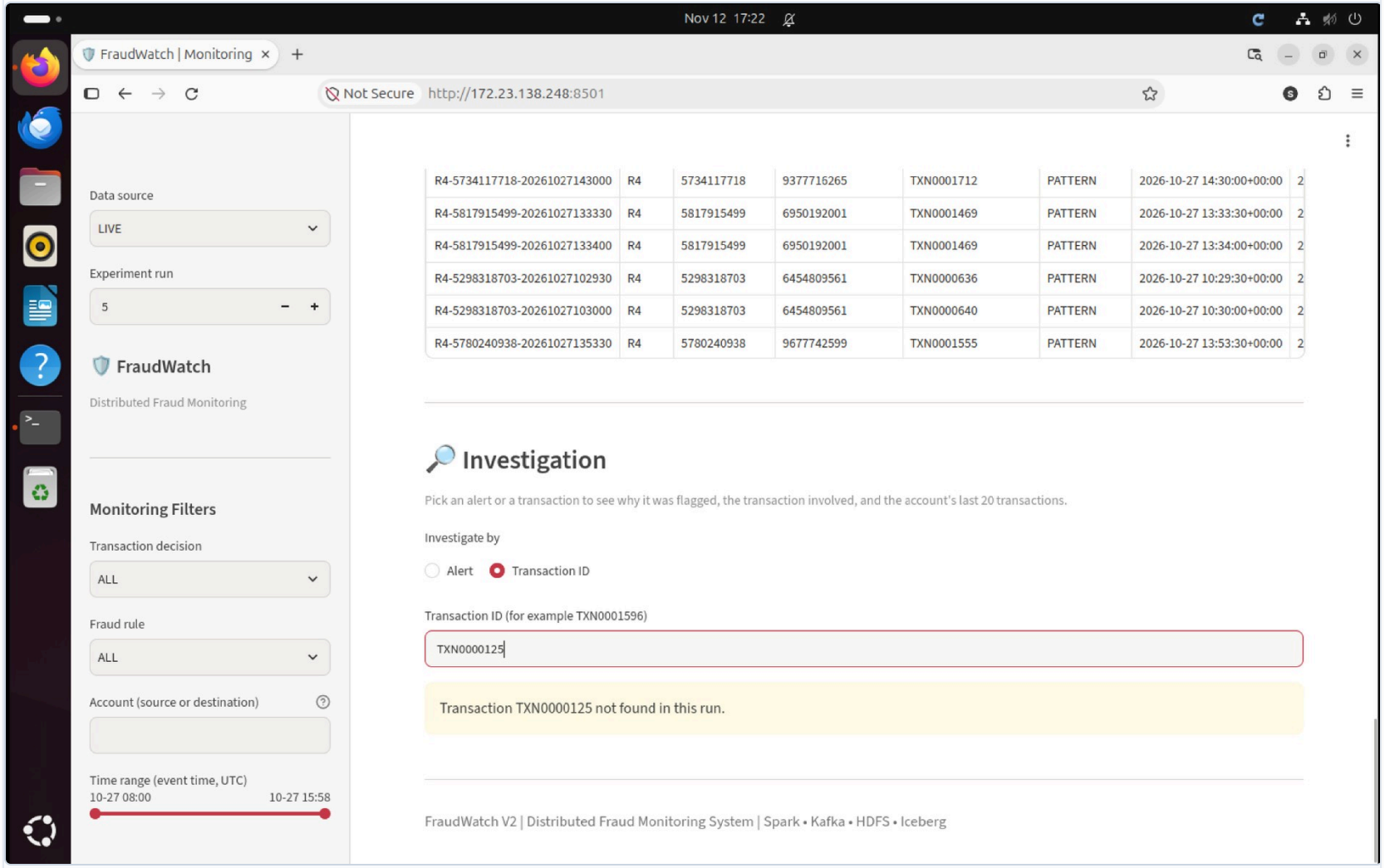
Experiment run 5: 1,983 transactions, 1,652 approved, 231 under review, 100 blocked. Rule counts are unchanged (R1 194, R2 203, R3 66, R4 98, R5 100) and the alert counts are unchanged (R2 90, R4 25, R3 14).

WHY IT MATTERS

The 7 missing decisions were all APPROVE decisions (1,659 to 1,652). No fraud alert and no rule count changed.

Investigation: a late event is not found

Fig 74 · Saatwik · VM1 (Kafka + producer)



WHAT YOU ARE LOOKING AT

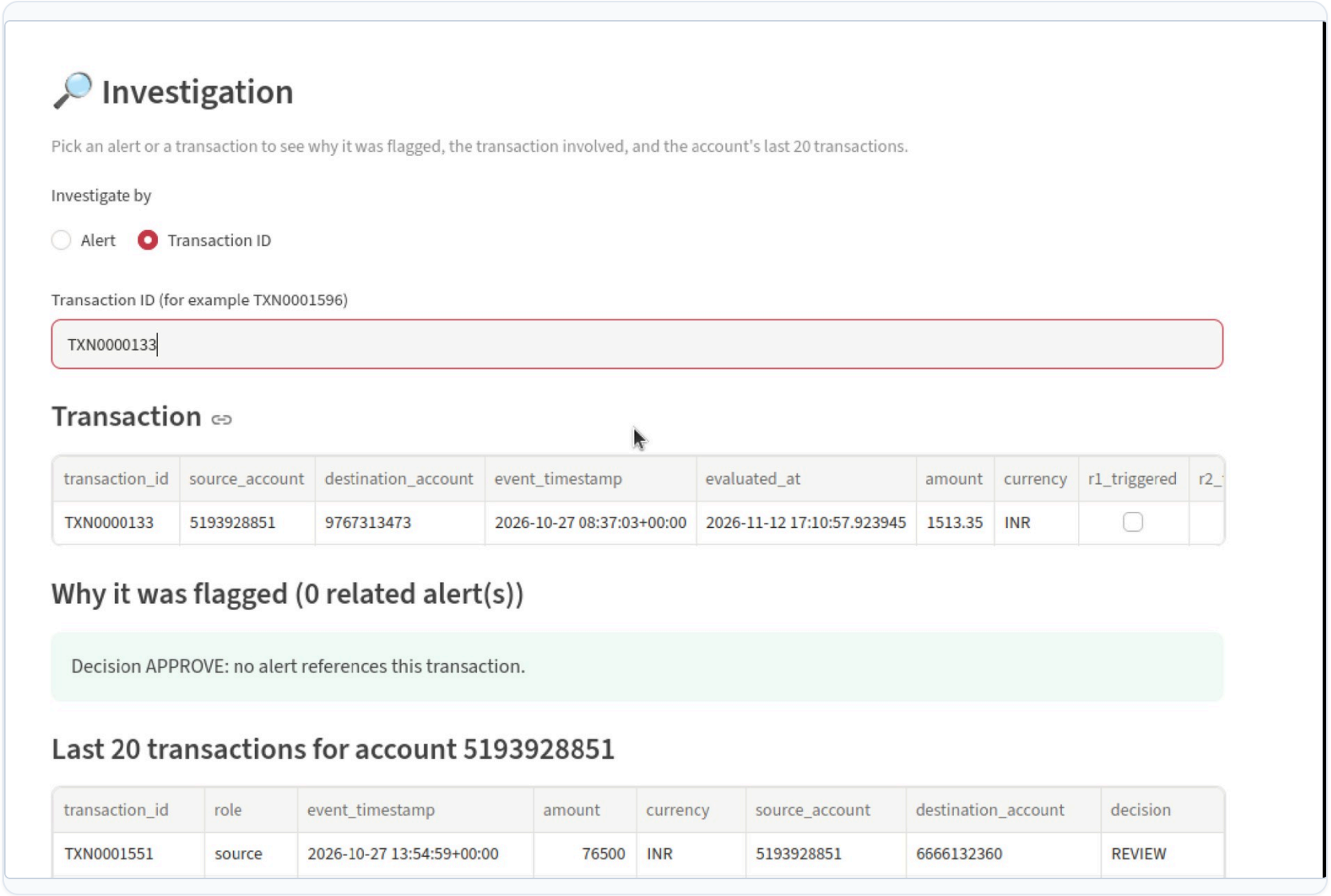
Investigation by Transaction ID for TXN0000125: Transaction TXN0000125 not found in this run.

WHY IT MATTERS

The dashboard shows a dropped event as not found. It cannot tell a dropped row from one that never arrived, so it has to be read with the producer log and the verifier report.

Investigation: a normal event is found (control)

Fig 75 · Saatwik · VM1 (Kafka + producer)



Investigation

Pick an alert or a transaction to see why it was flagged, the transaction involved, and the account's last 20 transactions.

Investigate by

○ Alert ◉ Transaction ID

Transaction ID (for example TXN0001596)

TXN0000133

Transaction

transaction_id	source_account	destination_account	event_timestamp	evaluated_at	amount	currency	r1_triggered	r2_
TXN0000133	5193928851	9767313473	2026-10-27 08:37:03+00:00	2026-11-12 17:10:57.923945	1513.35	INR	☐	

Why it was flagged (0 related alert(s))

Decision APPROVE: no alert references this transaction.

Last 20 transactions for account 5193928851

transaction_id	role	event_timestamp	amount	currency	source_account	destination_account	decision
TXN0001551	source	2026-10-27 13:54:59+00:00	76500	INR	5193928851	6666132360	REVIEW

WHAT YOU ARE LOOKING AT

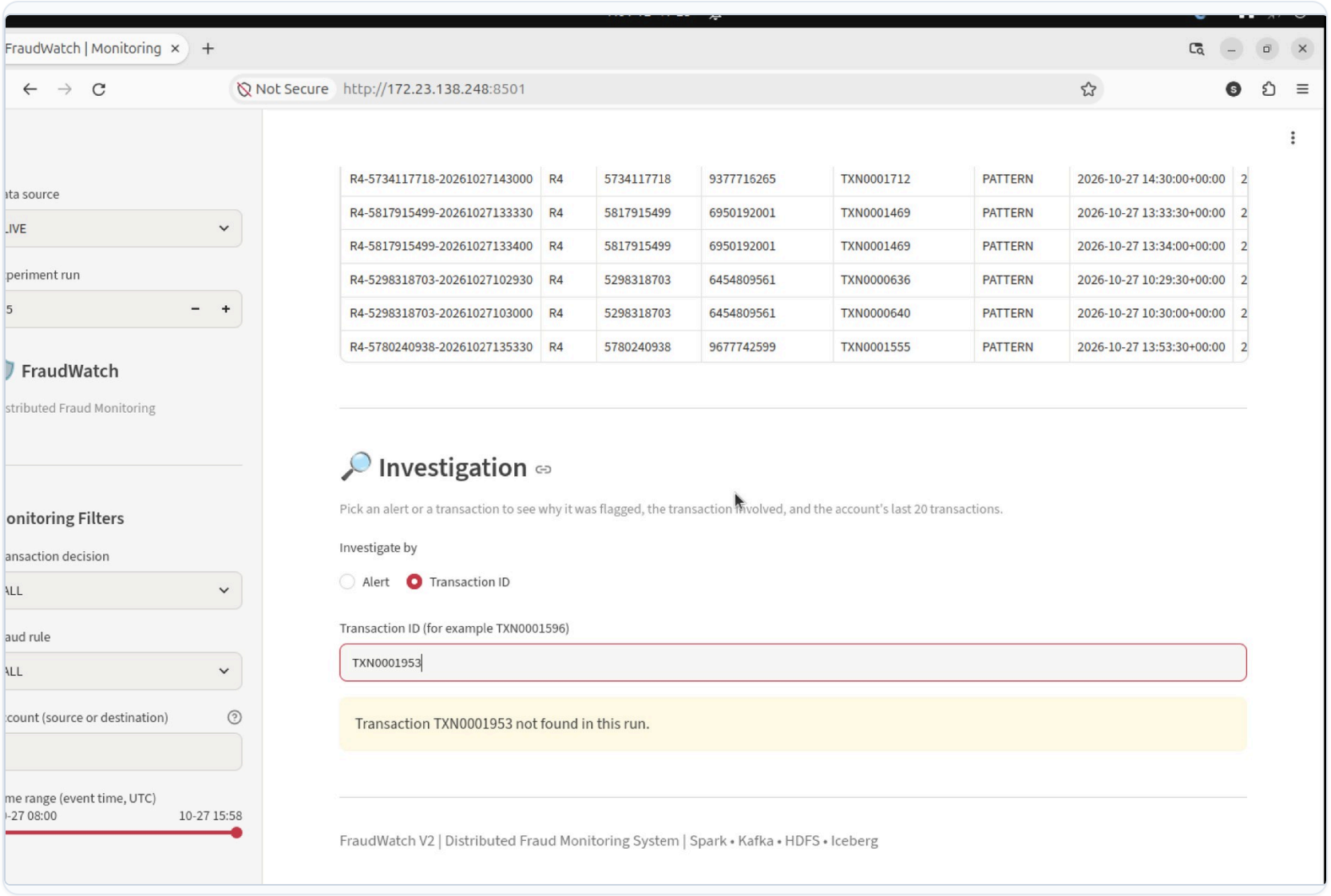
TXN0000133 was not late. It is found: source account 5193928851, amount 1513.35 INR, decision APPROVE, 0 related alerts, and the account's last 20 transactions are listed.

WHY IT MATTERS

The control. The lookup works, so the not found results are real.

Investigation: second late event is not found

Fig 76 · Saatwik · VM1 (Kafka + producer)



WHAT YOU ARE LOOKING AT

TXN0001953 (the last late id) is also not found in this run.

WHY IT MATTERS

Consistent with the verifier: late events beyond the watermark are dropped.

Findings, limits and what we would do next

WHAT THE EVIDENCE SHOWS

- **Correct baseline.** 1,990 decisions and 129 alerts equal the ground truth, 0 mismatches.
- **Executor loss is absorbed.** The job continued and finished correctly without manual action.
- **Full restart is safe.** After the master and driver were stopped mid-batch the job resumed from the checkpoint with no duplicates and no loss.
- **Lateness under 1 min is handled.** Seven events up to 48 s late changed nothing.
- **Lateness over 1 min loses data.** Seven events 93–129 s late were dropped. The verifier and the dashboard show it.

LIMITS WE ARE STATING OPENLY

- Micro-batches take 20–48 s against a 10 s trigger because every batch re-reads all stored decisions to re-evaluate the windows. Fine for 2,000 rows, it would not scale.
- Recovery from a master and driver failure is manual (about 2 min 48 s to resubmit). There is no standby master or automatic driver restart.
- Each experiment was run once. Timings are measurements from one run, not averages.
- The dashboard says not found for a dropped row and for a row that never arrived. It has to be read with the producer log and verifier report.
- Run 6 failed because of a timezone bug. It was fixed and is kept as evidence.

NEXT STEPS

- Process only new rows per batch, with keyed state, instead of re-evaluating the whole history.
- Run the driver in cluster mode with supervise, or add a standby master, to remove the manual step.
- Add a late-data side output so dropped events are counted and visible.

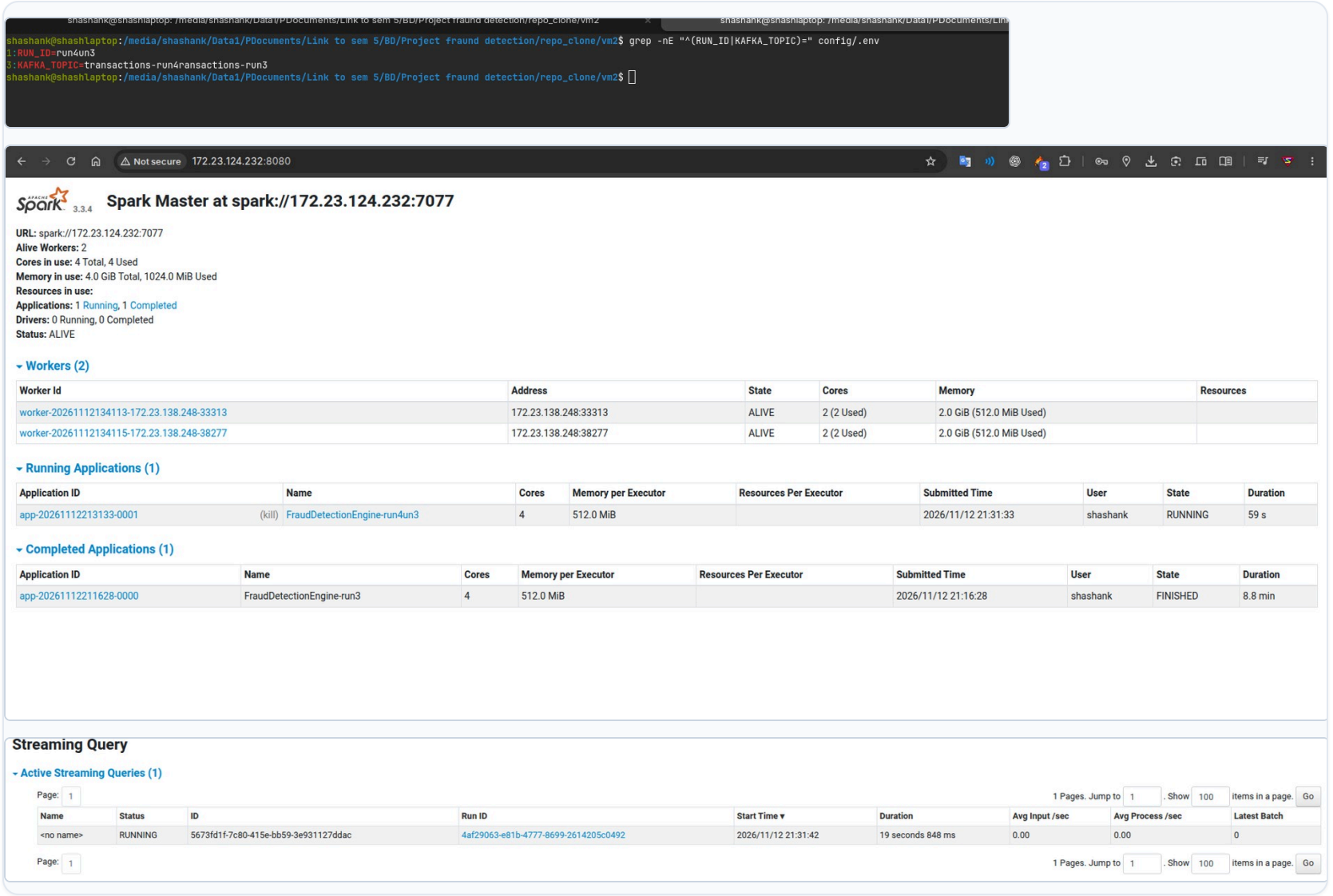
APPENDIX

Evidence archive and incident

All runs

Incident: mistyped run id (run4un3)

Fig 77 · Shashank · VM2 (Spark master + driver)



WHAT YOU ARE LOOKING AT

Top: grep of config/.env shows RUN_ID=run4un3 and KAFKA_TOPIC=transactions-run4ransactions-run3, a sed edit that mangled the values. Middle and bottom: the master page and streaming tab showing the wrongly named application FraudDetectionEngine-run4un3 running (19 s old) next to the finished run 3 app.

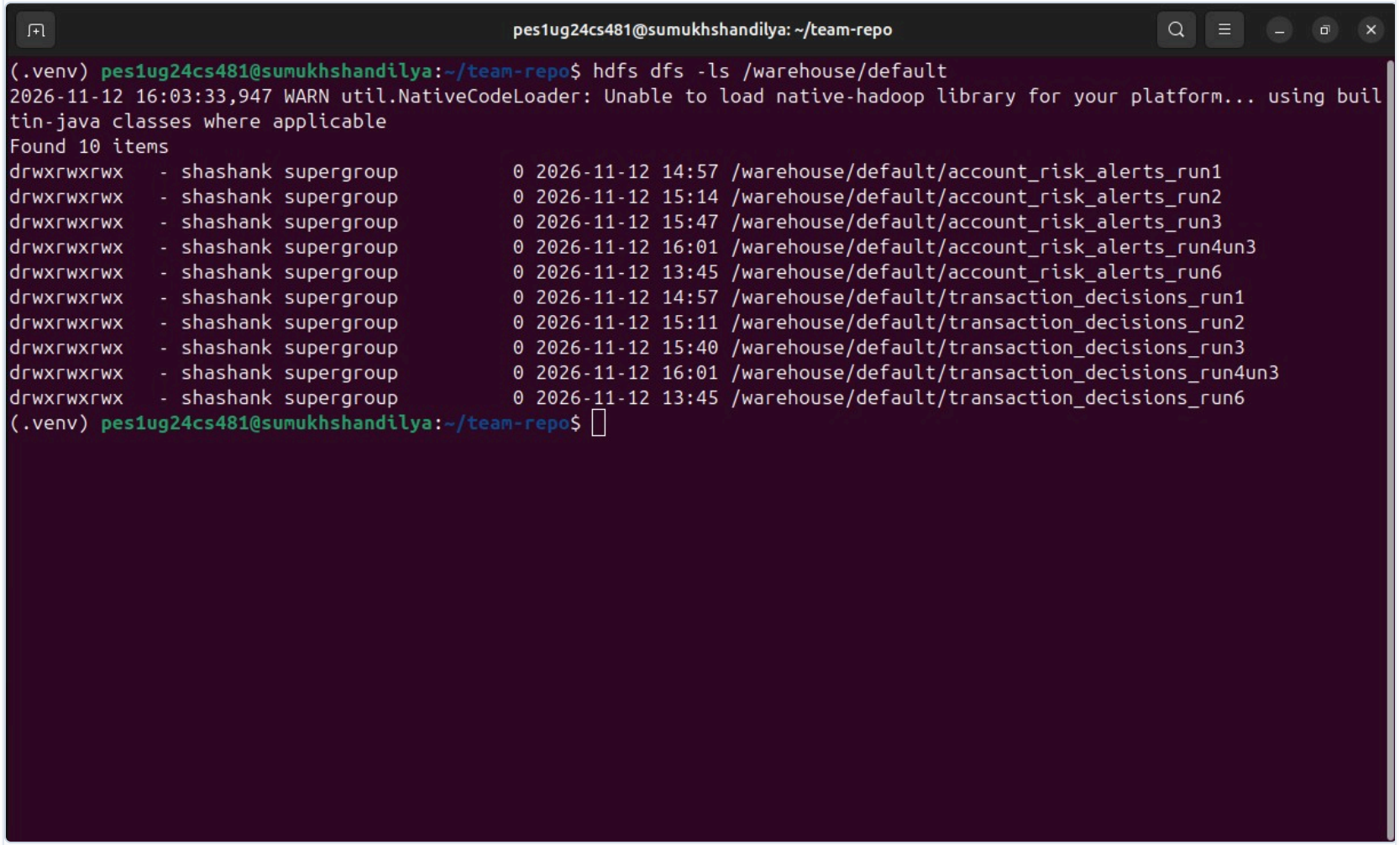
WHY IT MATTERS

A configuration mistake caught before any data was processed. The app was stopped, the stray HDFS paths (two tables and one checkpoint run4un3) were removed after checking, and run 4 was started again with correct values. It is not part of any result.

APPENDIX

Stray tables seen on HDFS

Fig 78 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)



WHAT YOU ARE LOOKING AT

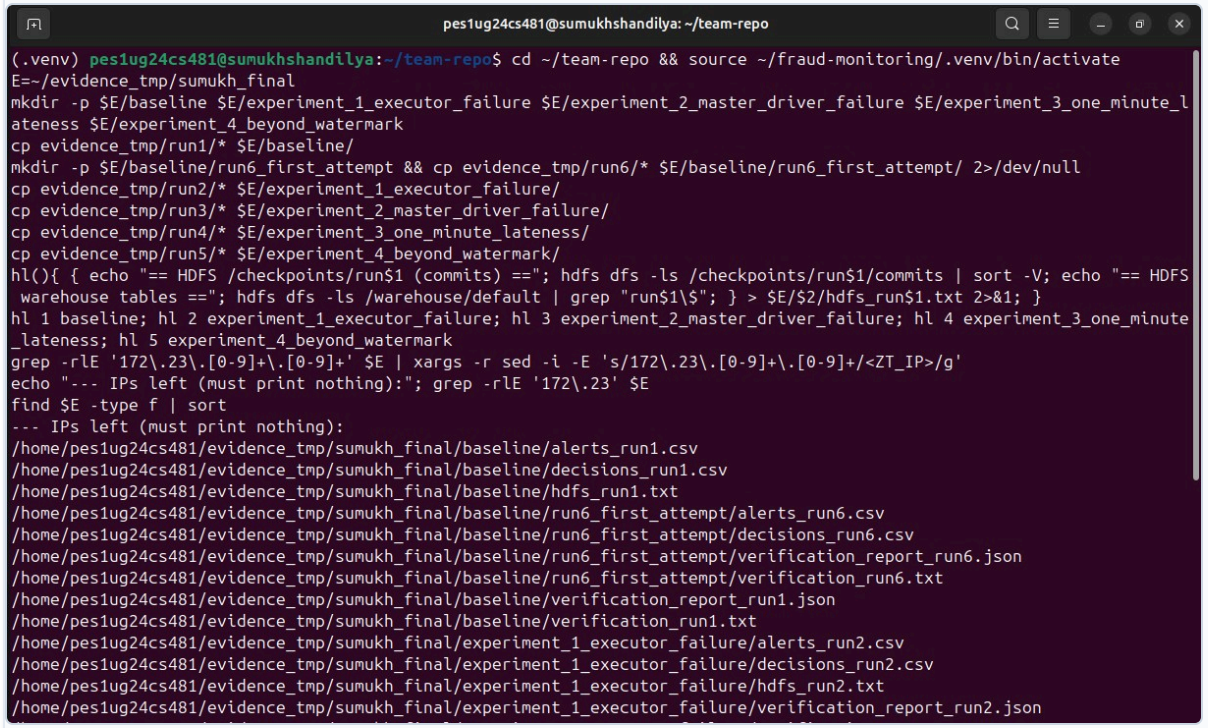
/warehouse/default listing includes transaction_decisions_run4un3 and account_risk_alerts_run4un3.

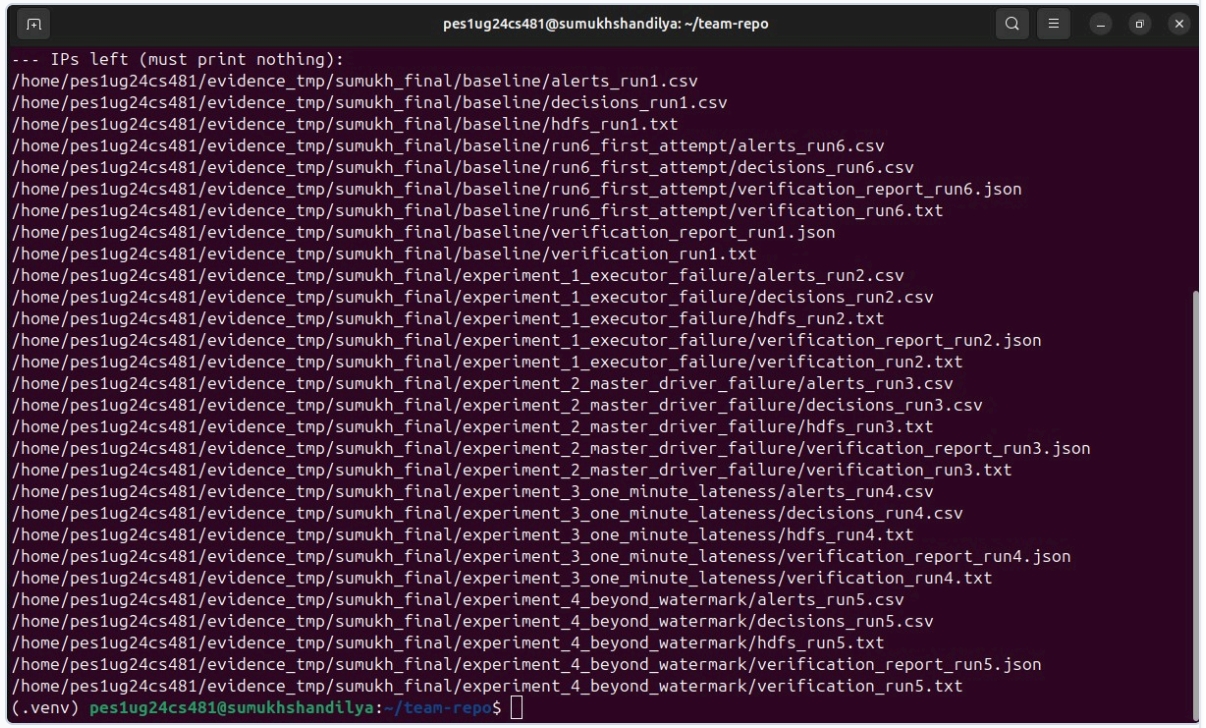
WHY IT MATTERS

These are the leftovers of the incident above. They were removed before run 4 was verified.

Evidence archive: Sumukh

Fig 79 · Sumukh · VM3/VM4 (HDFS, workers, dashboard, verifier)





WHAT YOU ARE LOOKING AT

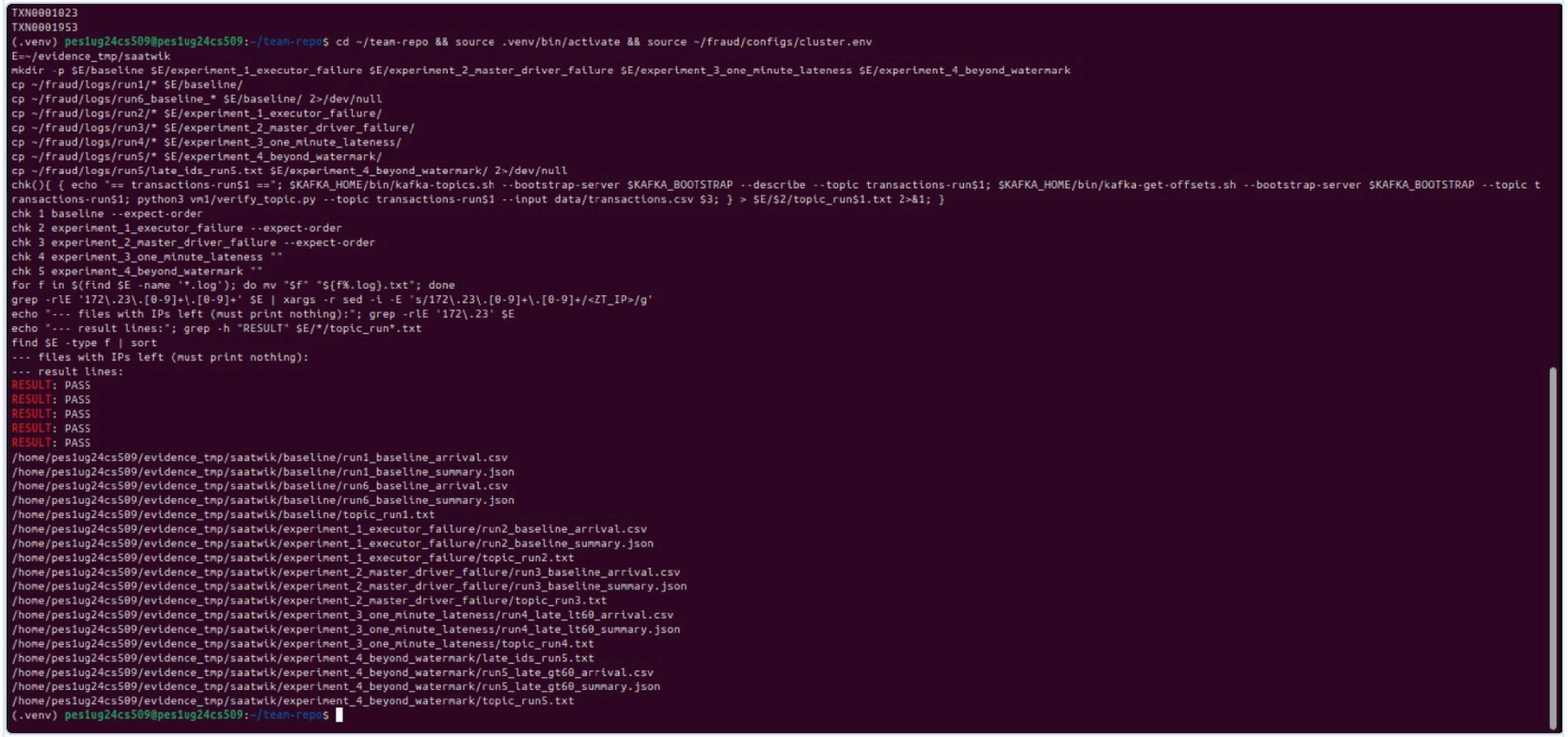
Evidence for runs 1 to 6 is copied into per-experiment folders: decisions and alerts CSV, verifier text and JSON, and HDFS listings. IP addresses are redacted and the check for leftover addresses prints nothing.

WHY IT MATTERS

Every claim in this document has a file behind it in the repository evidence folders.

Evidence archive: Saatwik

Fig 80 · Saatwik · VM1 (Kafka + producer)



WHAT YOU ARE LOOKING AT

Producer arrival logs and summaries, topic verification for runs 1 to 5 (all PASS), and the list of late ids for run 5 are copied per experiment.

WHY IT MATTERS

Producer-side proof for every run.

Evidence archive: Shashank (first staging)

Fig 81 · Shashank · VM2 (Spark master + driver)

```
shashank@shashlaptop:/media/shashank/Data1/PDocuments/Link to sem 5/BD/Project fraund
detection/repo_clone/vm2$ mkdir -p ~/evidence_tmp/shashank
for f in run1_driver_log run2_driver_log run3_driver_log run3_driver_log_resume run6_driver_log; do
  [ -f ~/$f.txt ] && sed -E 's/172\.23\.[0-9]+\.[0-9]+/<ZT_IP>/g' ~/$f.txt > ~/evidence_tmp/shashank/$f.txt
done
sed -E 's/172\.23\.[0-9]+\.[0-9]+/<ZT_IP>/g' config/.env > ~/evidence_tmp/shashank/env_run4_redacted.txt
ls -la ~/evidence_tmp/shashank
grep -rc '172\.23' ~/evidence_tmp/shashank
total 132
drwxrwxr-x 2 shashank shashank  4096 Nov 12 21:45 .
drwxrwxr-x 3 shashank shashank  4096 Nov 12 21:45 ..
-rw-rw-r-- 1 shashank shashank   501 Nov 12 21:45 env_run4_redacted.txt
-rw-rw-r-- 1 shashank shashank 16707 Nov 12 21:45 run1_driver_log.txt
-rw-rw-r-- 1 shashank shashank 43942 Nov 12 21:45 run2_driver_log.txt
-rw-rw-r-- 1 shashank shashank 18413 Nov 12 21:45 run3_driver_log_resume.txt
-rw-rw-r-- 1 shashank shashank 22495 Nov 12 21:45 run3_driver_log.txt
-rw-rw-r-- 1 shashank shashank  9230 Nov 12 21:45 run6_driver_log.txt
/home/shashank/evidence_tmp/shashank/env_run4_redacted.txt:0
/home/shashank/evidence_tmp/shashank/run6_driver_log.txt:0
/home/shashank/evidence_tmp/shashank/run2_driver_log.txt:0
/home/shashank/evidence_tmp/shashank/run3_driver_log.txt:0
/home/shashank/evidence_tmp/shashank/run1_driver_log.txt:0
/home/shashank/evidence_tmp/shashank/run3_driver_log_resume.txt:0
shashank@shashlaptop:/media/shashank/Data1/PDocuments/Link to sem 5/BD/Project fraund
detection/repo_clone/vm2$
```

WHAT YOU ARE LOOKING AT

Driver logs for runs 1, 2, 3 (before and after resume) and 6 and the run 4 environment copied with IP addresses redacted (0 matches remain).

WHY IT MATTERS

Spark-side proof. The final staging also includes runs 4 and 5.